

Universidad Central

Facultad de Ingeniería y Ciencias Básicas

Maestría en Analítica de Datos



Modelo de análisis de sentimiento y detección de PQRS en X (Twitter)
para la gestión de clientes del Grupo AVAL

Tesis de Maestría

María Angélica Moreno Cely

Nicolás Alejandro Fernández Espinosa

Trabajo de grado como requisito para optar el título de:

Magister

Bogotá, Colombia

2024

Universidad Central
Facultad de Ingeniería y Ciencias Básicas
Maestría en Analítica de Datos



Modelo de análisis de sentimiento y detección de PQRS en X (Twitter)
para la gestión de clientes del Grupo AVAL

Tesis de Maestría

María Angélica Moreno Cely

1.019.043.660

Nicolás Alejandro Fernández Espinosa

80.850.248

Director (a)

Miguel Hernández Bejarano

Bogotá, Colombia

2024

Aprobación

Director(a) de Tesis



Firma

Miguel Hernández Bejarano

PhD (c)

Codirector(a) de Tesis

Firma

[Nombres Completos]

[Títulos]

Jurados de la Tesis

Jurado

Firma

[Nombres Completos]

[Títulos]

Jurado

Firma

[Nombres Completos]

[Títulos]

Jurado

Firma

[Nombres Completos]

Agradecimientos

Expresamos el más sincero agradecimiento a todas las personas que han hecho posible la realización de este trabajo de grado. En primer lugar, como equipo académico, cuya colaboración, esfuerzo conjunto y compromiso han sido fundamentales para el desarrollo y culminación exitosa de este proyecto. La sinergia y el apoyo mutuo han sido pilares para enriquecer tanto los resultados como el proceso mismo. Asimismo, el más profundo agradecimiento a nuestra familia, cuyo apoyo incondicional ha sido una constante fuente de motivación y fortaleza.

Este trabajo es también un testimonio del poder del apoyo familiar y del trabajo en equipo, elementos sin los cuales este logro no hubiera sido posible. A todos ustedes, nuestra eterna gratitud.

Resumen

La gestión de relaciones con clientes en el sector financiero ha adquirido una importancia crucial debido a la creciente influencia de las redes sociales en la percepción pública de las marcas. Este estudio se enfoca en el desarrollo de un modelo de análisis de sentimiento y detección de PQRS en X (Twitter) para el Grupo AVAL, destacando la necesidad de gestionar eficazmente estas interacciones, como se menciona en los trabajos de (Mejía Monsalve & Salinas Rendón, 2015) y (Hurtado & Arias, 2022)

El problema central radica en la dificultad de las instituciones financieras para manejar y analizar grandes volúmenes de datos en redes sociales oportunamente. Este estudio aborda esta dificultad mediante la implementación de técnicas de procesamiento de lenguaje natural (NLP) y aprendizaje automático, con el objetivo de identificar tendencias y patrones en la retroalimentación de los usuarios del Grupo AVAL, mejorando así la gestión de clientes.

Para llevarlo a cabo, se sigue la metodología CRISP-DM, que abarca desde la recolección de datos hasta el despliegue y monitoreo del modelo. Se utilizan las técnicas mencionadas anteriormente con el apoyo de estudios previos que demuestran la efectividad en contextos similares (Ranjan & Bhatnagar, 2010) y (Kongar & Adebayo, 2021).

Los resultados muestran que los modelos desarrollados son capaces de clasificar sentimientos y detectar PQRS con alta precisión, contribuyendo significativamente a la estrategia de gestión de relaciones con los clientes del Grupo AVAL. En el modelo de categorización de BERT, ajustando validación cruzada con 10 épocas, se obtiene una precisión en el entrenamiento del 99,93%, una precisión en la validación del 97,87% y una pérdida en la validación de 0,3240, siendo especialmente efectivo en la clasificación de las 26 categorías.

En el análisis de sentimiento, se hace uso de un modelo BERT con validación cruzada y 10 épocas, obteniendo una precisión en el entrenamiento del 99,58%, una precisión en la validación del 96,23% y una pérdida en la validación de 0,7684.

Por ello, el estudio resalta el potencial transformador de la analítica de datos y la inteligencia artificial en la satisfacción del cliente y la gestión proactiva de las relaciones, alineándose con tendencias globales hacia la digitalización y la personalización del servicio al cliente (Acevedo, 2019) y (Planeación, 2019). Este modelo no solo mejora la eficiencia operativa, sino que también proporciona una herramienta valiosa para la toma de decisiones estratégicas en la gestión de clientes.

Palabras clave:

Análisis de sentimiento, aprendizaje automático, detección de PQRS, Grupo AVAL, procesamiento de lenguaje natural, redes sociales.

Abstract

The management of customer relationships in the financial sector has become crucial due to the growing influence of social media on public perception of brands. This study focuses on developing a sentiment analysis and PQRS detection model on X (Twitter) for Grupo AVAL, highlighting the need to effectively manage these interactions, as mentioned in the works of (Mejía Monsalve & Salinas Rendón, 2015) and (Hurtado & Arias, 2022).

The central problem lies in the difficulty financial institutions face in handling and analyzing large volumes of data on social media in real time. This study addresses this challenge by implementing natural language processing (NLP) and machine learning techniques to identify trends and patterns in user feedback for Grupo AVAL, thereby improving customer management.

To carry out the study, the CRISP-DM methodology is followed, encompassing data collection to model deployment and monitoring. The aforementioned techniques are used with the support of previous studies that demonstrate their effectiveness in similar contexts (Ranjan & Bhatnagar, 2010) and (Kongar & Adebayo, 2021).

The results show that the developed models can classify sentiments and detect PQRS with high accuracy, significantly contributing to Grupo AVAL's customer relationship management strategy. In the BERT categorization model, adjusting cross-validation with 10 epochs, it achieves a training accuracy of 99.93%, a validation accuracy of 97.87%, and a validation loss of 0.3240, being particularly effective in classifying the 26 categories.

For sentiment analysis, a BERT model with cross-validation and 10 epochs is used, obtaining a training accuracy of 99.58%, a validation accuracy of 96.23%, and a validation loss of 0.7684.

Therefore, the study highlights the transformative potential of data analytics and artificial intelligence in customer satisfaction and proactive relationship management, aligning with global trends toward digitalization and personalized customer service (Acevedo, 2019) and (Planeación, 2019) This model not only improves operational efficiency but also provides a valuable tool for strategic decision-making in customer management.

Keywords:

Sentiment analysis, machine learning, PQRS detection, Grupo AVAL, natural language processing, social media.

Contenido

Introducción	18
Planteamiento del Problema y Justificación.....	22
Causas del Problema	22
Manifestaciones del Problema	22
Contraste con el Conocimiento Existente	23
Importancia de Solucionar el Problema	23
Delimitación del Problema.....	23
Objetivo.....	25
Objetivos Específicos.....	25
Antecedentes	25
Metodología	34
Compresión del Negocio:.....	34
Entendimiento de los Datos:	35
Preparación de los Datos:	35
Modelado:.....	35
Evaluación:.....	36
Despliegue:.....	36
Monitorización:	36

Resultados	38
Entendimiento de los Datos.....	38
Fuente de los Datos.....	38
Cantidad de Datos.....	38
Distribución Temporal.....	39
Limpieza de los Datos	40
Normalización de Texto.....	40
Filtrado y Selección	42
Enriquecimiento de Datos	43
Transformación de Datos para Modelos.....	43
Conjuntos de Datos para Entrenamiento y Validación.....	44
Modelado	46
Discusión de los resultados.....	81
Rendimiento del Modelo.....	81
Sin Balance de Clases	81
Regularización L1 y L2.....	81
Evaluación de los Resultados Propios.....	82
Consistencia y Robustez del Modelo.....	82
Relevancia Práctica.....	82
Conclusiones	84
Recomendaciones	86

Referencias.....	88
Lista de anexos.....	93
Anexo I. Limpieza de texto.....	94
Anexo II. Modelo BERT - Balance - Validación Cruzada	97
Anexo III. Modelo RNN - Balance - Validación Cruzada.....	104
Anexo IV. Resultados de Modelo 3 – BERT - Sentimiento	124
Anexo V. Resultados de Modelo 3 – BERT – Sentimiento – Matrices y Curvas ROC	136
Anexo VI. Resultados de Modelo 3 – BERT – PQRS.....	142
Anexo VII. Resultados de Modelo 3 – BERT – PQRS – Matrices y Curvas ROC.....	154

Lista de figuras

Figura 1 Precio de mercado vs. Sentimiento del tweet (50 semillas principales) (Yang, Mo, & Zhu, 2014).	28
Figura 2 Imagen tomada de: https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/	34
Figura 3 Distribución temporal de tweets (2020 - 2023) (Elaboración propia, 2024).....	39
Figura 4 Ejemplos de Comunicaciones de Clientes sobre Problemas Bancarios en Twitter (Elaboración propia, 2024).	40
Figura 5 Casos de Servicio al Cliente: Textos Originales vs Normalizados (Elaboración propia, 2024).	41
Figura 6 Comparación de Textos: Textos Originales vs Normalizados (Elaboración propia, 2024).	41
Figura 7 Ejemplos de Textos Normalizados y sus Tokens Correspondientes (Elaboración propia, 2024).	43
Figura 11 Matriz de Confusión Modelo 3 - BERT - Sentimiento	74
Figura 12 ROC - BERT - Sentimiento.....	75
Figura 13 Matriz de Confusión Modelo 3 - BERT - PQRS.....	76
Figura 14 ROC - BERT - PQRS.....	77

Lista de tablas

Tabla 1 Regresión de las 50 semillas principales por rendimiento del mercado (Yang, Mo, & Zhu, 2014).	28
Tabla 2 Comparación de los modelos en precisión y tasas de pérdida (Huang, Hou, Chen, Yi, & Hu, 2023).	32
Tabla 3 Distribución de tweets por Banco (Elaboración propia, 2024).	38
Tabla 4 Comparación de distribuciones de Sentimiento antes y después de aplicar SMOTE (Elaboración propia, 2024).	45
Tabla 5 Comparación de Distribuciones de Categorías originales, antes y después de aplicar SMOTE (Elaboración propia, 2024).	45
Tabla 6 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	53
Tabla 7 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	54
Tabla 8 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	56
Tabla 9 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	56
Tabla 10 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	57
Tabla 11 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	58
Tabla 12 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	59
Tabla 13 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	60
Tabla 14 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	61
Tabla 15 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	63

Tabla 16 Resumen de Desempeño del Modelo (Elaboración propia, 2024).	63
Tabla 17 Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).	65
Tabla 18 Comparación de los modelos BERT por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	65
Tabla 19 Comparación de los modelos representativos BERT en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	66
Tabla 20 Comparación de los modelos RNN por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	67
Tabla 21 Comparación de los modelos representativos RNN en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	67
Tabla 22 Comparación de los modelos BERT por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	68
Tabla 23 Comparación de los modelos representativos BERT en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	69
Tabla 24 Comparación de los modelos RNN por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).	71
Tabla 25 Comparación de los modelos representativos RNN en Métricas de desempeño vs mejores clases (Elaboración propia, 2024).	72

Lista de siglas y abreviaturas

Sigla	Inglés (Descripción)	Español (Descripción)
AI	<i>Artificial Intelligence</i>	Inteligencia Artificial
AUC	<i>Area Under Curve</i>	Área bajo la Curva
BERT	<i>Bidirectional Encoder Representation from Transformers</i>	Representación Bidireccional del Codificador de Transformadores
BiLSTM	<i>Bidirectional Long Short-Term Memory</i>	Memoria a Largo Corto Plazo Bidireccional
CONPES	<i>National Council for Economic and Social Policy</i>	Consejo Nacional de Política Económica y Social
CRISP-DM	<i>Cross-Industry Standard Process for Data Mining</i>	Proceso Estándar Intersectorial para la Minería de Datos
DNP	<i>National Planning Department</i>	Departamento Nacional de Planeación
NLP	<i>Natural Language Processing</i>	Procesamiento del Lenguaje Natural
PQRS	<i>Requests, Complaints, Claims, and Suggestions</i>	Peticiones, Quejas, Reclamos y Sugerencias
RNN	<i>Recurrent Neural Network</i>	Red Neuronal Recurrente
SMOTE	<i>Synthetic Minority Over-sampling Technique</i>	Técnica de sobremuestreo de minoría sintética
SVM	<i>Support Vector Machines</i>	Máquinas de Soporte Vectorial

Glosario

- AI (*Artificial Intelligence*) término que describe la capacidad de las máquinas para imitar y realizar tareas inteligentes similares a las realizadas por los seres humanos.
- AUC (*Area Under Curve*) métrica para evaluar la capacidad de un modelo de clasificación para distinguir entre clases.

- BERT (*Bidirectional Encoder Representation from Transformers*) técnica avanzada de procesamiento del lenguaje natural que mejora la comprensión del contexto.
- BiLSTM (*Long Short-Term Memory Bidireccional*) es una variante de las redes neuronales que puede aprender dependencias de largo plazo de ambos lados de una secuencia de datos.
- CONPES (Consejo Nacional de Política Económica y Social) es un órgano asesor del Gobierno en todos los aspectos relacionados con el desarrollo económico y social del país.
- CRISP-DM (*Cross-Industry Standard Process for Data Mining*) método probado para orientar los proyectos de minería de datos.
- DNP (Departamento Nacional de Planeación) es una entidad gubernamental encargada de dirigir el proceso de planeación en Colombia.
- NLP (Procesamiento del Lenguaje Natural) se refiere a las técnicas informáticas que permiten a las máquinas entender, interpretar y generar lenguaje humano.
- PQRS (Peticiones, Quejas, Reclamos y Sugerencias) es un mecanismo utilizado por las organizaciones para gestionar y responder a las inquietudes y solicitudes de sus clientes o usuarios.
- RNN (Red Neuronal Recurrente) modelo de aprendizaje profundo que procesa y transforma datos secuenciales de entrada en datos secuenciales de salida específicos.
- SMOTE (Técnica de sobremuestreo de minoría sintética) técnica para preprocesamiento de datos desbalanceados, basada en generar de forma sintética nuevos elementos de la clase minoritaria usando como referencia los elementos de dicha clase ya presentes en el conjunto de datos.

- SVM (Máquinas de Soporte Vectorial) es un modelo de aprendizaje supervisado utilizado para clasificación y regresión.

Introducción

Las redes sociales han revolucionado la forma en que las personas se comunican, comparten información y establecen conexiones en todo el mundo (Cobe, 2023). Estas plataformas digitales se han convertido en una parte fundamental de la vida cotidiana de miles de millones de personas, y su influencia en la sociedad es innegable. Con la proliferación de Internet y los avances tecnológicos, las redes sociales han evolucionado de manera exponencial, transformándose en herramientas poderosas que influyen en diversos aspectos de la vida moderna (Ridge, 2023).

La omnipresencia de las redes ha derribado las barreras geográficas, brindando la posibilidad de comunicación oportuna y un acceso inmediato a la información. No obstante, su impacto va más allá del ámbito personal. En el mundo empresarial, las redes sociales desempeñan un papel crucial, ya que se utilizan como herramientas estratégicas para establecer una presencia sólida y alcanzar audiencias más amplias. El buen uso permite establecer una fuerte presencia en el mercado, interactuar con clientes y mantener una buena reputación empresarial, al tiempo que impulsan la innovación y la transformación en el ámbito financiero (Blasco Fontecilla, 2021).

En este contexto, el sistema financiero forma una parte integral de la economía de un país, permitiendo desarrollar actividades financieras en empresas de diferentes tamaños y tipos, así como en familias y gobiernos nacionales e internacionales (Torres Gómez, 2015). A su vez, el sector bancario está regulado en comparación con otras industrias (Shi, Duy Nguyen, & Wang, 2023). En Colombia, la Superintendencia Financiera de Colombia tiene el objetivo de supervisar el sistema financiero para preservar su estabilidad, seguridad y confianza entre los usuarios

financieros, inversionistas, ahorradores y asegurados (Superintendencia Financiera de Colombia, 2023).

Además, los usuarios del sector financiero pueden manifestar la inconformidad respecto de un producto o servicio adquirido, ofrecido o prestado por una entidad financiera, ante entidades como el defensor del consumidor financiero, la Superintendencia Financiera de Colombia o cualquier institución competente, según corresponda (Superintendencia Financiera de Colombia, 2023). Mediante el sistema de PQRS, se permite a las empresas atender las necesidades y expectativas de sus clientes, así como mejorar la calidad de sus productos o servicios.

En este sentido, se plantea realizar este estudio, que tiene como objetivo explorar la gestión de PQRS de los clientes del Grupo AVAL. Siendo esta la organización financiera más grande de Colombia, líder en el negocio de banca tradicional a través de sus cuatro bancos (Banco de Bogotá, Banco de Occidente, Banco Popular y Banco AV Villas). Además, el Sistema de PQRS es una herramienta que le permite al Grupo Aval, conocer las inquietudes y manifestaciones de los clientes a fin de fortalecer sus servicios (Grupo AVAL, 2023).

En Colombia, esta tendencia a expresar opiniones y experiencias no pasa desapercibida, especialmente por parte de los usuarios que adquieren y/o usan productos y servicios en el sector bancario. Esto es relevante para el sector, ya que busca recuperar la confianza de sus clientes (Mejía Monsalve & Salinas Rendón, 2015). Así pues, el sistema de atención de PQRS es de gran importancia para las organizaciones, ya que permite conocer los inconvenientes e inconformidades de los clientes y usuarios (Hurtado & Arias, 2022).

Es así como surge la pregunta de investigación: ¿Cómo puede un modelo de aprendizaje basado en análisis de sentimientos y detección de PQRS en X (Twitter) optimizar la gestión de clientes y contribuir a la satisfacción y fidelización de los usuarios del Grupo AVAL?

A continuación, se presentan las cifras que subrayan la importancia de evaluar y abordar las inquietudes de los usuarios en el contexto bancario. En 2022, se registraron un total de 1.262 millones de quejas recibidas de los principales bancos, lo cual está directamente relacionado con la cantidad de usuarios que tienen estas entidades (Salazar Castellanos, 2023). En particular, el Grupo AVAL muestra la siguiente distribución de quejas entre sus bancos: Popular se encuentra en la cuarta posición con 97.143 quejas, Occidente en la quinta posición con 86.150 quejas, Bogotá en el séptimo lugar con 49.596 y Av Villas en un onceavo lugar con 29.879 quejas (Superintendencia Financiera de Colombia, 2023). Esto subraya la importancia de evaluar y abordar las inquietudes de los usuarios en el contexto bancario.

Además, en la sociedad actual, las redes sociales se han convertido en un aspecto integral de la vida diaria para comunicarse y expresar inquietudes. Un número cada vez mayor de personas hace uso de estas plataformas, como X (Twitter), Facebook, Instagram, entre otros, para expresar sus emociones y puntos de vista, generando grandes volúmenes de datos (An & Wan Zainon, 2023). Asimismo, el análisis de sentimiento (positivo, neutro y negativo), también conocido como Minería de Opinión, se ha convertido en una herramienta o método para extraer y analizar las opiniones, tonos emocionales, actitudes, percepciones y más, en relación con diversas entidades como temas, productos y servicios (Birjali, Kasri, & Beni-Hssane, 2021). Este campo ha sido objeto de investigación en el aprendizaje automático y el procesamiento del lenguaje natural durante muchos años, con el propósito de comprender e interpretar el

sentimiento humano a través de diferentes modalidades de texto, tonos de voz o expresiones faciales (Zhu, Zhu, Zhang, Xu, & Kong, 2023).

Por tanto, se propone, fundamentado en la mencionada técnica, el desarrollo de un modelo de aprendizaje que habilite al Grupo AVAL para optimizar la gestión y respuesta de las PQRS de forma eficaz. Este enfoque tiene como objetivos mejorar la toma de decisiones, la satisfacción del cliente y la gestión de la reputación de la marca. Para el caso de estudio, se usan fuentes primarias de información a fin de garantizar un análisis riguroso y detallado.

Específicamente, los datos y testimonios fueron recolectados directamente de la red social X (Twitter), donde varios usuarios comparten experiencias y opiniones relevantes al objeto de investigación. Asimismo, se obtuvo información del Grupo Aval en Colombia, reconocida entidad del ámbito financiero y empresarial nacional. Para corroborar estas fuentes, se accede al sitio web oficial de la red social X (<https://developer.twitter.com/>) y al portal institucional de Grupo Aval (Grupo AVAL, 2023).

Dada la relevancia y complejidad de la gestión de PQRS en organizaciones de gran envergadura como el Grupo AVAL, la aplicación de un modelo de aprendizaje se presenta como una estrategia clave para potenciar su eficiencia y efectividad. Como respaldo a esta propuesta, diversos estudios han destacado la eficacia de los modelos de aprendizaje en la optimización de procesos empresariales y la mejora de la satisfacción del cliente. Por ejemplo, se han desarrollado investigaciones sobre la implementación de técnicas de aprendizaje en empresas del sector financiero, en busca de una mayor agilidad en la toma de decisiones y a una significativa elevación de la satisfacción del cliente (Ranjan & Bhatnagar, 2010). Además, la adopción de modelos de aprendizaje en la gestión de PQRS puede tener un impacto positivo en la reputación

de la marca, consolidando así la importancia de esta aproximación en el contexto del Grupo AVAL (Kongar & Adebayo, 2021).

Planteamiento del Problema y Justificación

En la era digital, las redes sociales como X (Twitter) se han convertido en plataformas clave para la expresión de opiniones y experiencias por parte de los usuarios, especialmente en sectores sensibles y regulados como lo es el financiero. El Grupo AVAL, siendo la mayor organización financiera en Colombia, enfrenta el reto de gestionar eficazmente las PQRS de sus clientes a través de estas plataformas digitales, debido al volumen masivo de datos generados, la diversidad de expresiones y sentimientos, y la necesidad de interpretar correctamente estas manifestaciones para tomar acciones oportunas y acertadas.

Causas del Problema

- **Volumen y velocidad de datos:** La cantidad de información generada en las redes sociales supera la capacidad de análisis manual, lo que dificulta la detección y gestión oportuna de PQRS.
- **Diversidad de expresiones:** La variabilidad en la forma en que los usuarios se expresan, complica la clasificación y el análisis de la percepción de las emociones.
- **Interpretación de sentimientos:** La complejidad inherente al lenguaje natural, incluyendo sarcasmo, regionalismos y jerga, plantea retos significativos para el entendimiento automático de sensaciones.

Manifestaciones del Problema

- **Insatisfacción del Cliente:** La gestión ineficiente de PQRS genera una percepción negativa del servicio, afectando la satisfacción y fidelización del cliente.

- Impacto en la Reputación: Fallos en la atención a las PQRS dañan la reputación del Grupo AVAL, aspecto crucial en el sector financiero.
- Retos Operacionales: La falta de un sistema eficiente para el análisis de sentimientos y detección de PQRS aumenta la carga operacional y reduce la eficiencia en la gestión de clientes.

Contraste con el Conocimiento Existente

A pesar de las prácticas de análisis de sentimiento y NLP, existe un vacío en la aplicación específica y adaptada de estas tecnologías para detectar y gestionar las PQRS en el contexto particular del sector financiero colombiano, más concretamente, en el Grupo AVAL. La mayoría de los modelos existentes no abordan de manera integral las peculiaridades lingüísticas y contextuales presentes en las comunicaciones de los colombianos en las redes sociales, especialmente en X (Twitter).

Importancia de Solucionar el Problema

A partir de los resultados obtenidos, resolver este problema espera permitir que el Grupo AVAL mejore significativamente la satisfacción y fidelización de sus clientes mediante una gestión proactiva y eficaz de PQRS. Además, optimizar los procesos internos, reduciendo tiempos de respuesta y mejorando la calidad del servicio. En un plano más amplio, contribuir a la innovación tecnológica y al avance en la aplicación de técnicas de NLP y aprendizaje automático en el sector financiero.

Delimitación del Problema

Este estudio se centra en la creación de un modelo de aprendizaje automático y NLP para analizar sentimientos y detectar PQRS específicamente en tweets relacionados con el Grupo

AVAL. Se delimita con el análisis de los textos en español, considerando las particularidades lingüísticas y culturales de los colombianos, y se enfoca en la eficacia de la herramienta en términos de precisión, sensibilidad y capacidad de respuesta ante las dinámicas de comunicación en X (Twitter).

Objetivo

Desarrollar un modelo de análisis de sentimientos y detección de PQRS en la plataforma X (Twitter), con el propósito de fortalecer la gestión de clientes del Grupo AVAL.

Objetivos Específicos

- Recopilar y preparar un conjunto de datos de Twitter relacionados con instituciones bancarias y servicios financieros, que incluya comentarios de clientes expresando sus opiniones y experiencias.
- Clasificar el sentimiento (tono emocional positivo, neutro y negativo) y detectar los PQRS, aplicando técnicas de preprocesamiento de texto para depurar y normalizar los datos recopilados, eliminando ruido y redundancias.
- Implementar y evaluar el sistema de detección de PQRS que identifique y categorice preguntas, quejas, reclamos y sugerencias en los comentarios de los clientes, y adicional la disposición afectiva.

Antecedentes

Para examinar la relación entre el estado de ánimo expresado en X (Twitter) y el comportamiento del mercado de valores nos respaldamos en el artículo “*Twitter Mood Predicts the Stock Market*”, donde los autores se basan en la idea de que las redes sociales reflejan el sentimiento colectivo de la sociedad y que este sentimiento puede influir en las decisiones financieras. Recopilan una gran cantidad de datos de Twitter y analizan el contenido de los tweets en busca de palabras clave relacionadas con el estado de ánimo, aplicando técnicas de NLP para determinar el sentimiento expresado en los tweets. En los resultados del estudio, se encontró que el estado de ánimo positivo en Twitter se correlaciona con aumentos en los precios de las acciones, mientras que el estado de ánimo negativo se asocia con disminuciones en los precios; la correlación se observó con una precisión evaluada en términos del Error Porcentual Absoluto Medio (MAPE, por sus siglas en inglés) entre el 1,79% y 2,13% y por la precisión de la dirección (subida o bajada) con un valor máximo de 86,7% durante el período de prueba, lo que indica que el análisis de sentimiento en las redes sociales puede tener valor predictivo en el contexto financiero (Johan, Huina, & Xiaojun, 2011).

Así mismo, con el estudio “*Twitter Sentiment Analysis Applied to Finance: A Case Study in the Retail Industry*” que se centra en aplicar el análisis de sentimiento en X (Twitter) al sector financiero en la industria minorista, apoya a comprender cómo los datos pueden utilizarse para evaluar el comportamiento de las acciones en el mercado de valores considerando sólo los extremos de polaridad (categorías positiva y negativa). Llevan a cabo la recopilación de una gran cantidad de datos implicando la obtención de tweets identificando palabras clave relevantes para la industria. Aplicaron técnicas de NLP para determinar el sentimiento expresado en los tweets, realizaron la construcción de modelos predictivos con técnicas de aprendizaje automático y análisis estadístico. La evaluación se realizó en función de su capacidad para predecir con

precisión los movimientos reales de las acciones de las empresas minoristas en el mercado de valores complementando con análisis comparativos enfocados en la precisión de las predicciones (Pinto Souza, Kolchyna, Treleaven, & Aste, 2015).

En esa misma línea, en “*Twitter sentiment and stock market: a COVID-19 analysis*” se utiliza un análisis textual de 1.616.007 tweets de enero a junio de 2021 de siete países para examinar cómo los sentimientos en redes sociales afectan los mercados de valores durante la pandemia de COVID-19. Empleando el analizador VADER para generar medidas de sentimiento positivo y negativo, se encontró que un mayor positivismo se asocia con un incremento a corto plazo en los precios de las acciones, mientras que el negativismo tiene un efecto inverso y a largo plazo en los precios en países de habla inglesa. Las conclusiones subrayan la importancia de considerar el sentimiento de las redes sociales en la toma de decisiones de política económica, especialmente durante períodos de crisis, y sugieren que tanto investigadores como inversores y responsables políticos podrían beneficiarse de estas métricas para mejorar la resiliencia económica y estabilizar los mercados financieros (Katsafados, Nikoloutsopoulos, & Leledakis, 2023).

Además, para evaluar la estructura de la red de la comunidad financiera en Twitter y analizar cómo esta red influye en la difusión de información financiera en la plataforma, mediante la investigación “*An empirical study of the financial community network on Twitter*” se identifican datos de la red social en mención, incluyendo tweets, perfiles de usuarios, y conversaciones o hilos relacionados en la estructura de la red. Aunque el estudio es más empírico, más allá de aplicar modelos estadísticos, hubo una proporción de información valiosa sobre cómo se organiza y opera la comunidad financiera en X (Twitter), así como quiénes son los actores clave en la difusión de información financiera en la plataforma. El análisis contribuyó a

una comprensión más profunda de la interacción entre la comunidad financiera y las redes sociales, utilizando medidas de centralidad de red para identificar influenciadores clave, documentando que el sentimiento negativo tiene un efecto más significativo en los retornos de mercado que el positivo, ya que como se observa en la Tabla 1, la regresión para el sentimiento de las 50 semillas principales de muestra, indica que el sentimiento negativo obtuvo un coeficiente de $-1,61\text{e-}05$ con un p-value de 0,0014, mostrando una fuerte significancia estadística en la predicción de los retornos del mercado, lo que puede ser relevante para la toma de decisiones en este ámbito financiero (Yang, Mo, & Zhu, An empirical study of the financial community network on Twitter, 2014).

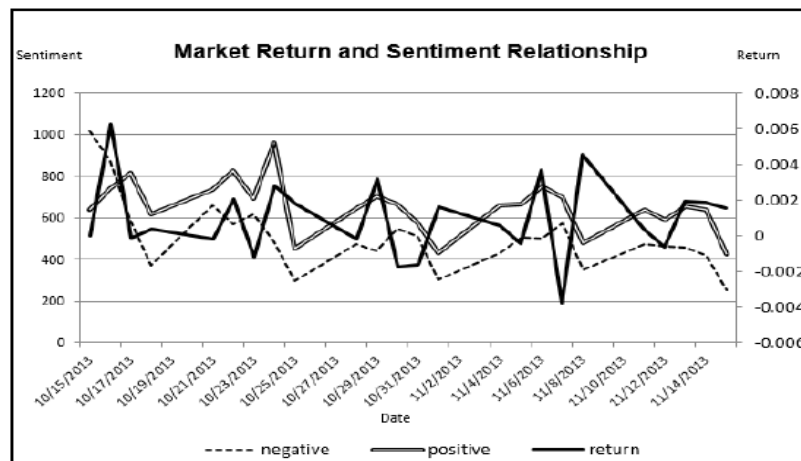
Tabla 1

Regresión de las 50 semillas principales por rendimiento del mercado (Yang, Mo, & Zhu, 2014).

Regression	Tweet Message (Top50 Seeds)	
Market Return	Coefficient	Prob.
Intercept	0,0022	0,1724
Return	-0,3326	0,0241**
Negative	-1,61e-05	0,0014***
Positive	9,76e-06	0,0096***

Figura 1

Precio de mercado vs. Sentimiento del tweet (50 semillas principales) (Yang, Mo, & Zhu, 2014).



En esta búsqueda de un impacto significativo en los mercados financieros y liderar movimientos en los precios de activos “*When Can Social Media Lead Financial Markets?*” investiga cuándo y cómo las redes sociales, en particular Twitter, lo pueden tener. Los autores evalúan la capacidad para predecir tendencias financieras basadas en los datos de sentimiento. Se utilizaron métodos como regresión logística, máquinas de soporte vectorial (SVM), y redes neuronales, entre otros. Los modelos de análisis de sentimiento y aprendizaje automático en el contexto financiero resaltan los sentimientos en Twitter sobre acciones y futuros comportamientos del índice *Standard & Poor's* 500, que contienen información significativa para predecir sus rendimientos en algunos horarios (Zheludev, Smith, & Aste, 2014).

En relación a aplicaciones bancarias, el estudio “*What Improves Customer Satisfaction in Mobile Banking Apps? An Application of Text Mining Analysis*” utiliza la minería de textos para analizar 96.140 reseñas de estas aplicaciones móviles de cuatro importantes bancos de EE. UU., identificando aspectos clave como la facilidad de uso y la seguridad como determinantes de la satisfacción del cliente. Los hallazgos sugieren que las valoraciones positivas están fuertemente vinculadas a la seguridad y funcionalidad conveniente de las aplicaciones, mientras que los comentarios negativos sobre la inseguridad y una mala experiencia de usuario tienen un efecto adverso en la valoración de las aplicaciones. La seguridad emerge como el elemento más crítico para la satisfacción del cliente en el ámbito de los servicios financieros móviles (Oh & Kim, 2021).

Por otro lado, para presentar la importancia en la detección automática de quejas financieras, el artículo “*Complaint and Severity Identification From Online Financial Content*” lo abarca planteado cómo puede beneficiar a empresas y comerciantes en línea, permitiendo monitorear y abordar problemas de manera más eficiente, direccionando a los equipos

adecuados. Menciona que las quejas se clasifican en categorías de gravedad según el nivel de riesgo que el denunciante está dispuesto a aceptar. Además, se reconoce que las emociones juegan un papel importante en la expresión de las quejas. Para tratar este tema, los autores presentan FINCORP, un recurso que recopila quejas y comentarios relacionados con instituciones financieras en Twitter. Este conjunto de datos se enriquece con información sobre la gravedad de las quejas, así como los sentimientos y emociones asociados. El corpus de datos y el código relacionado están disponibles en línea para su acceso y uso, buscando promover la transparencia y la responsabilidad en relación con productos y servicios financieros, lo que a su vez fortalece la reputación de la empresa (Singh, Bhatia, & Saha, 2024).

Por la misma línea de técnicas de minería de texto y análisis de sentimiento, en el artículo *“Determining banking service attributes from online reviews: text mining and sentiment analysis”* utiliza un conjunto de datos para analizar revisiones en línea relacionadas con servicios bancarios. Permitiendo identificar los atributos específicos mencionados en las revisiones y comprender la actitud de los usuarios hacia los servicios bancarios, lo que puede ser valioso para la toma de decisiones y la mejora de la calidad de los servicios bancarios. A través del análisis de texto, se identifican y categorizan los atributos específicos de los servicios bancarios mencionados. Esto implica reconocer palabras o frases que se refieren a aspectos como la atención al cliente, las tasas de interés, la facilidad de uso de la plataforma en línea, entre otros. El estudio también realiza un análisis de polaridad para evaluar si las opiniones expresadas son fuertemente positivas o negativas. Esto proporciona un mayor nivel de detalle sobre la actitud de los usuarios (Mittal & Agrawal, 2022).

Debido a la utilización de metodologías en minería de texto, el estudio *“Using text mining to measure mobile banking service quality”* propuso medir la calidad del servicio de banca móvil

y detectar quejas de clientes mediante el análisis de reseñas en línea. Se extrajeron y preprocesaron opiniones sobre el *Kakao Mobile Bank*, analizando el sentimiento de cada reseña, se midió la puntuación de la calidad del servicio y se identificaron quejas. Los hallazgos cuentan los aspectos valorados por los clientes en el contexto bancario móvil, que incluyen la practicidad, el disfrute del uso, la seguridad de las transacciones y la atención al cliente. Este enfoque permite a los gestores de bancos móviles detectar tempranamente las quejas de los clientes y prevenir fallos en el servicio, mejorando así la satisfacción del cliente y la calidad general (Leem & Eum, 2021).

Para centralizar el análisis de la gestión de PQR de los clientes en un contexto de establecimientos bancarios, con la “Evolución en las PQR de los clientes de los establecimientos bancarios con enfoque en la percepción de beneficio en términos de calidad, servicio, satisfacción y experiencia” se presta atención a la percepción de los clientes en términos de beneficios relacionados con la calidad del servicio, la satisfacción y la experiencia general. El estudio examina cómo los bancos han abordado las PQRS destacando un aumento de 824.571 en 2017 a 1.129.626 en 2021, atribuido parcialmente a la pandemia, y de cómo mejoraron la calidad del servicio para satisfacer las demandas y expectativas de los clientes. Además, evalúa los motivos de insatisfacción mediante la utilización de nubes de palabras y de cómo la experiencia general influye en la percepción de beneficio que los clientes obtienen al interactuar con los bancos (Suárez & Torres, 2022).

No obstante, para abordar la comprensión de contextos complejos y la captura de relaciones entre palabras en un texto, en el artículo “XLNet: Un modelo de lenguaje pre-entrenado para el procesamiento del lenguaje natural” se introduce el modelo XLNet, una variante de la arquitectura *Transformer* que se caracteriza por su capacidad para modelar

dependencias de largo alcance en texto, lo que lo convierte en un modelo de lenguaje altamente eficiente y preciso. Para el caso logra una mejora significativa sobre BERT (modelo tradicional de lenguaje pre-entrenado) en 20 tareas como comprensión de lectura, inferencia de lenguaje natural, análisis de sentimientos y clasificación de documentos, XLNet utiliza un esquema de permutación autoregresiva bidireccional que permite la consideración de todas las permutaciones posibles de palabras en una oración (Yang, Mo, & Zhu, An empirical study of the financial community network on Twitter, 2014).

Así mismo, para mejorar la predicción de sentimiento en textos, en el artículo “*Text Mining and Analysis Based on XLNet-BiLSTM Model*” se propone un modelo de clasificación emocional que combina XLNet y BiLSTM donde se alcanzó un 95,8%, con una tasa de pérdida de 0,182, aprovechando la habilidad de XLNet para entender la relación entre palabras enmascaradas y no enmascaradas y la eficiencia de BiLSTM en el análisis semántico, la comparación de resultados para seleccionar el modelo se identifica en la Tabla 2. Se inicia con la generación de vectores de palabras contextuales mediante XLNet, seguido por la interpretación de semántica profunda con BiLSTM, y concluye con la clasificación de emociones usando softmax. Probado en textos del Festival de los Faroles en plataformas de redes sociales y medios, el modelo mostró una alta precisión del 0,958 y baja tasa de error de 0,182 (Huang, Hou, Chen, & Hu, 2023).

Tabla 2
Comparación de los modelos en precisión y tasas de pérdida (Huang, Hou, Chen, Yi, & Hu, 2023).

Model	Accuracy	Loss rate
LSTM	0,787	0,523
BERT-LSTM	0,892	0,474
BERT-BiLSTM	0,909	0,472

XLNet-LSTM	0,927	0,191
XLNet-BiLSTM	0,958	0,182

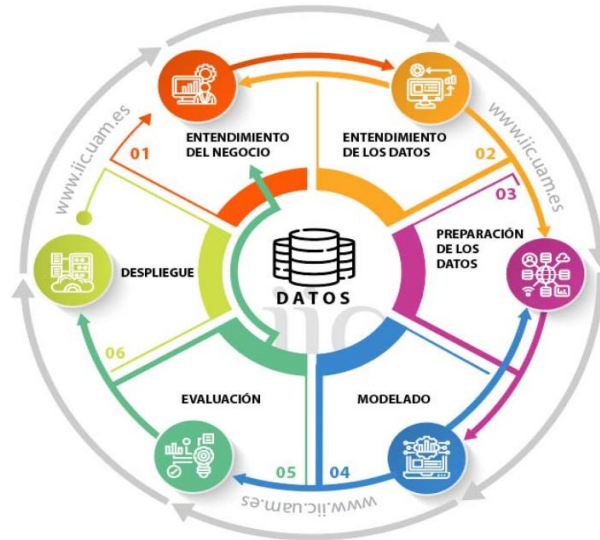
Ahora bien, el avance tecnológico y la digitalización de la información redefinen el panorama global, marcando un punto de inflexión en la manera en que las sociedades evolucionan y se organizan. En Colombia, se traduce en iniciativas estratégicas como el CONPES 3920 de 2018 y el CONPES 3975 de 2019, que buscan consolidar los cimientos de una economía más conectada, innovadora y adaptativa. El CONPES 3920, centrado en el Big Data, reconoce el poder de los datos masivos como un recurso clave para la toma de decisiones informadas y la generación de nuevo conocimiento, impulsando así una gestión pública más eficiente y un sector privado más competitivo (Acevedo, 2019). Paralelamente, el CONPES 3975 aborda la Transformación Digital y la IA, estableciendo directrices para integrar estas tecnologías en el desarrollo nacional, con el fin de mejorar la calidad de vida de los ciudadanos, optimizar procesos y promover la inclusión digital ((DNP), 2023).

Metodología

Para el desarrollo del proyecto, se utiliza la metodología CRISP- DM, representada en la Figura 2 que contempla las siguientes fases (Haya, 2021):

Figura 2

Imagen tomada de: <https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/>



Compresión del Negocio:

Objetivo: Definir claramente el problema empresarial, los objetivos del proyecto y cómo el análisis de sentimientos y la minería de opiniones contribuyen para mejorar la gestión de PQRS, la satisfacción del cliente y la gestión de la reputación de la marca del Grupo AVAL.

Procedimiento: Se realizan sesiones de entendimiento y conocimiento del proceso con usuarios clave dentro del Grupo AVAL para obtener una comprensión detallada de las necesidades empresariales, desafíos en la gestión de PQRS y expectativas del proyecto. Además, se revisa la literatura y estudios de caso relevantes para identificar mejores prácticas y lecciones aprendidas en proyectos similares.

Entendimiento de los Datos:

Objetivo: Identificar, recopilar y explorar los datos necesarios para el desarrollo del modelo.

Localización y población: Se extrae la información desde X (Twitter), enfocándose en las menciones y opiniones relacionadas con el Grupo AVAL y los bancos que lo componen. La población de estudio incluye todos los tweets relevantes publicados en 2020-2023.

Procedimiento: Utilizando la API de X (Twitter) se recopilan tweets que contengan palabras clave relevantes (nombres de los bancos, términos financieros específicos, etc.). Se explora inicialmente los datos para entender su estructura, el volumen y la variedad de las opiniones expresadas.

Preparación de los Datos:

Objetivo: Limpiar y transformar los datos recopilados para su análisis.

Procedimientos y técnicas: La limpieza contempla la eliminación de registros duplicados, irrelevantes o incompletos. Se codifican variables categóricas y normaliza el texto (eliminación de *stopwords*, *stemming*, lematización). Los datos son divididos en conjuntos de entrenamiento y prueba para validar el modelo.

Modelado:

Tipo de estudio: Desarrollo y validación de modelos predictivos basados en técnicas de NLP y aprendizaje automático.

Métodos y procedimientos: Experimentando con algoritmos de clasificación, SVM y redes neuronales, se analizan sentimientos y detectan PQRS. También se contemplan técnicas como la validación cruzada para evaluar el rendimiento de los modelos durante la fase de entrenamiento.

Modelos Estadísticos: Los modelos usados incluyen redes neuronales profundas, dependiendo de la complejidad de los datos y los resultados preliminares.

Evaluación:

Objetivo: Determinar la eficacia de los modelos desarrollados.

Métodos y técnicas: Utilizando métricas como precisión, *recall*, *F1-score* y AUC para evaluar el rendimiento de los modelos. Se compara el rendimiento de diferentes modelos para seleccionar el más adecuado para el despliegue.

Despliegue:

Objetivo: Implementar el modelo seleccionado en un entorno de producción para mejorar la gestión de PQRS en el Grupo AVAL.

Procedimientos: Desarrollando una interfaz o un sistema integrado que permite la aplicación del modelo manera pertinente o en lotes regulares para analizar los nuevos tweets. Se establecen protocolos para la respuesta y gestión automatizada o semiautomatizada de PQRS detectados.

Monitorización:

Objetivo: Supervisar continuamente el rendimiento del modelo en producción y realizar actualizaciones que lo ajusten según sea necesario.

Procedimientos: Se establece un sistema de monitoreo para evaluar el rendimiento del modelo en el tiempo, identificando posibles desviaciones o reducciones en la precisión debido a cambios en las tendencias de los datos o en el comportamiento del usuario. De ser necesario se establecen ajustes y reentrenamientos periódicos del modelo.

La metodología seleccionada se basa en prácticas estándar en ciencia de datos y aprendizaje automático, proporcionando un enfoque estructurado e iterativo que permite adaptarse a las particularidades y a los objetivos del proyecto. La combinación de técnicas de NLP y modelos de clasificación es adecuada para abordar la complejidad del lenguaje natural en

la información captada de las redes sociales, demostrando ser efectiva en estudios similares para análisis de sentimientos y detección de PQRS.

Las variables principales incluyen el texto de los tweets (variables independientes) y las categorías de sentimiento y tipo de PQRS (variables dependientes). Además, se consideran variables como la fecha de publicación, el número de *retweets* o *likes*, y la geolocalización (si está disponible), para enriquecer el análisis y proporcionar información adicional.

Resultados

Entendimiento de los Datos

Fuente de los Datos

Los datos son extraídos de X (Twitter) utilizando la API de Twitter versión 2 para acceder a tweets públicos. Para la recopilación de datos específicos de los bancos pertenecientes al Grupo AVAL, se utilizaron palabras clave asociadas a cada banco, así como *hashtags* relevantes y menciones a las cuentas oficiales de los bancos en X (Twitter). El rango de fechas de la recopilación abarca desde el 1 de diciembre de 2020 hasta el 21 de mayo de 2023, permitiendo un análisis exhaustivo de más de dos años de actividad en X (Twitter) relacionada con los bancos del Grupo AVAL.

Cantidad de Datos

Durante el periodo de recopilación, se lograron reunir un total de 90.205 tweets relacionados con los bancos del Grupo AVAL, generados por aproximadamente 43.619 usuarios únicos. Esta amplia recopilación de datos, que se representa en la Tabla 3, proporciona una base sólida para el análisis de sentimientos y la detección de PQRS, ofreciendo una visión integral de las percepciones y opiniones de los clientes.

Tabla 3

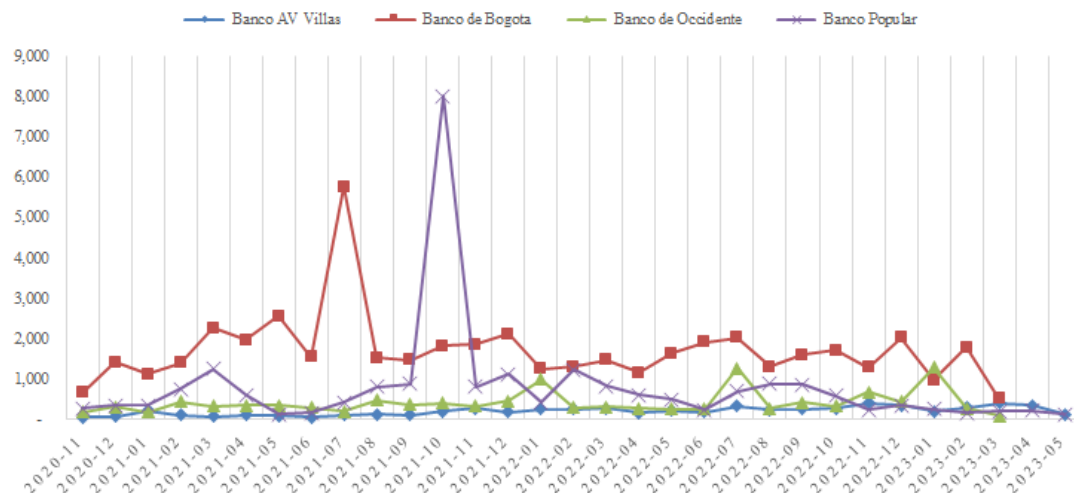
Distribución de tweets por Banco (Elaboración propia, 2024).

Banco	Cantidad Tweets	Usuarios únicos
Popular	23.817	10.284
Av Villas	5.795	3.743
Bogotá	48.998	23.127
Occidente	11.595	6.465

Distribución Temporal

Como se evidencia en la Figura 3, se visualiza cómo se distribuyen los datos a lo largo del tiempo, identificando tendencias, picos de actividad y periodos de poca actividad. En el gráfico de serie temporal se pueden observar picos de actividad en momentos clave, como durante campañas promocionales importantes de los bancos, anuncios significativos, o períodos de crisis financiera y problemas técnicos con los servicios bancarios.

Figura 3 Distribución temporal de tweets (2020 - 2023) (Elaboración propia, 2024).



Cada banco muestra un patrón distinto de actividad a lo largo del tiempo, lo que puede reflejar diferencias en las estrategias de comunicación, eventos específicos que afectan a cada uno o la cantidad de clientes de estos. Se observan varios picos significativos para algunos bancos en momentos específicos. Por ejemplo, el Banco de Bogotá muestra un pico pronunciado alrededor de julio de 2021, que corresponde a su 150º aniversario. El Banco Popular tuvo un aumento notable en octubre de 2021 debido a una campaña con la Copa Dimayor. Los bancos de Bogotá y de Occidente parecen tener una mayor variabilidad y volumen en la cantidad de tweets

en comparación con el Banco AV Villas y el Banco Popular, lo que podría indicar una mayor presencia en línea o una base de clientes más activa.

Aunque hay fluctuaciones mensuales, no parece haber una tendencia clara y constante de aumento o disminución a lo largo de los años para ningún banco, lo que sugiere que la actividad de los tweets puede estar más influenciada por eventos puntuales que por un cambio a largo plazo en la actividad de los usuarios de X (Twitter). En algunos períodos, varios bancos parecen tener picos simultáneos, lo que podría sugerir eventos o condiciones del mercado que afectan a toda la industria bancaria. En otros momentos, los picos son exclusivos de un banco, lo que podría señalar eventos únicos a esa institución.

Preparación de los Datos. En adelante se explica los pasos y actividades realizadas para la preparación, estos se pueden consultar directamente en el cuadernillo del Anexo I - Limpieza de texto.

Limpieza de los Datos

Normalización de Texto. Convertir el texto a un formato estándar donde se homogenizan mayúsculas y minúsculas, se eliminan los acentos y otros caracteres especiales que no aporten información relevante para el análisis de sentimientos, como se puede identificar en la Figura 4.

Figura 4

Ejemplos de Comunicaciones de Clientes sobre Problemas Bancarios en Twitter (Elaboración propia, 2024).

original_text	normalized_text
@BancodeBogota buenas noches, desde el día 21 me enviaron 200 USD desde EEUU y no han sido acreditados, hoy fui al banco no fue mucho lo que hicieron por mi, me dieron un número, llamé y supuestament iban a rastrear el dinero y aún no han hecho nada por mi, a donde debo ir ah	@bancodebogota buenas noches, desde el día 21 me enviaron 200 usd desde eeuu y no han sido acreditados, hoy fui al banco no fue mucho lo que hicieron por mi, me dieron un numero, llame y supuestament iban a rastrear el dinero y aun no han hecho nada por mi, a donde debo ir ah
RT @lumicaso: Una persona con NUEVE millones en cuenta de "ahorro programado" del @BancodeBogota necesita retirar CUATRO y no puede porque...	rt @lumicaso: una persona con nueve millones en cuenta de "ahorro programado" del @bancodebogota necesita retirar cuatro y no puede porque...
@Bco_Popular Pésima experiencia con estas tarjetas de crédito, hice pago total de la deuda y el pago no aparece!! Hoy cancelé una de ellas y no he podido cancelar la que no le aparece el pago, si necesitan una tarjeta de crédito, solicitenla en otro banco.	@bco_popular pesima experiencia con estas tarjetas de credito, hice pago total de la deuda y el pago no aparece!! hoy cancele una de ellas y no he podido cancelar la que no le aparece el pago, si necesitan una tarjeta de credito, solicitenla en otro banco.

Eliminación de Ruido. Retirar URL, nombres de usuario (precedidos por @), *hashtags* (cuando no son relevantes para el análisis), emoticones, símbolos especiales y números que no contribuyan a la detección de PQRS, tal como se observa en la Figura 5.

Figura 5 Casos de Servicio al Cliente: Textos Originales vs Normalizados (Elaboración propia, 2024).

original_text	normalized_text
@BancodeBogota buenas noches, desde el día 21 me enviaron 200 USD desde EEUU y no han sido acreditados, hoy fui al banco no fue mucho lo que hicieron por mi, me dieron un número, llamé y supuestament iban a rastrear el dinero y aún no han hecho nada por mi, a donde debo ir ah	buenas noches desde el día me enviaron usd desde eeuu y no han sido acreditados hoy fui al banco no fue mucho lo que hicieron por mi me dieron un numero llame y supuestament iban a rastrear el dinero y aun no han hecho nada por mi a donde debo ir ah
RT @lumicaso: Una persona con NUEVE millones en cuenta de "ahorro programado" del @BancodeBogota necesita retirar CUATRO y no puede porque...	rt una persona con nueve millones en cuenta de ahorro programado del necesita retirar cuatro y no puede porque
@Bco_Popular Pésima experiencia con estas tarjetas de crédito, hice pago total de la deuda y el pago no aparece!! Hoy cancelé una de ellas y no he podido cancelar la que no le aparece el pago, si necesitan una tarjeta de crédito, solicitenla en otro banco.	pesima experiencia con estas tarjetas de credito hice pago total de la deuda y el pago no aparece hoy cancele una de ellas y no he podido cancelar la que no le aparece el pago si necesitan una tarjeta de credito solicitenla en otro banco

Corrección Ortográfica. Aplicar algoritmos de corrección automática para mejorar la calidad del texto, entendiendo que los tweets a menudo contienen errores tipográficos y de jerga, mostrado en la **Figura 6**.

Figura 6 Comparación de Textos: Textos Originales vs Normalizados (Elaboración propia, 2024).

original_text	normalized_text
@BancodeBogota buenas noches, desde el día 21 me enviaron 200 USD desde EEUU y no han sido acreditados, hoy fui al banco no fue mucho lo que hicieron por mi, me dieron un número, llamé y supuestament iban a rastrear el dinero y aún no han hecho nada por mi, a donde debo ir ah	buenas noches desde el día me enviaron usd desde eeuu y no han sido acreditados hoy fui al banco no fue mucho lo que hicieron por mi me dieron un numero llame y supuestament iban a rastrear el dinero y aun no han hecho nada por mi a donde debo ir ah
RT @lumicaso: Una persona con NUEVE millones en cuenta de "ahorro programado" del @BancodeBogota necesita retirar CUATRO y no puede porque...	rt una persona con nueve millones en cuenta de ahorro programado del necesita retirar cuatro y no puede porque
@Bco_Popular Pésima experiencia con estas tarjetas de crédito, hice pago total de la deuda y el pago no aparece!! Hoy cancelé una de ellas y no he podido cancelar la que no le aparece el pago, si necesitan una tarjeta de crédito, solicitenla en otro banco.	pesima experiencia con estas tarjetas de credito hice pago total de la deuda y el pago no aparece hoy cancele una de ellas y no he podido cancelar la que no le aparece el pago si necesitan una tarjeta de credito solicitenla en otro banco

Segmentación de Tokens. Dividir el texto en unidades más pequeñas (*tokens*), como palabras o frases, que serán la entrada para el modelo.

```
['rt', 'una', 'persona', 'con', 'nueve', 'millones', 'en', 'cuenta', 'de', 'ahorro', 'programado', 'del',  
'necesita', 'retirar', 'cuatro', 'y', 'no', 'puede', 'porque']
```

Filtrado y Selección

Selección de Idioma: Asegurarse de que los tweets están en el idioma deseado (por ejemplo, español), utilizando herramientas de detección de idioma para filtrar los datos.

Detección de Spam y Bots: Utilizar técnicas para identificar y eliminar tweets generados por bots o que sean spam, ya que estos no reflejan la opinión o comportamiento de clientes reales.

- Listas de palabras clave para *spam*: Crear listas de palabras clave asociadas comúnmente con *spam* y verificar si los tweets las contienen.
- Frecuencia de publicación: Los *bots* publican con una frecuencia mayor que los humanos. Si existen *timestamps* de los tweets, se analiza la frecuencia de publicación para identificar posibles *bots* (Molinaro, 2022)
- Análisis de contenido: Los *bots* y el *spam* suelen tener mensajes repetitivos o enlaces promocionales. Pueden buscar repeticiones exactas o similares de mensajes.
- Interacciones y comportamiento: Pueden ser indicativos de *bots* los tweets que provienen de cuentas con pocos seguidores pero muchas publicaciones, o aquellos que no tienen interacciones (me gusta, *retweets*).

Enriquecimiento de Datos

Etiquetado de Sentimientos. Se emplea un modelo pre-entrenado y anotaciones manuales para etiquetar un conjunto de tweets con su sentimiento correspondiente (positivo, negativo, neutro) que servirán para entrenar o afinar el modelo BERT y RNN.

Categorización de PQRS. Definir un conjunto de categorías de PQRS (26 luego de la reclasificación, eran 28 inicialmente) y etiquetar manualmente un subconjunto de tweets que sirvan como base confiable para entrenar el modelo de clasificación.

Transformación de Datos para Modelos

Tokenización. Utilizar el tokenizador para convertir los tweets en *tokens* que el modelo pueda entender, en la **Figura 7** se identifica esta parte.

Figura 7 Ejemplos de Textos Normalizados y sus Tokens Correspondientes (Elaboración propia, 2024).

	normalized_text	tokens
0	buenas noches desde el día me enviaron usd ...	[, buenas, noches, desde, el, día, , me, env...
1	rt una persona con nueve millones en cuenta d...	[rt, , una, persona, con, nueve, millones, en...
2	pesima experiencia con estas tarjetas de cred...	[, pesima, experiencia, con, estas, tarjetas,...
3	tiene toda la razon cobran un seguro qu...	[, tiene, toda, la, razon, , cobran, un...
4	y hablando de viajeslo peor quise activar las...	[, y, hablando, de, viajeslo, peor, quise, ac...

Codificación de las Etiquetas de Clase. Asignar un número único a cada etiqueta de clase, tanto para sentimiento como para las categorías.

Padding y Truncamiento. Asegura que todos los *inputs* tengan el mismo tamaño mediante la adición de *padding* o truncando los tweets más largos para que se ajusten al tamaño máximo de entrada que BERT puede manejar.

Atención de Máscaras. Crear máscaras de atención para informar al modelo cuáles *tokens* son palabras reales y cuáles son relleno, por medio de un conjunto de datos organizado que indique al modelo qué *tokens* son parte del texto original y cuáles son añadidos como relleno. Esto se hace generalmente con valores binarios: 1 para los *tokens* reales y 0 para los *tokens* de *padding*.

Conjuntos de Datos para Entrenamiento y Validación

División de Datos. Separando el conjunto de datos en segmentos para su entrenamiento, validación y pruebas con el fin de entrenar y evaluar el modelo de manera efectiva. Inicialmente, con una distribución del 80% para entrenamiento y 20% para validación. Sin embargo, durante el proceso, se ajusta la distribución con un 90% para entrenamiento y 10% para validación que optimice el aprendizaje del modelo. La evaluación final es realizada utilizando un conjunto de datos completamente nuevo, asegurando así que la evaluación sea objetiva y representativa del rendimiento general del modelo.

Balanceo de Clases. Durante la implementación de los modelos de NLP, se identifica un desbalance significativo en las clases tanto en el modelo de sentimiento como en el de PQRS. Este desafío se aborda mediante el uso de distintas estrategias, según las necesidades específicas de cada modelo. Implementando la técnica SMOTE, generando muestras sintéticas de las clases minoritarias para equilibrar el conjunto de datos. En el modelo BERT para clasificación de PQRS, se opta por calcular los pesos de las clases utilizando la función *compute_class_weight* de la biblioteca *sklearn.utils.class_weight*¹ lo que permite asignar pesos inversamente

¹ La función *compute_class_weight* de la biblioteca *sklearn* es utilizada para calcular los pesos de clase en modelos de aprendizaje automático. *sklearn* (Scikit-learn) es una herramienta de software libre para el análisis de datos y aprendizaje automático en Python. Más información sobre esta función se puede encontrar en la documentación oficial de Scikit-learn: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html.

proporcionales a las frecuencias de las clases en los datos de entrenamiento, integrando estos pesos en el modelo para que las predicciones erróneas en clases minoritarias tengan un costo más alto. Esta variedad en la aplicación de técnicas subraya la adaptabilidad y el enfoque personalizado para maximizar la precisión y la equidad en los resultados de cada modelo.

- 1. Sentimiento: en la Tabla 4 se comparan las distribuciones de clases de acuerdo al balanceo realizado.

Tabla 4
Comparación de distribuciones de Sentimiento antes y después de aplicar SMOTE (Elaboración propia, 2024).

Distribución	Original	SMOTE
Negativo	57.046	57.046
Positivo	9.776	57.046
Neutro	86	57.046

- 2. PQRS: en la Tabla 5 se observan las distribuciones de clases de acuerdo al balanceo realizado.

Tabla 5
Comparación de Distribuciones de Categorías originales, antes y después de aplicar SMOTE (Elaboración propia, 2024).

Distribución	Original	SMOTE
Aplicación	1.156	17.416
Cajeros	863	17.416
Cancelación de Productos	909	17.416
Características	1.368	17.416
Características del producto	304	17.416
Cobros injustificados	2.101	17.416
Competencia	537	17.416
Críticas	17.416	17.416

Desconfianza	9.938	17.416
Entrega	920	17.416
Fallas en la plataforma	411	17.416
Grupo Aval	1.355	17.416
Información sobre espacios de atención	314	17.416
Interés	395	17.416
Luis Carlos Sarmiento Angulo	1.426	17.416
Marca	11.635	17.416
Comentario no asociado a servicio	16.469	17.416
Oficina	2.385	17.416
Página Web	1.678	17.416
Procesos de Solicitud	12.840	17.416
Reacciones frente al mismo	979	17.416
Requisitos para adquirir productos	7	17.416
Respuesta	1.992	17.416
Servilínea	3.959	17.416
Solicitud de nuevos productos	608	17.416
Solicitud Específica	8.557	17.416
Temas externos	10.739	17.416
WhatsApp	323	17.416

En la implementación de los modelos para las PQRS, se mitiga el problema del desbalance de clases fusionando algunas categorías y se realiza una reclasificación manual de los tweets que se encontraban imprecisos. Estas medidas mejoran la homogeneidad y precisión en la categorización, facilitando así un análisis más robusto y fiable de los datos.

Modelado

En el proyecto de clasificación de sentimientos y detección de PQRS utilizando tweets, se comparan dos enfoques: un modelo de *Transformers* basado en BERT pre-entrenado para NLP y

una RNN, que ha sido tradicionalmente utilizada para tareas de secuencia como el procesamiento de texto.

Selección de Modelos.

BERT:

- Ventajas: es capaz de capturar el contexto bidireccional de las palabras en un texto, lo que puede resultar en una mejor comprensión del sentimiento y la detección de PQRS.
- Cómo usarlo: Mediante una versión de BERT específicamente ajustada para clasificación de texto, como *bert-base-uncased* para textos en inglés o una variante en el idioma relevante. La librería *Transformers* de *Hugging Face* ofrece una implementación fácil de usar de BERT y otros modelos de *Transformers*.

RNN:

- Ventajas: Son buenas para el manejo de secuencias de datos, como el texto, porque pueden capturar la información temporal entre las palabras.
- Cómo usarlo: Una estructura común de RNN para tareas de NLP incluye capas LSTM (*Long Short-Term Memory*) o GRU (*Gated Recurrent Unit*) para manejar dependencias a largo plazo en los datos de texto.

Entrenamiento de Modelos

Tratamiento de los Datos.

Para las metodologías propuestas, que requieren que las entradas y las etiquetas estén en un formato específico adecuado para modelos de NLP, se emplea el *LabelEncoder* de la

biblioteca *sklearn.preprocessing*, paso fundamental para convertir las etiquetas categóricas en una forma numérica que el modelo pueda procesar eficientemente.

Preprocesamiento de Texto y Preparación del Modelo.

Preparación de las Etiquetas: Codificación y creación de listas de textos y etiquetas para su uso en el entrenamiento y la evaluación del modelo.

Preparación de Listas de Textos: Se crean listas con los textos y sus respectivas etiquetas codificadas. Este formato facilita la integración directa y eficiente con las rutinas de entrenamiento y evaluación del modelo.

Paradas Tempranas: Se utilizan *callbacks* como paradas tempranas (*EarlyStopping*) para detener el entrenamiento cuando el rendimiento en el conjunto de validación deje de mejorar. También se puede usar *ModelCheckpoint* para guardar automáticamente el mejor modelo durante el entrenamiento.

Validación Cruzada: Como técnica de evaluación para asegurar que los resultados del modelo sean robustos y menos susceptibles a las fluctuaciones en el conjunto de datos de entrenamiento. La validación cruzada de k-pliegues permite que de manera eficiente en el conjunto de datos completo (entrenamiento y prueba), minimice el riesgo de sobreajuste y proporcione una estimación más fiable del rendimiento del modelo en datos no vistos.

Configuración de KFold para Validación Cruzada: Utilizando KFold de *sklearn.model_selection* con hasta 5 divisiones, configurado para barajar los datos antes de cada división, asegurando confiabilidad del modelo. La elección de mantener 5 divisiones se respalda

por el hecho de que, desde la primera división, las métricas demuestran ser satisfactorias, indicando una buena generalización del modelo desde las etapas iniciales.

Configuración de Argumentos de Entrenamiento: Es esencial para optimizar el rendimiento del modelo, mediante la clase *TrainingArguments* de la biblioteca *Transformers* de *Hugging Face*:

Número de Épocas: Se opta por un número reducido de épocas dado que las métricas de precisión alcanzaron niveles altos rápidamente, lo cual indica una rápida convergencia del modelo y reduce el riesgo de sobreajuste.

Learning Rate: Estableciéndose al iniciar el modelo, un valor bajo puede ralentizar el aprendizaje, mientras que un valor demasiado alto puede generar inestabilidad en la convergencia. La elección precisa de este parámetro es crucial para equilibrar la eficiencia y la estabilidad durante el entrenamiento.

Warmup Ratio: Para ajustar gradualmente el *learning rate* durante las primeras etapas del entrenamiento. Este enfoque ayuda a prevenir ajustes bruscos que desestabilicen el aprendizaje, especialmente útil en modelos complejos como los basados en *Transformers*.

Definición de Métricas de Evaluación: Las métricas clave utilizadas para evaluar la eficacia del modelo incluyen exactitud (*accuracy*), precisión (*precision*), *recall* y puntaje F1 (*F1-Score*). Estas métricas proporcionan una visión integral del rendimiento del modelo en tareas de clasificación, evaluando tanto la exactitud global como la capacidad del modelo para manejar clases individuales eficazmente.

Arquitectura

Sentimiento

- BERT: Bajo una configuración de BERT pre-entrenado con ajustes finos específicos de la tarea. Los hiperparámetros clave ajustados incluyen la tasa de aprendizaje, establecida en $2e-5$, y el número de épocas de entrenamiento, fijado en 4. Estos parámetros se seleccionan basándose en la respuesta del modelo durante la fase piloto de pruebas.
- RNN: Configurando una red neuronal recurrente utilizando células LSTM (de 64 a 128 unidades) para capturar dependencias a largo plazo en los datos de texto y una tasa de aprendizaje de $1e-3$, optimizada mediante el uso de un descenso de gradiente estocástico.

PQRS

- BERT: Del modelo específico cuenta con *BertForSequenceClassification*, una variante de BERT adaptada para la tarea de clasificación de textos. Este modelo agrega una capa densa sobre la salida del codificador, donde la última capa oculta del *token* especial [CLS] se utiliza para la clasificación final.

En la configuración del modelo para la clasificación de secuencias, se establecen una serie de parámetros críticos en *TrainingArguments* que se identifica en el Anexo II y III para optimizar el entrenamiento y la evaluación del modelo. El *learning_rate* inicial está fijado en $2e-5$, que es un valor típicamente recomendado para estos modelos en tareas de *finetuning*². La duración del entrenamiento se

² metodología y configuración específica utilizada para el afinamiento

establece en cinco épocas, con un *weight_decay* de 0,01 para mitigar el sobreajuste, influyendo en la complejidad del modelo durante el entrenamiento. Los tamaños de lote por dispositivo se configuran en 16 para entrenamiento y 64 para evaluación, equilibrando la eficiencia computacional y la memoria disponible. La estrategia para guardar los modelos se basa en almacenar automáticamente el mejor modelo al final de cada época, usando la precisión de evaluación (*eval_accuracy*) como criterio principal para la selección del modelo. Se puede consultar directamente en el cuadernillo del Anexo II - Modelo BERT - Balance - Validación Cruzada.

- RNN: se configura con parámetros específicos optimizados para garantizar un equilibrio entre precisión y eficiencia computacional. El modelo comienza con una capa de *Embedding*, con un *vocab_size* calculado como el número total de palabras únicas en el conjunto de datos más uno, permitiendo la inclusión de un índice cero para palabras desconocidas. Se establece un *embedding_dim* de 100, lo que proporciona una dimensionalidad suficiente para capturar relaciones semánticas sin sobrecargar el modelo computacionalmente. La capa de LSTM con 64 unidades, elegida por su capacidad para manejar dependencias de largo plazo en datos de texto, con *dropout* y *recurrent_dropout* ambos configurados en 0,2 para prevenir el sobreajuste. La red se completa con una capa *Dense* que utiliza una función de activación softmax, adecuada para clasificación multiclase, donde el número de clases (*num_classes*) corresponde al número de etiquetas únicas en el conjunto de datos.

Para el entrenamiento, el modelo utiliza un tamaño de lote (*batch_size*) de 700 y

se entrena por 10 épocas, un compromiso entre el tiempo de entrenamiento y la convergencia del modelo. Se emplea una división de validación del 10% (*validation_split=0,1*) para monitorear el desempeño del modelo en datos no vistos durante el entrenamiento, lo cual facilita la detección temprana de sobreajuste mediante el uso de la técnica de parada temprana. Esta técnica observa *val_loss* y detiene el entrenamiento si no hay mejora después de tres épocas (*patience=3*), restaurando los pesos del mejor modelo observado en términos de pérdida en el conjunto de validación. Esta configuración cuidadosamente calibrada asegura que el modelo esté bien ajustado para generalizar sobre nuevos datos, manteniendo un entrenamiento eficiente y efectivo. Se puede consultar directamente en el cuadernillo del Anexo III - Modelo RNN - Balance - Validación Cruzada.

Validación y Ajuste de Modelos

Además de las configuraciones mencionadas previamente en la preparación del modelo, se usan técnicas de regularización, específicas para modelos de *Transformers*, como el *Layer-wise Learning Rate Decay*, donde la tasa de aprendizaje disminuye a medida que se adentra en las capas del modelo.

Evaluación de Modelos

Se explora la efectividad de los diferentes modelos:

Sentimiento: El desarrollo de este modelo se centra en clasificar textos en tres categorías distintas. Para optimizar el rendimiento y la precisión del modelo, se realizan pruebas de

configuración y ajuste de parámetros, explorando diversas técnicas de balanceo de clases para contrarrestar las disparidades en la distribución de los datos.

Este enfoque permite refinar la capacidad del modelo para reconocer y clasificar adecuadamente los sentimientos expresados, asegurando que cada categoría fuese tratada con igual importancia y precisión. Las pruebas exhaustivas no solo ayudan a mejorar la fiabilidad del modelo, sino que proporcionan valiosos *insights* sobre la efectividad de distintas configuraciones y técnicas en el contexto específico del análisis de sentimiento.

- BERT: Se representan los resultados obtenidos por cada modelo, los cuales se pueden encontrar en la Tabla 6, Tabla 7 y Tabla 8.

Modelo 1. Balance de clases, parada temprana, 5 épocas.

Tabla 6
Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,9535
Precisión	0,9531
Recall	0,9535
F1-score	0,9533
Pérdida	0,6114

Con el *Accuracy* se indica que el modelo es capaz de clasificar correctamente el 95,35% de las muestras del conjunto de prueba, sugiriendo que es generalmente preciso. La precisión igualmente alta indica que cuando el modelo predice una clase, es correcto aproximadamente el 95,31% del tiempo. Esto es crucial en aplicaciones donde los falsos positivos tienen consecuencias significativas. El alto *recall* indica que el modelo identifica correctamente el

95,35% de todas las instancias positivas reales en el conjunto de datos. Esto es importante en situaciones donde es crítico no pasar por alto casos positivos. El puntaje F1 también es alto, lo que indica un buen balance entre precisión y *recall*, pero la pérdida es relativamente alta.

Modelo 2. Balance de clases, parada temprana, 1 época, validación cruzada con 2 *folds*.

Tabla 7
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2
Accuracy	0,9148	0,9144
Precisión	0,9199	0,9197
Recall	0,9148	0,9144
F1-score	0,9166	0,9160
Pérdida	0,5649	0,5609

Consistencia en los resultados entre *folds*: Las métricas de *accuracy* , precisión, *recall* y *F1-score* son bastante similares entre ambos *folds*, lo que indica que el modelo es estable y generaliza bien entre diferentes subconjuntos del *dataset*, siendo positivo al sugerir que el modelo no está sobreajustado a un conjunto particular de datos.

Alto rendimiento en todas las métricas: Las métricas son en su totalidad superiores al 90%, indicativo de un buen rendimiento en esta tarea de clasificación.

Pequeña diferencia en la pérdida entre los *folds*: La diferencia en las pérdidas entre los dos *folds* es pequeña (0,5649 vs 0,5609), apoyando la idea de que el modelo es consistente en su rendimiento a través de diferentes segmentos.

Posible consideración sobre la parada temprana: Dado el uso de una sola época para el entrenamiento, y los resultados son altos, podría ser que el modelo haya empezado ya con una buena capacidad de generalización o que el *dataset* no sea especialmente desafiante para el modelo.

Impacto del balance de clases: Representa un efecto favorable en la capacidad del modelo para generalizar bien, dado el alto *recall* y precisión.

Alto rendimiento del Negativo: Precisión de 97% indica que el modelo es bueno para identificar correctamente los casos negativos, minimizando los falsos positivos. El *Recall* de 94% muestra que también es capaz de capturar la mayoría de los casos negativos reales. *F1-score* de 95% confirma el alto rendimiento en esta categoría.

Rendimiento moderado al Positivo: Precisión de 70% sugiere que hay una cantidad considerable de falsos positivos. *Recall* de 82% es relativamente alto, lo que significa que el modelo identifica bien los casos positivos, pero a costa de incluir algunos incorrectos. *F1-score* de 75% es decente, pero hay margen de mejora, especialmente en precisión.

Rendimiento más bajo en Neutro: La precisión del 69% es moderada, lo que indica errores al clasificar otros tipos de comentarios como neutros. El *Recall* de 59% es bajo, lo que sugiere que muchos comentarios neutros no están siendo detectados correctamente. *F1-score* de 63% es el más bajo de los tres, reflejando el desafío de clasificar correctamente esta categoría.

Modelo 3. Balance de clases, parada temprana, 10 épocas, validación cruzada con 5 *folds*.

Tabla 8
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Accuracy	0,9582	0,9619	0,9626	0,9601	0,9628
Precisión	0,9571	0,9611	0,9620	0,9595	0,9617
Recall	0,9582	0,9619	0,9626	0,9601	0,9623
F1-score	0,9570	0,9609	0,9620	0,9596	0,9620
Pérdida	0,6969	0,8121	0,6973	0,8668	0,6523

Los resultados de validación cruzada del Modelo 3 son consistentes y altos en todas las métricas clave a lo largo de los cinco *folds*. Un *accuracy* que oscila entre 95,82% y 96,28% muestra una alta capacidad de generalización del modelo. La precisión también muestra consistencia, con valores entre 95,71% y 96,20%, lo que indica que el modelo tiene una baja tasa de falsos positivos. El *recall* refleja la habilidad del modelo para identificar todas las instancias relevantes, indicando un buen balance entre la detección de clases positivas y la minimización de omisiones. La pérdida varía más que las otras métricas, con el valor más bajo en 0,6523 y el más alto en 0,8668 en los diferentes *folds*, lo cual podría sugerir ciertas variaciones en cómo el modelo se ajusta a los subconjuntos de datos específicos.

- RNN: Se representan los resultados obtenidos por cada modelo, los cuales se pueden encontrar en las Tabla 9, Tabla 10 y Tabla 11.

Modelo 1. Sin balance de clases, 8 épocas.

Tabla 9
Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,962

Precisión	0,961
Recall	0,962
F1-score	0,961
Pérdida	0,147

El modelo clasifica en general correctamente el 96,2% de los ejemplos en el conjunto de datos de prueba. La precisión del 96,1% indica que, de todas las predicciones positivas realizadas por el modelo, el 96,1% fueron correctas. El *recall* del 96,2% muestra que, de todas las instancias positivas reales, el modelo identificó correctamente el 96,2%. El *F1-score* es de 0,961, confirmando que el modelo tiene un buen balance entre precisión y *recall*, lo cual se espera en muchos casos de clasificación.

Modelo 2. Sin balance de clases, regularización L1=0,01, L2=0,01 y 16 épocas.

Tabla 10

Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,962
Precisión	0,961
Recall	0,962
F1-score	0,961
Pérdida	0,160

Clasifica correctamente en general el 96,2% de exactitud de los ejemplos en el conjunto de datos de prueba. La pérdida de 0,1603 es relativamente baja, lo que sugiere que el modelo tiene un buen ajuste a los datos. Aunque es ligeramente más alta que en el ejemplo anterior, sigue

indicando que las predicciones están cerca de las etiquetas verdaderas. A pesar de no haber implementado técnicas de balance de clases, el modelo aún logra métricas altas.

Modelo 3. Sin balance de clases, validación cruzada y 8 épocas.

Tabla 11
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Accuracy	0,9582	0,9610	0,9601	0,9583	0,9584
Precisión	0,9571	0,9602	0,9591	0,9573	0,9575
Recall	0,9582	0,9610	0,9601	0,9583	0,9584
F1-score	0,9571	0,9600	0,9593	0,9574	0,9574
Pérdida	0,1585	0,1475	0,1476	0,16	0,1566

La variabilidad es mínima en la precisión y sugiere que el modelo es estable y confiable. La pérdida varía entre 0,1475 y 0,1600. Aunque hay una ligera variabilidad, los valores son bajos en general, indicando un buen ajuste del modelo a los datos. El uso de validación cruzada proporciona una evaluación más robusta del modelo, asegurando que el rendimiento es consistente y no depende de una sola división de los datos.

PQRS: En el desarrollo de un modelo BERT para la clasificación de PQRS, se aplican conocimientos y técnicas aprendidas previamente en la construcción de modelos de análisis de sentimiento. A diferencia de los modelos de sentimiento, que requerían menos recursos computacionales, el modelo de clasificación de PQRS también enfrenta los desafíos del desbalance de clases, exigiendo una atención meticulosa a la configuración de los parámetros y una utilización más intensiva de recursos de GPU.

Este aumento en la demanda de procesamiento gráfico está justificado por la necesidad de manejar un volumen mayor de datos y una complejidad incrementada en la clasificación. El enfoque en el desbalance de clases es crucial, ya que afecta directamente la precisión y efectividad del modelo, obligándolos a explorar otras técnicas para ajustar el modelo y asegurar que ofrezca un equilibrio y eficiencia en la clasificación de cada tipo de PQRS.

BERT.

Modelo 1. Balance de clases, parada temprana, 5 épocas.

En resumen, los resultados son prometedores y sugieren que el modelo está bien configurado y es capaz de realizar la tarea para la que fue entrenado con alta efectividad, estos resultados se visualizan en la Tabla 12.

Tabla 12
Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,9508
Precisión	0,9516
Recall	0,9508
F1-score	0,9508
Perdida	0,3427

Esto indica que el modelo es capaz de predecir correctamente el 95,08% de los casos en el conjunto de validación. La precisión indica que, de todas las instancias clasificadas positivamente por el modelo, el 95,16% de esas clasificaciones son correctas. Un *recall* de 0,9508, es esencial en escenarios donde no se deben omitir casos positivos. El *F1-score* de 0,9508 es excelente, indicando un buen equilibrio entre precisión y sensibilidad. Un valor de

pérdida de 0,3427 en la evaluación final sugiere que, en promedio, el modelo está bastante seguro de sus predicciones.

Modelo 2. Balance de clases, parada temprana, 1 época, validación cruzada con 2 *folds*, el cual se observa en la Tabla 13.

Tabla 13
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2
Accuracy	0,7558	0,7609
Precisión	0,7770	0,7797
Recall	0,7558	0,7609
F1-score	0,7536	0,7603
Perdida	0,9393	0,8857

La exactitud del modelo es relativamente alta y consistente entre ambos *folds*. Esto indica que el modelo es capaz de clasificar correctamente alrededor de tres cuartos de las instancias en ambos subconjuntos de datos. La precisión también es consistente y ligeramente superior a la exactitud, lo que sugiere que, de las predicciones clasificadas positivamente por el modelo, alrededor del 78% son correctas. El *recall* es idéntico a la exactitud en este caso, lo cual es común en clasificaciones binarias balanceadas o multi-clasificaciones con similar proporción de muestras por clase. Indica que el modelo es capaz de identificar correctamente el 76% de todas las instancias positivas reales.

El *F1-score* es ligeramente inferior a estas métricas por separado. Esto puede sugerir que hay un equilibrio entre la precisión y la sensibilidad, pero con espacio para mejorar en alinear estas métricas más estrechamente. Los valores de pérdida indican cuánto error está generando el

modelo en el conjunto de validación. Aunque la pérdida en el *Fold 2* es menor que en el *Fold 1*, ambos valores son altos, lo que podría indicar que el modelo todavía puede estar lejos de converger a un mínimo en el error de predicción, o que la función de pérdida y la estructura del modelo podrían necesitar ajustes adicionales.

Parada temprana y 1 época: El uso de parada temprana con solo 1 época podría ser problemático, ya que no permite que el modelo explore suficientemente el espacio de soluciones. Aumentar el número de épocas podría proporcionar al modelo más oportunidades para ajustar sus pesos y mejorar el rendimiento en todas las métricas.

Modelo 3. Balance de clases, parada temprana, 10 épocas, validación cruzada con 5 *folds*.

Tabla 14
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2
Accuracy	0,9753	0,9770
Precisión	0,9754	0,9770
Recall	0,9753	0,9769
F1-score	0,9753	0,9769
Perdida	0,3412	0,2659

De acuerdo a los resultados de la Tabla 14, las cifras de exactitud son altas, consistentemente por encima del 97,5% en todos los *folds*, lo que indica que el modelo es preciso en clasificar correctamente las instancias en todas las particiones del *dataset*, superior a los modelos anteriores. El *recall* también muestra valores altos y parecidos, lo que significa que el modelo es capaz de identificar casi todas las instancias positivas reales en el conjunto de datos. El *F1-score*, es igualmente alto, reflejando un balance excelente entre estas dos métricas. A pesar

del alto rendimiento en otras métricas, los valores de pérdida varían más comparativamente y son más altos en algunos *folds*. Esto podría ser un área para explorar mejoras potenciales, posiblemente ajustando la arquitectura del modelo o la función de pérdida.

El uso de 10 épocas con parada temprana parece haber sido efectivo, permitiendo que el modelo aprenda suficientemente sin sobre ajustarse. El manejo del balance de clases también parece funcionar bien, dado el alto rendimiento en todas las métricas.

Alto Rendimiento General:

Precisión y F1-score: La mayoría de las categorías los muestran altos, frecuentemente superando el 0,95.

Recall: Similarmente, es alto en la mayoría de las categorías, indicando que el modelo es capaz de detectar la gran mayoría de las instancias positivas reales en cada categoría.

RNN.

Modelo 1. Sin balance de clases, ES de 5 épocas, que se puede identificar en la Tabla 15.

Todas las métricas están alrededor de 78,3%, lo que sugiere un nivel decente de rendimiento en general, indicando que el modelo predice correctamente aproximadamente el 78,3% de los casos. Los valores iguales para *accuracy*, precisión y *recall* sugieren que el conjunto de datos puede estar bien equilibrado o que el modelo está actuando uniformemente a través de las clases. Presenta una pérdida de 0,752 lo que indica que hay margen de mejora ya que las predicciones del modelo tienen una desviación significativa de los valores reales.

Tabla 15

Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,783
Precisión	0,783
Recall	0,783
F1-score	0,781
Pérdida	0,752

Modelo 2. Balance de clases, ES de 4 épocas representado en la Tabla 16.

Con el *accuracy* ligeramente reducido a 75,8%, posiblemente debido a que el modelo tiene dificultades para ajustarse a las clases balanceadas artificialmente. En cuanto a la precisión que es ligeramente mejor que la *accuracy*, indicando que cuando el modelo predice una clase positiva, es correcto aproximadamente el 76,5% del tiempo.

El *recall* es idéntico a la *accuracy*, lo que sugiere que el modelo tiene una sensibilidad uniforme en todo el conjunto de datos y, por último, se refleja un equilibrio entre precisión y sensibilidad. La pérdida sugiere que el balance de clases no impacta en el error promedio de las predicciones, el equilibrio podría estar impactando negativamente en la capacidad del modelo para generalizar, es adecuado probar diferentes métodos de equilibrio o incluso experimentar con aprendizaje sensible al costo.

Tabla 16

Resumen de Desempeño del Modelo (Elaboración propia, 2024).

Métrica	Valor
Accuracy	0,758
Precisión	0,765

Recall	0,758
F1-score	0,758
Pérdida	0,752

Modelo 3. Sin balance de clases, validación cruzada, 3 épocas y 5 *Folds*, que se observa en la Tabla 17.

Accuracy: Varía de 78,40% a 79,26% a través de los pliegues, mostrando una ligera variabilidad, pero generalmente un rendimiento consistente.

Precisión: Más variación (73,83% a 75,60%) sugiere que la corrección de las predicciones positivas del modelo fluctúa más a través de diferentes subconjuntos de datos.

Recall: Más bajo que la *accuracy* y la *precisión* (64,26% a 66,56%), indicando que el modelo omite una proporción mayor de casos positivos. Esto podría ser crítico dependiendo de la aplicación. Se deben usar técnicas como ajustar el umbral de decisión o centrarse en la ingeniería de características para ayudar a mejorarlo.

F1-score: Demuestra una ligera mejora en los pliegues posteriores, lo que podría indicar que el modelo actúa mejor en ciertos subconjuntos de datos o que se beneficia de ciertas características presentes en esos subconjuntos.

Pérdida: Disminuye a través de los pliegues, lo que sugiere que el modelo mejora progresivamente en sus predicciones o que ciertos pliegues son inherentemente más fáciles para el modelo debido a sus características específicas, el modelo puede ser sensible a los datos específicos en los que se entrena.

Tabla 17
Resumen de Desempeño por Fold en Validación Cruzada (Elaboración propia, 2024).

Métrica	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Accuracy	0,7840	0,7916	0,7926	0,7899	0,7917
Precisión	0,7383	0,7560	0,7497	0,7421	0,7514
Recall	0,6426	0,6549	0,6656	0,6565	0,6589
F1-score	0,6771	0,6895	0,6973	0,6869	0,6907
Pérdida	0,7296	0,7169	0,7099	0,7164	0,6977

Comparación de Modelos

Al evaluar el desempeño de los modelos, es importante no solo considerar la precisión como única métrica de éxito. Un modelo puede tener una alta precisión, pero si presenta un bajo *recall*, puede no ser el más adecuado, especialmente en contextos donde el costo de los falsos negativos es significativo. Por ello, se busca un equilibrio óptimo entre precisión, *recall* y el *F1-score*, que combina ambas métricas para proporcionar una visión más holística del rendimiento del modelo. Esta evaluación integral asegura que seleccionemos un modelo que no solo identifica correctamente las instancias positivas, sino que también minimiza las omisiones críticas, adecuándose efectivamente a las necesidades del proyecto.

Sentimiento.

BERT. En la Tabla 18 y la Tabla 19 se visualizan los resultados obtenidos:

Tabla 18
*Comparación de los modelos BERT por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Epochs	Training Acc	Validation Acc	Validation Loss	Observaciones
--------	--------	-----------------	-------------------	--------------------	---------------

1	5	97,70%	95,31%	0,655*	Perdida alta
2	VL+1	93,29%	92,22%	0,6069*	Perdida alta
3	VL+10	99,58%*	96,23	0,7684	VL = Validación Cruzada, mejor 'Neutro'

Tabla 19
*Comparación de los modelos representativos BERT en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Negativo			Positivo			Neutro		
	Precisión	Recall	F1-Score	Precisión	Recall	F1-Score	Precisión	Recall	F1-Score
1	97%	98%	98%	88%	82%	85%	83%	76%	79%
2	97%	94%	95%	70%	82%	75%	69%	59%	63%
3	98%*	98%*	98%*	90%*	86%	88%	88%	82%	85%

Sobreajuste: El Modelo 3, aunque tiene el rendimiento más alto en las métricas de clasificación, parece tener un potencial sobreajuste dado su alta pérdida de validación. Esto puede ser problemático si el modelo se expone a nuevos datos fuera de los utilizados para entrenamiento y prueba.

Validación Cruzada y Entrenamiento: Incrementar el número de particiones en la validación cruzada y las épocas de entrenamiento ha mejorado el desempeño del modelo en general (Modelo 3), pero también ha aumentado la complejidad del modelo y su tendencia a ajustarse demasiado a los datos de entrenamiento.

Balance entre Precisión y Pérdida: Un área de mejora sería explorar cómo reducir la pérdida de validación mientras se mantiene o mejora la precisión y el resto de métricas. Esto

podría implicar ajustar parámetros del modelo, probar diferentes funciones de pérdida, o implementar técnicas para mitigar el sobreajuste como la regularización o el *dropout*.

RNN. Como se identifica en la Tabla 20 y la Tabla 21 se visualizan los resultados obtenidos:

Tabla 20
*Comparación de los modelos RNN por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Epochs	Training Acc	Validation Acc	Validation Loss	Observaciones
1	8	96,85%*	96,29%*	0,1449*	Mejor balance en f1-score
2	16	96,75%*	96,26%*	0,1499*	Mejor f1 para 'Neutro'
3	VL+8	97,43*	95,99*	0,1599	VL = Validación Cruzada

Tabla 21
*Comparación de los modelos representativos RNN en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Negativo			Positivo			Neutro		
	Precisión	Recall	F1-Score	Precisión	Recall	F1-Score	Precisión	Recall	F1-Score
1	93%	87%	90%*	97%	99%*	98%*	91%*	72%	80%
2	90%	88%*	89%	98%*	98%	98%*	89%	76%*	82%*
3	90%*	84%	87%	97%*	99%*	98%*	95%*	77%*	85%*

Los modelos 1 y 2 destacan por tener un balance superior en el *F1-score* y una precisión de validación entre las más altas. Estos modelos son robustos debido a su consistencia en ambos entrenamiento y validación, lo cual sugiere un buen equilibrio entre precisión y capacidad de generalización.

El Modelo 3 utiliza Validación Cruzada con 8 épocas, alcanzando la precisión más alta registrada en este conjunto de entrenamiento (97,43%), pero con una precisión de validación que no refleja este alto rendimiento, posiblemente por una variabilidad en el conjunto de datos de validación.

Los Modelos 1 y 3 son los más prometedores para futuras implementaciones y pruebas adicionales. Estos modelos logran un equilibrio óptimo entre alto rendimiento y generalización sin caer en sobreajuste. Sería recomendable explorar más a fondo las configuraciones de estos modelos, quizás ajustando parámetros adicionales como el tamaño del *batch* o profundizando en técnicas de regularización para mejorar aún más el rendimiento en la validación.

PQRS.

BERT. En la Tabla 22 y la Tabla 23 se visualizan los resultados obtenidos:

Tabla 22

*Comparación de los modelos BERT por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Epochs	Training Acc	Validation Acc	Validation Loss	Observaciones
1	5	95,08%	94,93%	0,3427*	Mayor pérdida.
2	VL+1	75,83%	74,10%*	0,9532	Baja precisión
3	VL+10	99,93%*	97,87%*	0,3240*	VL = Validación Cruzada. Mejor precisión general.

Tabla 23

*Comparación de los modelos representativos BERT en Métricas de desempeño vs clases, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Categoría	Precisión	Recall	F1-Score	Support
1	Cajeros	98%	98%	98%	197
1	Grupo Aval	98%	98%	98%	265
1	Procesos de Solicitud	98%	95%	96%	2.641
2	Interés	95%	69%	80%	214
2	Desconfianza	91%	81%	85%	4.973
2	Comentario no asociado a servicio	87%	75%	80%	224
3	Cancelación de Productos	99%	100%	99%	172
3	Características	99%	98%	98%	275
3	Oficina	99%	99%	99%	479

El Modelo 1, aunque con una pérdida relativamente alta en validación, muestra una buena consistencia entre la precisión de entrenamiento y validación. Su rendimiento en las diferentes categorías es alto, con valores de precisión, *recall* y *F1-score* consistentemente altos, lo cual es indicativo de un modelo bastante equilibrado. A su vez, tiene un rendimiento sólido en categorías con un volumen significativo de datos (como "Procesos de Solicitud"), lo que puede ayudar a mejorar la robustez del modelo para categorías comunes.

El Modelo 2 tiene la precisión y el *recall* más bajos en validación, y también la pérdida de validación más alta. Esto sugiere que el modelo está luchando para generalizar bien a partir de los datos de entrenamiento. Las métricas de rendimiento específicas por categoría también son menores comparadas con los otros modelos, especialmente en *recall*. Ahora bien, muestra

debilidad en categorías con menos soporte o datos (como "Interés"), y en general, tiene problemas de precisión y *recall*. Esto sugiere que el entrenamiento con una sola época y validación cruzada no fue suficiente para capturar la complejidad de los datos.

El Modelo 3 exhibe el mejor rendimiento en términos de precisión de entrenamiento y validación. Aunque su pérdida de validación sigue siendo significativa, es la más baja entre los tres modelos. Este modelo también muestra métricas favorables de clasificación por categoría, alcanzando casi la perfección en varias de ellas, lo cual puede indicar un cierto sobreajuste, aunque mantiene una buena generalización según la precisión de validación. Se destaca especialmente en categorías que podrían considerarse críticas o específicas como "Cancelación de Productos" y "Características". La precisión y *recall* cercanos o iguales al 100% son indicativos de un excelente aprendizaje, aunque siempre hay que estar alerta ante el posible sobreajuste.

Para el Modelo 3, sería prudente implementar técnicas adicionales para verificar y mitigar el sobreajuste. Esto podría incluir aumentar el conjunto de datos de validación, aplicar técnicas de regularización, o utilizar técnicas de aumento de datos. Además, dado que la pérdida de validación sigue siendo alta en todos los modelos, podría ser útil revisar si la función de pérdida utilizada es la más adecuada para el tipo de datos y las tareas de clasificación que se realizan.

Categorías con Desempeño Excepcional: "Cajeros" y "Desconfianza" muestran rendimientos casi perfectos en todas las métricas, lo que podría indicar que estas categorías tienen características distintivas que el modelo puede aprender fácilmente.

Áreas de Preocupación: "Fallas en la plataforma" muestra un *recall* relativamente bajo (88%) comparado con su precisión (100%), lo que sugiere que mientras las predicciones hechas

son precisas, el modelo está fallando en identificar todas las instancias reales, posiblemente debido a la limitada cantidad de datos. "WhatsApp" también muestra un *recall* más bajo (87%) y un *F1-score* (91%), lo cual podría ser un área para investigar y posiblemente mejorar, quizás a través de más datos de entrenamiento o ajustes en el modelo para esta categoría específica.

Soporte (Número de Instancias): "Críticas", "Desconfianza", y "No identificado" son algunas de las categorías con más instancias, y el modelo maneja bien estas categorías con altos valores en todas las métricas. Categorías con menos instancias, como "Comentario no asociado a servicio" e "Información sobre espacios de atención", aun así, muestran buenos resultados, lo que es un buen indicativo de que el balance de clases y las técnicas de manejo están funcionando adecuadamente.

RNN. Los resultados obtenidos se visualizan en la Tabla 24 y la Tabla 25:

Tabla 24
*Comparación de los modelos RNN por Accuracy, el * indica que los resultados son estadísticamente significativos (Elaboración propia, 2024).*

Modelo	Epochs	Training Acc	Validation Acc	Validation Loss	Observaciones
1	ES	86,82%	79,08%	0,767	ES = Early Stopping si no hay mejora después de 2 epochs
2	Sob + ES	96,66%*	77,41%*	1,23*	Sobremuestreo + ES el cual fue 3
3	VC + 3	82,64%	78,85%	0,708	VC + 3 epoch

Tabla 25
Comparación de los modelos representativos RNN en Métricas de desempeño vs mejores clases (Elaboración propia, 2024).

Modelo	Categoría	Precisión	Recall	F1-Score	Support
1	Críticas	73%	78%	75%	3.449
1	Desconfianza	90%	83%	86%	2.024
1	No identificado	86%	94%	90%	3.118
2	Comentario no asociado a servicio	95%	82%	88%	99
2	No identificado	82%	93%	87%	3.118
2	Oficina	74%	85%	79%	488
3	Luis Carlos Sarmiento Angulo	88%	91%	90%	285
3	No identificado	84%	96%	90%	3.201
3	Temas externos	91%	84%	87%	2.148

Modelo 1 (Paradas tempranas): Tiene buen rendimiento en categorías con alto soporte como "Críticas" y "Desconfianza", cuenta con una buena precisión en la mayoría de las categorías, aún así, presenta un bajo rendimiento en "Fallas en la plataforma" y las categorías con bajo soporte.

Modelo 2 (Balanceado y paradas tempranas): Mejora en *recall* en varias categorías comparado con el Modelo 1, indicando una mejor identificación de casos positivos, especialmente para "Cobros injustificados", sin embargo, la precisión es sacrificada en algunas categorías, lo cual puede ser resultado de la técnica SMOTE que puede introducir ruido al modelo.

Modelo 3 (Validación cruzada): Consistentemente alto en precisión y *F1-score* en la mayoría de las categorías, particularmente fuerte en "Grupo Aval", "Luis Carlos Sarmiento

Angulo" y "Temas externos", aunque cuenta con un bajo rendimiento en categorías con bajo soporte, indicando un posible sobreajuste o una insuficiente generalización.

Selección de Modelo

En la categorización del sentimiento, los modelos BERT muestran un mejor equilibrio general entre precisión, *recall* y pérdida de validación. El Modelo 3 de BERT refleja ser la opción más equilibrada, ofreciendo un buen balance en *F1-score* con una pérdida de validación bastante baja, lo que sugiere una buena capacidad de generalización. Los modelos RNN muestran un desempeño robusto en este conjunto de datos específico para análisis de sentimientos, especialmente considerando la pérdida de validación y el equilibrio de las métricas de desempeño.

Figura 8
Matriz de Confusión Modelo 3 - BERT - Sentimiento

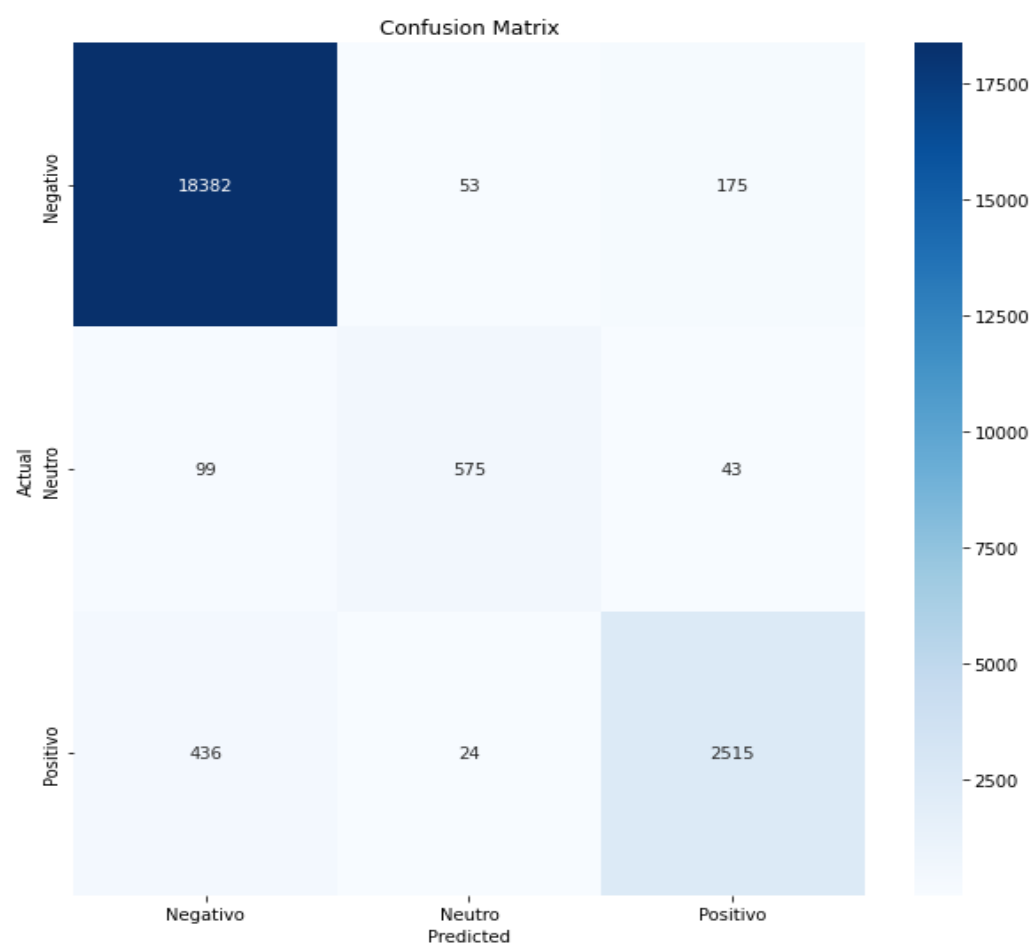
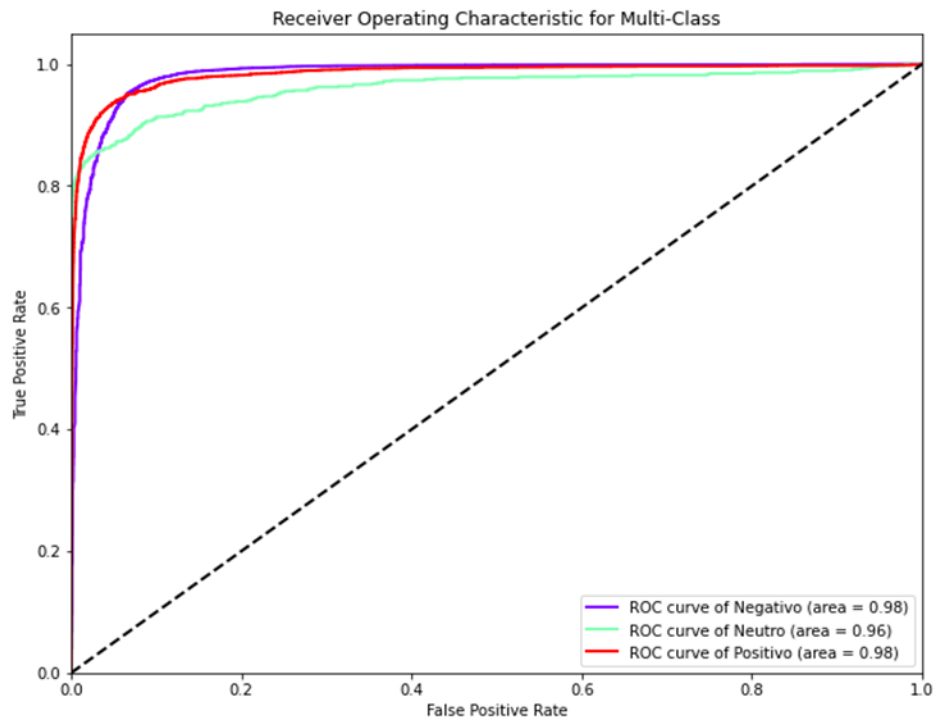


Figura 9
ROC - BERT - Sentimiento



Para la clasificación de las PQRS y basándose en la consistencia entre precisión de entrenamiento y validación, junto con la pérdida de validación, el Modelo 3 de BERT muestra ser la mejor opción entre todos los modelos. Ofrece un equilibrio entre alta precisión y baja pérdida, lo que indica una buena capacidad de generalización. Aunque la alta precisión de entrenamiento puede sugerir sobreajuste, la baja pérdida de validación y alta precisión de validación sugieren que el modelo está bien ajustado. Entre los modelos RNN, el Modelo 3 parece ser el más equilibrado, aunque todos los modelos RNN tienen problemas de precisión y pérdida en comparación con BERT. En resumen, el Modelo 3 de BERT indica ser el más adecuado para implementar en entornos de producción, dado su rendimiento superior y su eficiencia en generalización comparado con los otros modelos. Se puede consultar directamente

en el Anexo IV. Resultados de Modelo 3 – BERT - Sentimiento y Anexo V. Resultados de Modelo 3 – BERT – Sentimiento – Matrices y Curvas ROC.

Figura 10
Matriz de Confusión Modelo 3 - BERT - PQRS

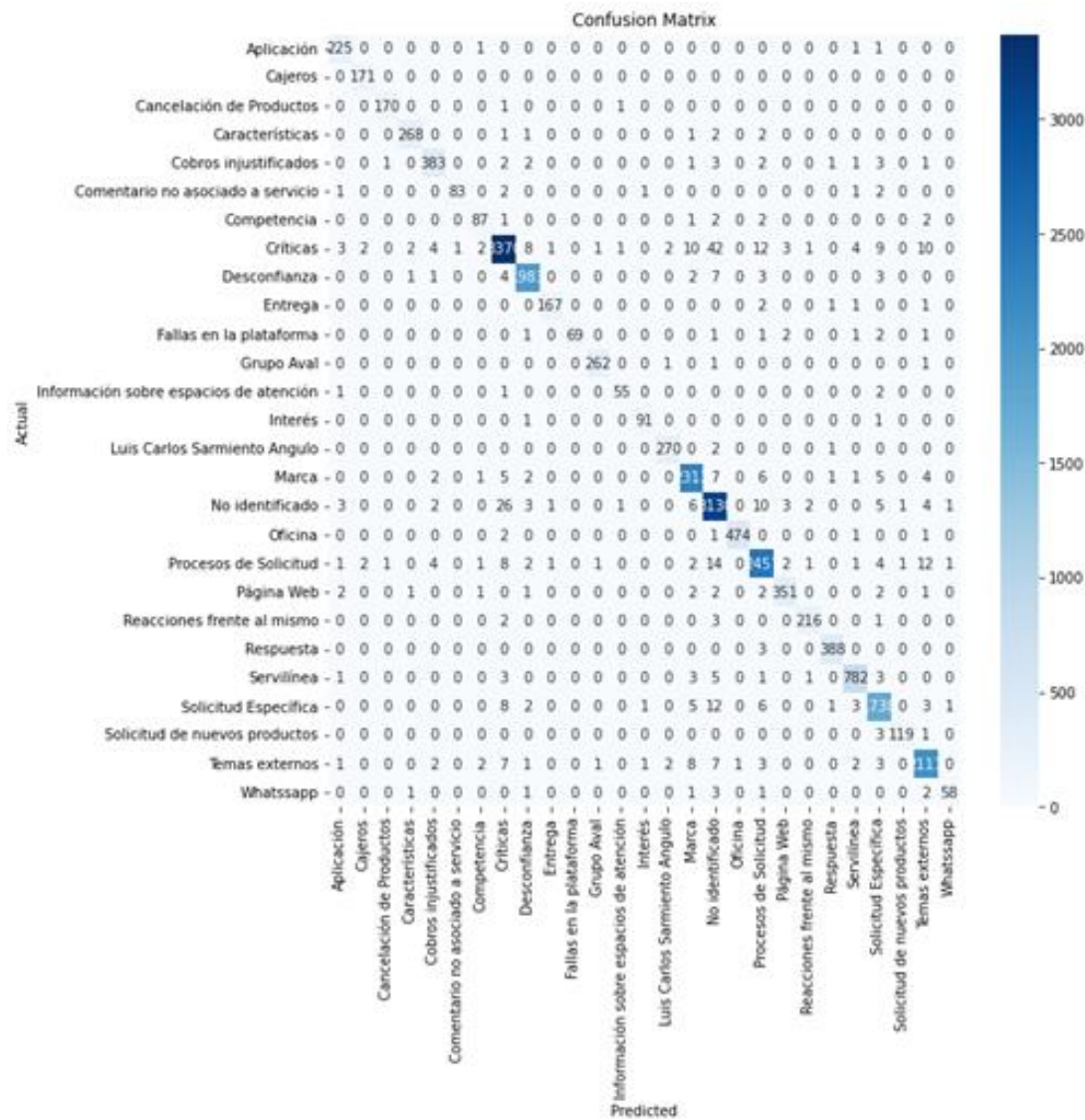
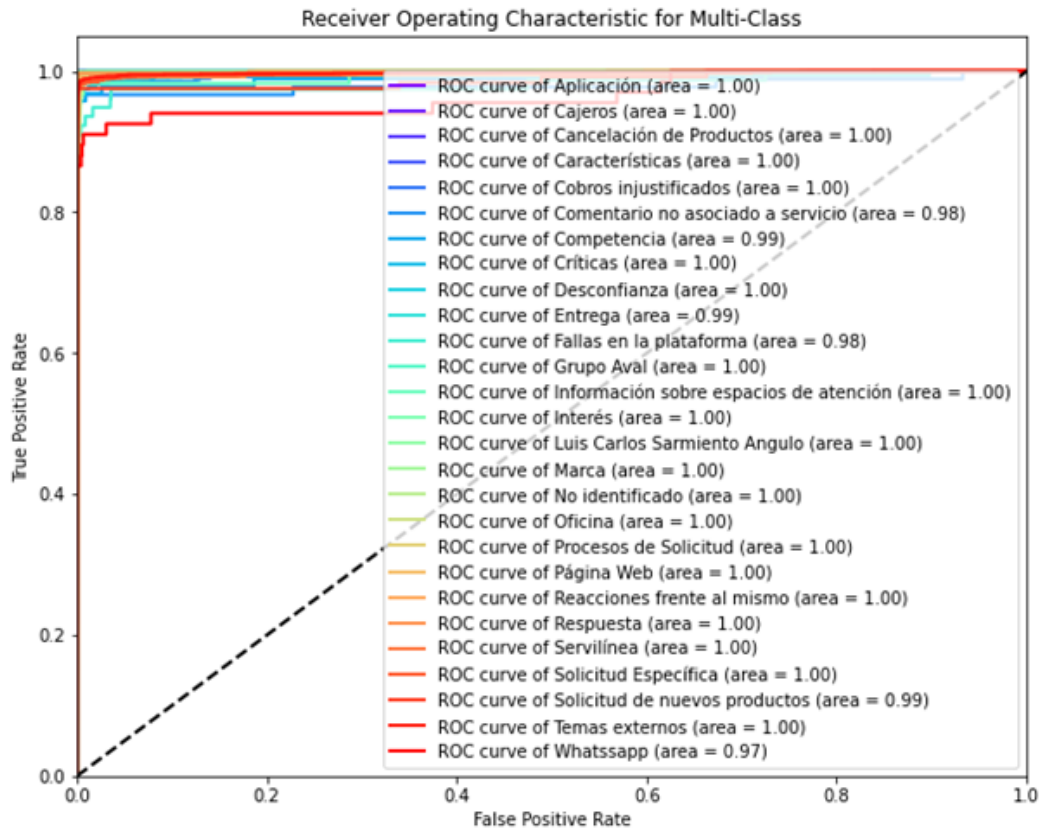


Figura 11
ROC - BERT - PQRS



En el análisis de Sentimiento, las categorías son relativamente generales y menos numerosas (positivo, negativo, neutro), lo que puede ser adecuado para las RNN dado que no requiere un nivel de diferenciación tan detallado. Las RNN podrían manejar eficazmente estas categorías porque la tarea es más directa y menos susceptible al problema de las dependencias a largo plazo y la pérdida de contexto. Pero en la categorización de las PQRS (26 categorías), con un número mayor de categorías, aumenta la complejidad y la necesidad de entender contextos específicos y sutilezas en las diferencias entre categorías. BERT, con su capacidad de análisis contextual avanzado y paralelo, es naturalmente más adecuado para manejar esta complejidad. Se puede consultar directamente en el Anexo VI. Resultados de Modelo 3 – BERT – PQRS y Anexo VII. Resultados de Modelo 3 – BERT – PQRS – Matrices y Curvas ROC.

Es bastante común que los modelos pre-entrenados, como los basados en BERT, muestren un alto rendimiento incluso con configuraciones relativamente simples de entrenamiento, debido a varias razones clave:

- **Ventaja del Pre-entrenamiento:** Los modelos como BERT se pre-entrenan en grandes corpora³ de texto y aprenden una representación rica y generalizable del lenguaje natural. Este pre-entrenamiento ayuda al modelo a entender la estructura y el contexto del lenguaje antes de que se especialice en una tarea específica a través del *fine-tuning*. Por lo tanto, incluso con un entrenamiento adicional limitado, estos modelos pueden realizar tareas específicas con una efectividad sorprendentemente alta.
- **Transferencia de Aprendizaje:** Al transferir el conocimiento desde tareas generales de lenguaje a tareas más específicas, los modelos preentrenados a menudo necesitan sólo ajustes menores para adaptarse eficazmente a nuevas tareas. Esto puede resultar en una alta precisión y otras métricas de rendimiento incluso si el entrenamiento específico del dominio no es extenso.
- **Impacto de las Paradas Tempranas:** Esta técnica evita el sobreajuste para asegurar que el modelo no se entrene demasiado en el conjunto de datos de entrenamiento. Aunque puede mejorar la generalización del modelo en datos no vistos, si el conjunto de datos de validación no es lo suficientemente representativo o es demasiado pequeño, el modelo podría no ser verdaderamente robusto a pesar de mostrar un alto rendimiento durante las pruebas.

³ El término "corpora" es el plural de "corpus" en latín, y en el contexto del procesamiento del lenguaje natural (NLP), se refiere a colecciones de textos escritos o grabaciones de habla que se utilizan para compilar datos de lenguaje. Un corpus puede incluir datos compilados de libros, artículos, transcripciones de discursos, y otros tipos de textos, y es frecuentemente anotado o etiquetado para facilitar tareas específicas de investigación o desarrollo en NLP.

- **Naturaleza del Conjunto de Datos:** Si el conjunto de datos sobre el que se entrena es relativamente "fácil" o si las características que necesita aprender el modelo para realizar predicciones precisas son prominentes y numerosas en los datos de entrenamiento, entonces es posible lograr un alto rendimiento sin técnicas extensivas de validación cruzada.
- **Posible Sesgo en el Conjunto de Validación:** Un alto rendimiento también podría deberse a un conjunto de validación que no desafía adecuadamente al modelo, es decir, contiene ejemplos demasiado similares a los vistos durante el entrenamiento o no presenta suficiente variabilidad en los casos de uso o en las etiquetas.

Con las métricas de precisión, *recall* y *F1-score* consistentemente altas a través de las clases y los valores de pérdida bajos (tanto en entrenamiento como en validación) son indicaciones por tener presente para identificar el modelo que podría considerarse el mejor. Además, con un sobreajuste mínimo que puede inferir si la pérdida de validación no es significativamente mayor que la pérdida de entrenamiento y si la precisión de validación está en línea con la precisión de entrenamiento.

Si una clase se clasifica incorrectamente de manera consistente, se revisa el balance de clases o la representación de características para esa clase en particular. Adicionalmente se examina que los valores de pérdida sean bajos (tanto en entrenamiento como en validación).

Análisis de errores

Se analizaron algunos de los posibles motivos que generaron inexactitudes en la predicción para ambos modelos, entre ellos se encuentra que inicialmente, al ser un etiquetado manual, la clasificación tanto de categoría como el sentimiento presentaba imprecisiones, obligando a realizar una revisión exhaustiva, nuevamente manual, del conjunto de datos y así

reclasificar adecuadamente la información, bajo este ajuste los modelos presentaron una mejoría en sus métricas de desempeño.

El sentimiento ‘Neutro’ se predice incorrectamente, una causa es por ser una clase menos representada en el conjunto de datos o dada la falta de información explícita del usuario que registra el tweet ocasionando ambigüedad y confusión en el modelo.

Errores de interpretación, ya que algunas expresiones tienden a ser semántica y composítivamente similares generando una clasificación errónea al tener mayoría de términos de algún sentimiento en particular.

Registros con sarcasmo o satíricos donde la emoción subyacente puede ser positivo o neutral, pero la clasificación real es negativa, ocasionando inexactitud; otra razón es que la sentencia cuenta con un tono neutro, pero con un uso de palabras menos explícitas para expresar queja o inconformidad.

Clasificaciones erradas por identificación de un tono débil negativo, claridad en la objeción o falta de información relevante en aspectos clave de la frase.

Discusión de los resultados

Rendimiento del Modelo

En comparación con estudios previos como el de (Kongar & Adebayo, 2021) que demostraron la efectividad de los modelos de aprendizaje automático en la optimización de procesos empresariales, el modelo RNN ha mostrado un rendimiento alto con una *accuracy* de aproximadamente 97%, cifra consistente con los resultados obtenidos en estudios similares, lo que refuerza la validez del enfoque propuesto.

Sin Balance de Clases

A pesar de no utilizar técnicas de balance de clases en los modelos RNN, los resultados son robustos. Un hallazgo representativo ya que estudios previos (Blasco Fontecilla, 2021) han señalado que el desbalance de clases puede afectar negativamente el rendimiento de los modelos. La capacidad para mantener métricas altas sin balance de clases indica que las técnicas de regularización y arquitectura del modelo son suficientemente robustas para manejar este desafío.

Regularización L1 y L2

El uso de la regularización ha mostrado ser efectivo en la prevención del sobreajuste, como se refleja en nuestras bajas pérdidas de validación en los modelos RNN. Siendo consistente con los hallazgos de otros investigadores que han utilizado técnicas de regularización para mejorar la generalización de sus modelos (Ranjan & Bhatnagar, 2010).

Evaluación de los Resultados Propios

Consistencia y Robustez del Modelo

- La consistencia de las métricas a través de los diferentes *folds* de validación cruzada (*accuracy*, precisión, *recall* y *F1-score* alrededor del 96%) sugiere que el modelo es robusto y generaliza bien a diferentes subconjuntos de datos.
- La baja variabilidad en la pérdida entre los folds *también* refuerza esta conclusión, indicando que el modelo mantiene un rendimiento estable independientemente de la partición de datos utilizada.

Entrenar el modelo durante 10 épocas ha demostrado ser suficiente para alcanzar un alto rendimiento, evitando tanto el sobreajuste como el subajuste. Esto es un equilibrio importante que ha sido logrado mediante a una cuidadosa selección de hiperparámetros y técnicas de regularización.

Relevancia Práctica

Los resultados obtenidos tienen una relevancia práctica significativa para el Grupo AVAL y otras instituciones financieras. La alta precisión y *recall* del modelo indican que puede ser utilizado efectivamente para clasificar sentimientos y detectar PQRS, contribuyendo así a mejorar la gestión de relaciones con los clientes y la satisfacción del cliente.

Además, el uso de redes sociales como fuente de datos (X en este caso) y la aplicación de técnicas avanzadas de análisis de sentimientos y detección de PQRS reflejan tendencias actuales en el manejo de grandes volúmenes de datos y la digitalización de servicios al cliente, como se menciona en estudios contemporáneos (An & Wan Zainon, 2023).

La comparación con otros trabajos de investigación y la evaluación crítica de los propios resultados demuestran que nuestro enfoque y metodología son sólidos y efectivos. Los resultados

alcanzados no solo están en línea con lo encontrado en la literatura, sino que también ofrecen nuevas perspectivas sobre la gestión de relaciones con clientes en el contexto de las redes sociales y la banca. Estos hallazgos destacan el potencial transformador de la inteligencia artificial y el análisis de datos en la mejora de la satisfacción del cliente y la eficiencia operativa.

Conclusiones

Al aplicar las técnicas, se evidencia una buena clasificación de palabras clave asociadas al sentimiento. Sin embargo, al tener un conjunto con una distribución mayor en calificaciones negativas, los otros atributos presentan una menor precisión, ocasionando la necesidad de reevaluar las clasificaciones inicialmente realizadas y, de ser necesario, modificarlas.

Asimismo, con el uso de BERT en el análisis de sentimiento, mejora el modelo ya que contiene una mayor comprensión del contexto en las opiniones y experiencias de los usuarios en X (Twitter). Esto se demuestra al obtener un rendimiento sobresaliente frente a RNN. En esa misma línea, el desempeño de los modelos de análisis de sentimientos y detección de PQRS desarrollados en este trabajo es sólido en general. Los modelos BERT, debido a su capacidad de aprendizaje profundo y análisis contextual avanzado, presentaron resultados destacables, especialmente en la clasificación de categorías específicas en PQRS. En particular, el Modelo 3 de BERT demuestra un alto rendimiento con una precisión y *recall* superiores al 95% en varias categorías, indicando su capacidad para manejar la complejidad y el volumen de datos de manera efectiva. Sin embargo, se observa un potencial sobreajuste en algunos casos, reflejado en la alta pérdida de validación.

Además, el desbalance de clases fue identificado como un factor crítico afectando el rendimiento del modelo, especialmente en la clasificación de sentimientos. Las categorías 'Neutro' y algunas específicas dentro de PQRS, como 'Fallas en la plataforma' y 'WhatsApp', presentan mayores desafíos, mostrando menores valores de precisión y *recall*. Requiriendo el uso de técnicas más sofisticadas para el balance de clases y la generación de características distintivas para estas categorías.

Aun así, la validación cruzada demuestra ser una técnica eficaz para evaluar la robustez y la capacidad de generalización de los modelos. El uso de esta con múltiples *folds* (especialmente en el Modelo 3 de BERT) permite identificar la estabilidad y consistencia del modelo a través de diferentes subconjuntos de datos, relevante para asegurar que el modelo no se sobreajuste a un conjunto específico de datos de entrenamiento.

Además, los modelos basados en RNN muestran un rendimiento competitivo, especialmente en la tarea de análisis de sentimientos. A pesar de no alcanzar la precisión de BERT, las RNN son una opción viable y eficiente, con un buen balance entre precisión y capacidad de generalización, particularmente en tareas con un menor número de categorías.

El objetivo de desarrollar un modelo efectivo para el análisis de sentimientos y la detección de PQRS en X (Twitter) se logra exitosamente. En esa misma línea, los modelos desarrollados no solo identificaron de manera precisa los sentimientos y categorías de PQRS, sino que también proporcionaron una herramienta valiosa para la gestión proactiva de las interacciones con los clientes del Grupo AVAL. Esto representa una contribución significativa para fortalecer la reputación y las relaciones comerciales de la institución en el entorno digital.

Recomendaciones

Ajuste y Optimización de Modelos

Continuar con la optimización de parámetros y el ajuste de modelos para mejorar la precisión y el *recall* en categorías desafiantes. Contemplar técnicas de regularización y ajuste de la tasa de aprendizaje que pueden ayudar a mitigar el sobreajuste observado en algunos modelos BERT.

Balance de Clases

Implementar técnicas más sofisticadas para el balance de clases, como el uso de algoritmos de sobremuestreo y submuestreo combinados, o la generación sintética de datos a través de técnicas como SMOTE.

Ampliación del Conjunto de Datos

Aumentar el volumen y la diversidad del conjunto de datos de entrenamiento, especialmente para las categorías menos representadas, para mejorar la capacidad del modelo para generalizar a nuevos datos. Esto puede incluir la recopilación de datos adicionales o el uso de datos sintéticos.

Evaluación Continua

Establecer un sistema de evaluación continua para los modelos en producción, utilizando métricas de rendimiento actualizadas y retroalimentación de los usuarios para ajustar y mejorar los modelos continuamente.

Exploración de Nuevas Técnicas

Investigar y probar nuevas técnicas y arquitecturas de modelos de aprendizaje automático y NLP, como *transformers* de última generación y modelos híbridos que combinan las fortalezas de RNN y BERT.

Desarrollo de Herramientas de Visualización

Desarrollar herramientas de visualización y *dashboards* interactivos para monitorear y analizar el rendimiento de los modelos en tiempo pertinente, facilitando la interpretación de los resultados y la toma de decisiones basada en datos.

Colaboración Multidisciplinaria

Fomentar la colaboración entre equipos de desarrollo, expertos en datos y profesionales de atención al cliente para asegurar que los modelos desarrollados sean prácticos, útiles y alineados con las necesidades reales de los usuarios.

Pruebas en Entornos Reales

Realizar pruebas piloto en entornos reales antes de la implementación completa, para identificar y solucionar posibles problemas en el desempeño del modelo y su integración con los sistemas de gestión existentes.

Referencias

- (DNP), D. N. (2023, Noviembre 8). *DNP*. Retrieved from <https://www.dnp.gov.co/>
- Acevedo, M. (2019). Hallenges involved by analysis of citizens 'data-the case of the big data colombian public policy-first in the region. *Revista General de Derecho Administrativo*, 50.
- An, J., & Wan Zainon, W. (2023). Integrating color cues to improve multimodal sentiment analysis in social media. *Engineering Applications of Artificial Intelligence*, 126. Retrieved from <https://www.sciencedirect.com/science/article/abs/pii/S0952197623010588?via%3Dihub>
- Birjali, M., Kasri, M., & Beni-Hssane, A. (2021). A comprehensive survey on sentiment analysis: Approaches, challenges and trends. *Knowledge-Based Systems*.
- Blasco Fontecilla, H. (2021). El impacto de las redes sociales en las personas y en la sociedad: redes sociales, redil social, ¿o telaraña? Tarbiya El impacto de las redes sociales en las personas y en la sociedad: redes sociales, redil social, ¿o telaraña? *Tarbiya, Revista de Investigación e Innovación Educativa*, 49. Retrieved from <https://revistas.uam.es/tarbiya/article/view/15232>
- Cobe, E. (2023, Septiembre 11). *Ecobe*. Retrieved from Redes Sociales: Su Importancia y Funciones en el Mundo Actual: <https://blog.ecobe.digital/redes-sociales/redes-sociales-para-que-sirven/>
- Grupo AVAL. (2023). *Grupo AVAL*. Retrieved from <https://www.grupoaval.com/nuestras-companias>

Haya, P. (2021). *La metodología CRISP-DM en ciencia de datos*. Retrieved from Instituto de Ingeniería del Conocimiento: <https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/>

Huang, L., Hou, J., Chen, Y. Y., & Hu, J. (2023). Text Mining and Analysis Based on XLNet-BiLSTM Model. *Proceedings - 2023 International Conference on Intelligent Media, Big Data and Knowledge Mining, IMBDKM 2023*, (pp. 57–61). Retrieved from <https://doi.org/10.1109/IMBDKM57416.2023.00018>

Hurtado, L., & Arias, P. (2022). Avances en el análisis de datos y aplicaciones en redes sociales. 67-89.

Johan, B., Huina, M., & Xiaojun, Z. (2011). Twitter mood predicts the stock market. *Journal of Computational Science*, 1-8.

Katsafados, A., Nikoloutsopoulos, S., & Leledakis, G. (2023). Twitter sentiment and stock market: a COVID-19 analysis. *Journal of Economic Studies*.

Kongar, E., & Adebayo, O. (2021). Impact of Social Media Marketing on Business Performance: A Hybrid Performance Measurement Approach Using Data Analytics and Machine Learning. *IEEE Engineering Management Review*, 133 - 147.
[doi:10.1109/EMR.2021.3055036](https://doi.org/10.1109/EMR.2021.3055036)

Leem, B. H., & Eum, S. W. (2021). Using text mining to measure mobile banking service quality. *Industrial Management and Data Systems*, 121(5), 993–1007. Retrieved from <https://doi.org/10.1108/IMDS-09-2020-0545>

- Mejía Monsalve, F., & Salinas Rendón, M. (2015). En Estudios sobre la gestión de relaciones con clientes. 123-135.
- Mittal, D., & Agrawal, S. R. (2022). Determining banking service attributes from online reviews: text mining and sentiment analysis. *International Journal of Bank Marketing*, 40(3). Retrieved from <https://doi.org/10.1108/IJBM-08-2021-0380>
- Molinaro, D. (2022, Enero 27). Avast. Retrieved from <https://www.avast.com/es-es/c-what-is-a-bot>
- Oh, Y. K., & Kim, J. M. (2021). What Improves Customer Satisfaction in Mobile Banking Apps? An Application of Text Mining Analysis. *Asia Marketing Journal*, 23(4), 28–37. doi:10.53728/2765-6500.1581
- Pinto Souza, T. T., Kolchyna, O., Treleaven, P., & Aste, T. (2015). Twitter sentiment analysis applied to Finance: a case study in the retail industry. *Handbook of Sentiment Analysis in Finance*. Mitra, G. and Yu, X. (Eds.). (2016). ISBN 1910571571., 2016.
- Planeación, D. N. (2019). *Politica Nacional Para La Transformacion Digital e Inteligencia Artificial*, Consejo Nacional de Politica Economica y Social - Republica de Colombia. Documento Conpes 3975. Retrieved from <https://colaboracion.dnp.gov.co/CDT/Conpes/Econ%C3%B3micos/3975.pdf>
- Ranjan, J., & Bhatnagar, V. (2010). Application of data mining techniques in the financial sector for profitable customer relationship management. *International Journal of Information and Communication Technology*, 2(4).
- Ranjan, R., & Bhatnagar, S. (2010). Técnicas avanzadas en la minería de datos. 98-112.

Ridge, B. V. (2023, 10 14). *mediummultimedia.com*. Retrieved from <https://www.mediummultimedia.com/social-media/como-afecta-el-uso-de-las-redes-sociales-en-la-comunicacion/>

Salazar Castellanos, D. (2023). *Lista: Así es el servicio al cliente por parte de los bancos en Colombia*. Retrieved from www.bloomberglinea.com: <https://www.bloomberglinea.com/latinoamerica/colombia/lista-asi-es-el-servicio-al-cliente-por-parte-de-los-bancos-en-colombia/>

Shi, X., Duy Nguyen, D., & Wang, M. (2023). Earnings expectations and the quality of financial services. *Journal of Accounting and Public Policy*, 107-115.

Singh, A., Bhatia, R., & Saha, S. (2024). Complaint and Severity Identification From Online Financial Content. *IEEE Transactions on Computational Social Systems*, 11(1), <https://doi.org/10.1109/TCSS.2022.3215528>.

Suárez, J. P., & Torres, J. A. (2022, Septiembre 16). *Colegio de Estudios Superiores de Administración-CESA*. Retrieved from <https://repository.cesa.edu.co/handle/10726/4950>

Superintendencia Financiera de Colombia. (2023). *superfinanciera.gov.co*. Retrieved from <https://www.superfinanciera.gov.co/IngresoPowerBI>

Torres Gómez, M. d. (2015). Identificación de niveles de calidad en el servicio a partir de peticiones, quejas y reclamos, en entidades bancarias de Colombia 2007-2014. *Libre Empresa*.

Yang, S. Y., Mo, S. Y., & Zhu, X. (2014). An empirical study of the financial community network on Twitter. *IEEE/IAFE Conference on Computational Intelligence for Financial Engineering, Proceedings (CIFEr)*.

Yang, S. Y., Mo, S. Y., & Zhu, X. (2014). An empirical study of the financial community network on Twitter. *IEEE/IAFE Conference on Computational Intelligence for Financial Engineering, Proceedings (CIFEr)*.

Zheludev, I., Smith, R., & Aste, T. (2014). When can social media lead financial markets? *Scientific Reports*, 4. Retrieved from <https://doi.org/10.1038/srep04213>

Zhu, L., Zhu, Z., Zhang, C., Xu, Y., & Kong, X. (2023). Multimodal sentiment analysis based on fusion methods: A survey. *Information Fusion*.

Lista de anexos

Anexo I. Limpieza de texto

Anexo II. Modelo BERT - Balance - Validación Cruzada

Anexo III. Modelo RNN - Balance - Validación Cruzada

Anexo IV. Resultados de Modelo 3 – BERT - Sentimiento

Anexo V. Resultados de Modelo 3 – BERT – Sentimiento – Matrices y Curvas ROC

Anexo VI. Resultados de Modelo 3 – BERT – PQRS

Anexo VII. Resultados de Modelo 3 – BERT – PQRS – Matrices y Curvas ROC

Anexo I. Limpieza de texto

El código realiza la limpieza y preparación de datos de tweets para su posterior análisis. Primero, carga los datos, verificando su correcta importación. Convierte las fechas a formato datetime y visualiza la distribución temporal de los tweets. Luego, muestra algunos tweets aleatorios para entender el contenido antes de proceder con la normalización, donde elimina acentos, URLs, nombres de usuario, hashtags, caracteres especiales y números. Adicionalmente, define una lista de palabras clave para identificar y filtrar tweets spam. Finalmente, aplica estas transformaciones y presenta una muestra de los tweets limpios y filtrados.

```
##### LIMPIEZA #####
# Importaciones necesarias
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import unicodedata
import re # Importa el módulo de expresiones regulares para manipular textos con patrones
          # específicos
from textblob import TextBlob # Librería para procesamiento de texto natural

#-----

# Montar el drive de Google para acceder a los archivos almacenados allí
from google.colab import drive
drive.mount('/content/drive')

# Cargar los datos desde un archivo CSV especificando valores nulos y el separador
data = pd.read_csv("/content/drive/MyDrive/NPL/Proyecto/Twitter.csv", na_values="-", sep=";")
print(data.head()) # Muestra las primeras filas para verificar la correcta carga de datos

#-----

# Contar y mostrar el número total de tweets en el dataset
total_tweets = len(data)
print(f"Total de tweets: {total_tweets}")

#-----
```

```

# Convertir la columna de fecha a tipo datetime y mostrar el rango de fechas de los tweets
data['date'] = pd.to_datetime(data['date'])
print(f"Rango de Fechas: {data['date'].min()} a {data['date'].max()}")

#-----

# Visualizar la distribución de tweets a lo largo del tiempo usando un gráfico
data.set_index('date', inplace=True) # Establecer la columna 'date' como índice para facilitar la
resampling
data.resample('M').size().plot(title='Distribución de Tweets por Mes') # Agrupa los datos por
mes y los grafica
plt.ylabel('Número de Tweets') # Etiqueta del eje Y
plt.show() # Muestra el gráfico

#-----

# Ver algunos tweets al azar para entender mejor el contenido antes de procesar
print(data['original_text'].sample(5))

#-----

# Función para normalizar y limpiar el texto de los tweets
def normalize_text(text):
    if pd.isnull(text):
        return text # Retorna NaN para mantener coherencia en el dataset en caso de valores nulos

    text = str(text).lower() # Convierte a minúsculas para estandarizar el texto
    text = ".join(c for c in unicodedata.normalize('NFD', text) if unicodedata.category(c) != 'Mn')
# Elimina acentos

    # Uso de expresiones regulares para eliminar URLs, nombres de usuario, hashtags, símbolos
    especiales y números
    text = re.sub(r'http\S+', "", text) # Elimina URLs
    text = re.sub(r'@\w+', "", text) # Elimina nombres de usuario
    text = re.sub(r'#S+', "", text) # Elimina hashtags
    text = re.sub(r'[^\w\s]', "", text) # Elimina caracteres especiales, dejando solo palabras y
    espacios
    text = re.sub(r'\d+', "", text) # Elimina números

    # Opcional: corrección ortográfica (puede ser lento, considerar su necesidad)
    corrected_text = TextBlob(text)
    return str(corrected_text.correct())

# Aplicar la función de normalización al dataset
data['normalized_text'] = data['original_text'].apply(normalize_text)
print(data[['original_text', 'normalized_text']].head()) # Mostrar los cambios

```

```
#-----  
  
# Lista de palabras clave asociadas a spam para filtrar mensajes no deseados  
spam_keywords = ['compra ahora', 'descuento', 'gratis', 'no te lo pierdas', 'click aquí', 'suscríbete']  
  
# Función para determinar si un tweet contiene spam basado en palabras clave  
def is_spam(text):  
    return any(keyword in text for keyword in spam_keywords) # Devuelve True si alguna  
    palabra clave está en el texto  
  
# Aplicar la función para identificar tweets spam y filtrarlos  
data['is_spam'] = data['normalized_text'].apply(is_spam)  
filtered_data = data[~data['is_spam']] # Elimina tweets identificados como spam  
  
print(filtered_data[['original_text', 'is_spam']].head()) # Muestra los tweets filtrados  
  
#-----
```


Anexo II. Modelo BERT - Balance - Validación Cruzada

El código realiza la limpieza, preparación y entrenamiento de un modelo BERT para la clasificación de tweets. Comienza con la importación de librerías esenciales y la carga de datos desde un archivo Excel. Luego, prepara las etiquetas y divide los datos en conjuntos de entrenamiento y validación usando KFold. Configura los argumentos de entrenamiento y define un conjunto personalizado de datos para BERT. Se implementan funciones de métricas y callbacks para monitorizar y detener el entrenamiento según la precisión. Finalmente, el modelo se entrena y evalúa en múltiples pliegues, guardando los resultados, generando curvas ROC y matrices de confusión para cada pliegue.

```
#-----  
# Librerías:  
import os  
import pandas as pd  
import numpy as np  
import json  
import pickle  
import torch  
from sklearn.metrics import accuracy_score, precision_recall_fscore_support  
from transformers import BertTokenizer, BertConfig, BertForSequenceClassification,  
TrainingArguments, Trainer, IntervalStrategy, TrainerCallback, TrainerControl, TrainerState  
from transformers.integrations import TensorBoardCallback  
from sklearn.model_selection import train_test_split, KFold  
from torch.nn import CrossEntropyLoss  
from torch.utils.data import Dataset, DataLoader  
from sklearn.preprocessing import LabelEncoder  
from sklearn.utils.class_weight import compute_class_weight  
from sklearn.metrics import roc_curve, auc  
import torch.nn.functional as F  
import matplotlib.pyplot as plt  
from sklearn.metrics import confusion_matrix, classification_report  
  
#-----  
  
# Carga de datos  
df = pd.read_excel('D:\\Angelica Moreno\\Proyecto de grado\\data.xlsx', engine='openpyxl')
```

```

#-----

# Definir rutas y variables iniciales
path_balance = "D:/Angelica Moreno/Proyecto de
grado/Modelo/ModeloNoBalanceoVC/model_saved"
log_dir = os.path.join(path_balance, 'logs') # Construye el path completo para los logs

#-----

# Preparación de las etiquetas
le = LabelEncoder()
df['subcategoria_encoded'] = le.fit_transform(df['subcategoria'])

#-----

# Preparación de listas de textos y etiquetas
texts = df['text_new'].tolist()
labels = le.transform(df['subcategoria'].tolist())

#-----

# Configuración de KFold
n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True, random_state=777)

#-----

# Variables para acumular resultados
accumulated_results = []

#-----

# Definir las rutas de almacenamiento para cada modelo

# La configuración para TrainingArguments en la biblioteca Hugging Face Transformers es
crucial para definir cómo se comportará el entrenamiento del modelo.

# Configuración de los argumentos de entrenamiento
training_args = TrainingArguments(
    output_dir=path_balance,          # Directorio de salida para los modelos guardados.
    evaluation_strategy="epoch",      # Realizar evaluación al final de cada época.
    learning_rate=2e-5,               # Establece tu learning rate inicial aquí
    num_train_epochs=5,               # Número de épocas de entrenamiento.
    weight_decay=0.01,                # Puedes ajustar el weight_decay si es necesario!
    per_device_train_batch_size=16,   # Tamaño del batch de entrenamiento por dispositivo.
    per_device_eval_batch_size=64,    # Tamaño del batch de evaluación por dispositivo.

```

```

    logging_dir=log_dir,          # Directorio para almacenar logs.
    logging_strategy="epoch",     # Registrar métricas en cada época.
    load_best_model_at_end=True,   # Cargar el mejor modelo encontrado al final del
entrenamiento.
    save_strategy="epoch",        # Guardar el modelo al final de cada época.
    lr_scheduler_type='linear',   # Usa un decay lineal del learning rate
    warmup_ratio=0.1,            # La proporción de los pasos totales de entrenamiento para
el warmup
    metric_for_best_model='eval_accuracy' # Métrica para determinar el mejor modelo.
)

```

#-----

La función `compute_metrics` es una función de ayuda que se utiliza en el contexto de entrenamiento de modelos de aprendizaje automático con PyTorch, específicamente con la biblioteca Transformers de Hugging Face. Esta función se utiliza para calcular varias métricas importantes que evalúan la efectividad del modelo en tareas de clasificación.

Definición de la función `compute_metrics`

```

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    accuracy = accuracy_score(labels, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='weighted')
    return {
        "eval_accuracy": accuracy,
        "eval_precision": precision,
        "eval_recall": recall,
        "eval_f1": f1,
    }

```

#-----

El fragmento define una clase personalizada de PyTorch Dataset, llamada `TweetsDataset`. Esta clase está diseñada para preparar datos textuales para su uso con un modelo de lenguaje transformador, en este caso, un modelo BERT.

Definición del dataset personalizado

```

class TweetsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):

```

```

        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            truncation=True,
            padding='max_length',
            return_attention_mask=True,
            return_tensors='pt',
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
        }

#-----

# Para imprimir la evolución del loss y del accuracy después de cada época, puedes utilizar un
callback personalizado como el siguiente:

class MetricsPrinter(TrainerCallback):
    def on_evaluate(self, args, state: TrainerState, control: TrainerControl, metrics=None,
**kwargs):
        print(f"Epoch: {state.epoch}")
        print(f" Loss: {metrics['eval_loss']}")
        print(f" Accuracy: {metrics['eval_accuracy']}")

#-----

# Definir un Callback Personalizado para la Parada Temprana:

class EarlyStoppingCallback(TrainerCallback):
    def __init__(self, accuracy_threshold=0.99):
        super().__init__()
        self.accuracy_threshold = accuracy_threshold

    def on_evaluate(self, args: TrainingArguments, state: TrainerState, control: TrainerControl,
metrics=None, **kwargs):
        # Imprimir el accuracy cada vez que se evalúa el modelo
        print("Callback de evaluación llamado.")
        print(f"Epoch: {state.epoch} - Eval Metrics: {metrics}")

```

```

        # Verificar si el accuracy supera el umbral
        if metrics['eval_accuracy'] >= self.accuracy_threshold:
            print(f"Stopping early - model achieved target accuracy of {self.accuracy_threshold}")
            control.should_training_stop = True

#-----

# Configuración inicial del tokenizer (puede ser único para todos los pliegues)
tokenizer = BertTokenizer.from_pretrained('dccuchile/bert-base-spanish-wwm-cased')

# Bucle de validación cruzada
for fold, (train_index, val_index) in enumerate(kf.split(texts)):
    print(f"Training fold {fold+1}/{n_splits}")

    # Configuración del modelo (se reinicia para cada pliegue)
    config = BertConfig.from_pretrained('dccuchile/bert-base-spanish-wwm-cased',
num_labels=len(le.classes_))
    model = BertForSequenceClassification(config)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    # Dividir los datos en entrenamiento y validación para la iteración actual
    train_texts = [texts[i] for i in train_index]
    train_labels = [labels[i] for i in train_index]
    val_texts = [texts[i] for i in val_index]
    val_labels = [labels[i] for i in val_index]

    # Crear conjuntos de datos de entrenamiento y validación
    train_dataset = TweetsDataset(train_texts, train_labels, tokenizer, max_len=128)
    val_dataset = TweetsDataset(val_texts, val_labels, tokenizer, max_len=128)
    train_loader = DataLoader(train_dataset, batch_size=700, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=700, shuffle=False)

    # Crear una nueva instancia de Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        compute_metrics=compute_metrics,
        callbacks=[TensorBoardCallback(), MetricsPrinter(), EarlyStoppingCallback()]
    )

    # Entrenar el modelo
    trainer.train()

```

```

# Evaluar el modelo y guardar los resultados
results = trainer.evaluate()
accumulated_results.append(results)

# Guardar los resultados de evaluación
results_file = os.path.join(path_balance, f"evaluation_results_fold_{fold}.json")
with open(results_file, 'w') as f:
    json.dump(results, f)

# Agregar resultados al acumulado
accumulated_results.append(results)

print(f"Finished training fold {fold+1}/{n_splits}")

# Obtener las predicciones para el conjunto de validación
predictions = trainer.predict(val_dataset)

# Calcular probabilidades y etiquetas verdaderas
y_probs = F.softmax(torch.from_numpy(predictions.predictions), dim=-1).numpy()
y_true = predictions.label_ids

# Ruta graficos y matriz de confusión
folder_path = 'D:/Angelica Moreno/Proyecto de grado/Modelo'
os.makedirs(folder_path, exist_ok=True)

# Calcular ROC para cada clase
fpr = {}
tpr = {}
roc_auc = {}
for i in range(config.num_labels):
    fpr[i], tpr[i], _ = roc_curve(y_true == i, y_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Graficar las curvas ROC
plt.figure(figsize=(10, 8))
colors = iter(plt.cm.rainbow(np.linspace(0, 1, config.num_labels)))
for i in range(config.num_labels):
    color = next(colors)
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label='ROC curve of class {0} (area =
{1:0.2f})'.format(i, roc_auc[i]))

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')

```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for Multi-Class')
plt.legend(loc="lower right")
plt.savefig(os.path.join(folder_path, f'roc_curves_fold_{fold}.png'))
plt.close()

# Guardar los resultados de evaluación
results_file = os.path.join(folder_path, f'evaluation_results_fold_{fold}.json')
with open(results_file, 'w') as f:
    json.dump(results, f)

# Imprimir y guardar la matriz de confusión y el informe de clasificación
predictions_labels = np.argmax(predictions.predictions, axis=1)
conf_matrix = confusion_matrix(y_true=predictions.label_ids, y_pred=predictions_labels)
print("Matriz de Confusión:")
print(conf_matrix)
conf_matrix_file = os.path.join(folder_path, f'confusion_matrix_fold_{fold}.csv')
pd.DataFrame(conf_matrix).to_csv(conf_matrix_file, index=False, header=False)
print(f"Matriz de confusión guardada en {conf_matrix_file}")

print("\nInforme de Clasificación:")
print(classification_report(y_true=predictions.label_ids, y_pred=predictions_labels))
```

Anexo III. Modelo RNN - Balance - Validación Cruzada

El código detalla el entrenamiento y validación de un modelo de red neuronal recurrente (RNN) para la clasificación de tweets. Incluye la instalación de bibliotecas, carga y limpieza de datos, tokenización y vectorización de textos, y uso de sobremuestreo con SMOTE para balancear clases. Se define y entrena un modelo secuencial de Keras con técnicas de regularización como dropout y normalización por lotes, utilizando validación cruzada y early stopping para prevenir el sobreajuste. Finalmente, el rendimiento del modelo se evalúa con métricas como precisión, recall y F1-score, visualizando los resultados mediante matrices de confusión y reportes de clasificación.

```
# -*- coding: utf-8 -*-  
"""Anexo III. Modelo RNN - Balance - Validación Cruzada.ipynb
```

Automatically generated by Colab.

```
Original file is located at  
    https://colab.research.google.com/drive/1dbrDFvmT48XJYwXzyl32ve0MPio_5l2f  
"""
```

```
#pip install tensorflow==2.1  
!pip install tensorflow==2.8.0
```

```
#pip install keras==2.3.1  
from tensorflow.keras.models import Sequential
```

```
!pip install --upgrade livelossplot
```

```
!pip install imbalanced-learn
```

```
import nltk  
nltk.download('punkt')  
nltk.download('stopwords')  
from nltk.stem.snowball import SnowballStemmer  
from nltk.stem import WordNetLemmatizer  
from nltk.corpus import stopwords  
from nltk import word_tokenize  
from nltk.data import load
```



```
from nltk.stem import SnowballStemmer
```

```
import string  
from string import punctuation
```

```
import sklearn  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold  
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer  
from sklearn.metrics import classification_report, confusion_matrix
```

```
import keras  
from keras.models import Sequential  
from keras.utils import np_utils  
from keras.layers import Dense, Dropout, Activation, BatchNormalization  
#from keras.optimizers import RMSprop  
from tensorflow.keras.optimizers import RMSprop  
from keras.callbacks import  
from livelossplot.inputs.tf_keras import PlotLossesCallback
```

```
import livelossplot  
from livelossplot import PlotLossesKeras  
import tensorflow as tf  
from tensorflow.keras import backend
```

```
import pandas as pd  
import numpy as np  
import re  
import nltk  
import csv  
from tensorflow import keras  
from nltk.corpus import stopwords
```

```
from numpy import array  
from keras_preprocessing.text import one_hot  
from keras_preprocessing.sequence import pad_sequences  
#from tensorflow.keras.models import Sequential  
#from keras.models import Sequential  
from tensorflow.keras.layers import Activation, Dropout, Dense  
from tensorflow.keras.layers import Flatten  
from tensorflow.keras.layers import GlobalMaxPooling1D  
from tensorflow.keras.layers import Embedding  
#from keras.layers.embeddings import Embedding  
from sklearn.model_selection import train_test_split  
from tensorflow.keras.preprocessing.text import Tokenizer
```

```

from google.colab import drive
drive.mount('/content/drive')

# df = pd.read_csv("/content/drive/MyDrive/NLP/Twitter.csv", na_values="-", sep = ";")
#df = pd.read_csv('/content/drive/MyDrive/2° - Procesamiento LN/Proyecto/Twitter.csv',
encoding='latin-1', quoting=csv.QUOTE_ALL, sep=";", on_bad_lines='skip')
df = pd.read_excel('/content/drive/MyDrive/3° - Profundización 1/Datos/Twitter.xlsx',
engine='openpyxl')

# Verificar valores NaN en la columna 'text'
nan_rows = df['text_new'].isna().sum()
print("Cantidad de valores NaN en 'text_new':", nan_rows)

# Eliminar filas con valores NaN
df = df.dropna(subset=['text_new'])

# O reemplazar los valores NaN por un texto apropiado
df['text_new'] = df['text_new'].fillna("Texto faltante")

# Verificar si hay valores NaN en resultado
print("Valores NaN en resultado:", df['resultado'].isna().sum())

"""Sobremuestreo (oversampling): El sobremuestreo implica aumentar la cantidad de ejemplos
en las categorías minoritarias. Puedes utilizar la biblioteca imbalanced-learn, que proporciona
métodos específicos para el balanceo de clases."""

import seaborn as sns
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
# Seleccionar las columnas relevantes para la clasificación
df = df[['text_new', 'categoria', 'subcategoria', 'resultado']]

# Dividir los datos en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(df['text_new'], df[['categoria', 'subcategoria',
'resultado']], test_size=0.2, random_state=42)

# Crear el vectorizador TF-IDF para representar los textos como características numéricas
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Entrenar un clasificador SVM lineal
classifier_cat = LinearSVC()
classifier_cat.fit(X_train_vec, y_train['categoria']) # Utilizar las tres columnas de y_train o #
Utilizar solo la columna 'sentimiento' de y_train

```

```

classifier_subcat = LinearSVC()
classifier_subcat.fit(X_train_vec, y_train['subcategoria']) # Utilizar las tres columnas de y_train
o # Utilizar solo la columna 'sentimiento' de y_train

classifier_sent = LinearSVC()
classifier_sent.fit(X_train_vec, y_train['resultado']) # Utilizar las tres columnas de y_train o #
Utilizar solo la columna 'sentimiento' de y_train

# Evaluar el modelo en el conjunto de prueba
y_pred_cat = classifier_cat.predict(X_test_vec)
y_pred_subcat = classifier_subcat.predict(X_test_vec)
y_pred_sent = classifier_sent.predict(X_test_vec)

# print("Reporte de clasificación para la variable 'categoria':")
# print(classification_report(y_test['categoria'], y_pred_cat))
# print("Reporte de clasificación para la variable 'subcategoria':")
# print(classification_report(y_test['subcategoria'], y_pred_subcat))
# print("Reporte de clasificación para la variable 'resultado':")
# print(classification_report(y_test['resultado'], y_pred_sent))

# Clasificar un nuevo texto
def classify_text(text):
    text_vec = vectorizer.transform([text])
    #predictions = classifier.predict(text_vec)
    category = classifier_cat.predict(text_vec)[0]
    subcategory = classifier_subcat.predict(text_vec)[0]
    sentiment = classifier_sent.predict(text_vec)[0]
    #sentiment = classifier.predict(text_vec)
    #category, subcategory, sentiment = classifier.predict(text_vec)
    return category, subcategory, sentiment

# Ejemplo de clasificación de un nuevo texto
# new_text = "Este es un texto de prueba"
# category, subcategory, sentiment = classify_text(new_text)#category, subcategory,

from imblearn.over_sampling import RandomOverSampler

# Instancia del sobremuestreador
oversampler = RandomOverSampler(random_state=42)

# Aplicar sobremuestreo "resultado"
X_train_balanced, y_train_resultado_balanced = oversampler.fit_resample(X_train_vec,
y_train['resultado'])

y_train_balanced_df = pd.DataFrame(y_train_resultado_balanced, columns=['resultado'])

```

```

palette_colors = {'Positivo': 'green', 'Negativo': 'red', 'Neutro': 'blue'}
# Graficar la distribución después del sobremuestreo
sns.countplot(x='resultado', data=y_train_balanced_df, palette=palette_colors)

palette_colors = {'Positivo': 'green', 'Negativo': 'red', 'Neutro': 'blue'}
sns.countplot(x='resultado', data=df, palette=palette_colors)

X = df['text_new']
X = X.astype(str)
y = df['resultado'].map({'Positivo':0, 'Negativo':1, 'Neutro':2})

# Verificar si hay valores NaN en resultado
print("Valores NaN en resultado:", y.isna().sum())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.0549, random_state=42)
print("La cantidad de comentarios empleados para entrenamiento es de:")
display(X_train.shape)
print("Y, la cantidad de comentarios empleados para validación es de:")
display(X_test.shape)

"""## Tokenización y vectorización"""

# Create Vectorizer
spanish_stopwords = stopwords.words('spanish') # -> Eliminar palabras sin “significado”
stemmer = SnowballStemmer('spanish') # -> Reducir una palabra a su raíz
non_words = list(punctuation)
non_words.extend(['¿', '¡'])
non_words.extend(map(str, range(10))) # -> Eliminar caracteres “innecesarios”:
símbolos, puntuación y números
def stem_tokens(tokens, stemmer):
    stemmed = []
    for item in tokens:
        stemmed.append(stemmer.stem(item))
    return stemmed
def tokenize(text):
    text = ".join([c for c in text if c not in non_words])
    tokens = word_tokenize(text)
    # stem
    try:
        stems = stem_tokens(tokens, stemmer)
    except Exception as e:
        print(e)
        print(text)
        stems = []
    return stems
vect = CountVectorizer(

```

```

        analyzer = 'word',
        encoding='utf-8',
        tokenizer = tokenize,
        lowercase = True,
        stop_words = spanish_stopwords,
        max_features=20000,          # -> Limitar el número de columnas de la matriz a
20000
        #ngram_range=(1, 2),        # -> Incluir 1-grams y 2-grams
        min_df=2)                   # -> Incluir términos que aparecen al menos 2 veces

# Verificar si hay valores NaN en y_train
print("Valores NaN en y_train antes del mapeo:", y_train.isna().sum())

# Verificar nuevamente por valores NaN
print("Valores NaN en y después del mapeo:", y.isna().sum())

print("Dimensiones de X_dtm:", y_train.shape) # Debería mostrar (n_samples, n_features)
print("Dimensiones de y:", y.shape)          # Debería mostrar (n_samples,)
print("Valores únicos en y:", y_train.unique())

y.dropna(inplace=True)

# Asegurarse de que no hay valores fuera de rango
print("Valores únicos en y:", y.unique())

X_dtm = vect.fit_transform(X_train)
Y_dtm = np_utils.to_categorical(y_train, 3)
print("Tamaño de matriz X_Train:")
display(X_dtm.shape)
print("Tamaño de matriz Y_Train:")
display(Y_dtm.shape)

# Create the model
model = Sequential()
model.add(Dense(units=128, input_shape=X_dtm.shape[1:]))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))                # -> model.add(Dropout(0.2))
model.add(Dense(units=3, input_dim=1))
model.add(Activation('softmax'))
model.summary()

```

""El modelo presenta una estructura secuencial simple con una capa oculta de 128 neuronas seguida de una capa de normalización por lotes y dropout para reducir el sobreajuste, seguido de una capa densa final para clasificación, que corresponden a las clases 'Negativo', 'Positivo' y

'Neutro'. El número total de parámetros es más de 1.6 millones, lo que sugiere que el modelo tiene una capacidad considerable para aprender de los datos.'"""

```
# Compilar el modelo
model.compile(loss='categorical_crossentropy', optimizer='rmsprop', metrics=['accuracy'])
```

```
import numpy, scipy.sparse
spX_dtm = scipy.sparse.csr_matrix(X_dtm)
spY_dtm = scipy.sparse.csr_matrix(Y_dtm)
spX_dtmTest = scipy.sparse.csr_matrix(X_dtmTest)
spY_dtmTest = scipy.sparse.csr_matrix(Y_dtmTest)
```

```
spX_dtm.sort_indices()
spY_dtm.sort_indices()
spX_dtmTest.sort_indices()
spY_dtmTest.sort_indices()
```

```
spX_dtm
```

```
X_dtm
```

```
# Fit the model
model.fit(X_dtm, Y_dtm, validation_data=(X_dtmTest, Y_dtmTest), epochs=1, verbose=1,
callbacks=[PlotLossesCallback()])
```

```
# Realiza las predicciones
predictions = model.predict(X_dtmTest)
```

```
ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)
```

```
# Confusion Matrix Epoch = 1
print('Confusion Matrix Epoch = 1')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report Epoch = 1
print('Classification Report Epoch = 1')
```

```

print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

# Fit the model
model.fit(X_dtm, Y_dtm, validation_data=(X_dtmTest, Y_dtmTest), epochs=8, verbose=1,
callbacks=[PlotLossesCallback()])

"""###Epoch = 1
- La precisión de entrenamiento es de alrededor del 92.35%, mientras que la precisión de
validación es del 94.9%. Esto sugiere que el modelo generaliza bien ya que la precisión en los
datos de validación es incluso más alta que en los datos de entrenamiento.
- La curva de pérdida (loss) final fue de 0.242 y la de validación de 0.1644, lo que indica un buen
ajuste del modelo a los datos sin señales evidentes de sobreajuste.
###Epoch = 8
- La precisión de entrenamiento es de alrededor del 96.77%, mientras que la precisión de
validación es del 95.93%. Esto sugiere que el modelo generaliza bien ya que la precisión en los
datos de validación es incluso más alta que en los datos de entrenamiento.
- La curva de pérdida (loss) final fue de 0.1186 y la de validación de 0.1448, lo que indica un
buen ajuste del modelo a los datos sin señales evidentes de sobreajuste.
"""

# Evaluate the model
scores = model.evaluate(X_dtmTest, Y_dtmTest)
print("\n%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))

""" Hay métricas altas de precisión y pérdida bajas, lo que está en línea con los gráficos
mostrados. Un accuracy de más del 96% en la validación es excelente para la mayoría de las
aplicaciones, aunque se debe considerar la complejidad y el balance de clases.

"""

# Realiza las predicciones
predictions = model.predict(X_dtmTest)

ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)

# Confusion Matrix
print('Confusion Matrix Epoch = 8')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)

```

```

print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report Epoch = 8
print('Classification Report Epoch = 8')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

```

""- La matriz de confusión para las clases 'Negativo' y 'Positivo' muestra que el modelo tiene un rendimiento bastante bueno en la clasificación de 'Positivos', pero hay un número significativo de 'Negativos' que se clasifican incorrectamente como 'Positivos'. No hay errores entre 'Negativo' y 'Neutro', lo que podría sugerir que la clase 'Neutro' es bastante distinta de las otras dos en términos de características aprendidas por el modelo o que hay muy pocos datos 'Neutro' que confirmen las pruebas.

- La clase 'Positivo' tiene una alta precisión y recall, lo que sugiere que el modelo es bastante bueno identificando correctamente esta clase
- Con las métricas de precisión, recall y f1-score son altas para las clases 'Negativo' y 'Positivo'. Aun así, 'Neutro' tiene un recall bajo, indicando que el modelo casi siempre falla al identificar esta clase. Una razón puede ser por un desbalance en el conjunto de datos, donde las muestras 'Neutro' son mucho menos frecuentes que las otras clases, lo que resulta en menos oportunidades para que el modelo aprenda a identificarlas correctamente.

#Reporte de Clasificación:

- Las métricas en el reporte de clasificación confirman que las clases 'Negativo' y 'Positivo' son predichas con alta precisión y recall, resultando en puntajes F1 sólidos.
- La clase 'Neutro', sin embargo, tiene un recall de 0.5 y una precisión de 0.88, lo que resulta en un puntaje F1 de 0.64. Esto podría ser una señal de que la clase 'Neutro' está menos representada en los datos o es más difícil de distinguir.
- El puntaje de precisión ponderado y el recall son altos, lo que indica un buen rendimiento general del modelo, pero podría ser engañoso si la clase 'Neutro' es importante para tus objetivos.

##Validación Cruzada al modelo 2

""

```

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

```

Configuración de la validación cruzada

n_splits = 5

kfold = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

Modificación de los pliegues del modelo actual


```

accuracy_scores = []
precision_scores = []
recall_scores = []
f1_scores = []

# Proceso de validación cruzada
for train_index, test_index in kfold.split(X_dtm, y_train):
    X_trainvc, X_test = X_dtm[train_index], X_dtm[test_index]
    y_trainvc, y_test = Y_dtm[train_index], Y_dtm[test_index]

    # Reconstrucción del modelo de ser necesario
    model = Sequential()
    model.add(Dense(units=128, input_dim=X_trainvc.shape[1]))
    model.add(Activation('relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.5))
    model.add(Dense(units=3))
    model.add(Activation('softmax'))

    model.compile(loss='categorical_crossentropy', optimizer='rmsprop', metrics=['accuracy'])

    # Entrenar el modelo
    model.fit(X_trainvc, y_trainvc, epochs=8, verbose=0)

    # Evaluar el modelo
    y_pred = np.argmax(model.predict(X_test), axis=1)
    y_true = np.argmax(y_test, axis=1)

    # Calcular y almacenar las métricas
    accuracy_scores.append(accuracy_score(y_true, y_pred))
    precision_scores.append(precision_score(y_true, y_pred, average='macro'))
    recall_scores.append(recall_score(y_true, y_pred, average='macro'))
    f1_scores.append(f1_score(y_true, y_pred, average='macro'))

# Imprimir los resultados promedios de las métricas
print(f"Accuracy promedio: {np.mean(accuracy_scores):.3f}")
print(f"Precision promedio: {np.mean(precision_scores):.3f}")
print(f"Recall promedio: {np.mean(recall_scores):.3f}")
print(f"F1-Score promedio: {np.mean(f1_scores):.3f}")

print(f"Accuracy promedio: {np.mean(accuracy_scores) * 100:.3f}%")
print(f"Precision promedio: {np.mean(precision_scores) * 100:.3f}%")
print(f"Recall promedio: {np.mean(recall_scores) * 100:.3f}%")
print(f"F1-Score promedio: {np.mean(f1_scores) * 100:.3f}%")

# model

```

```

# model.save(f'/content/drive/MyDrive/Modelo.h5')
# import pickle

# pickle_out = open(f'/content/drive/MyDrive/Modelo.pickle','wb')
# pickle.dump(model, pickle_out)
# pickle_out.close()

# pickle_out = open(f'/content/drive/MyDrive/Vectorizador.pickle','wb')
# pickle.dump(vect, pickle_out)
# pickle_out.close()

"""#Mejora del modelo

###Early Stopping:
Detiene el entrenamiento cuando una métrica monitoreada deja de mejorar
"""

from tensorflow.keras.callbacks import EarlyStopping

# Configurar early stopping
early_stopping = EarlyStopping(
    monitor='val_loss', # Otra opción común es 'val_accuracy'
    patience=10,      # Número de epochs sin mejora tras los cuales se detiene el entrenamiento
    restore_best_weights=True # Restaura los pesos del modelo desde la epoch con el mejor valor
    de la métrica monitoreada.
)

# Incluir early stopping en los callbacks durante el entrenamiento
model.fit(
    X_dtm, Y_dtm,
    validation_data=(X_dtmTest, Y_dtmTest),
    epochs=50, # Puedes aumentar el número de epochs ya que early stopping puede detener el
    entrenamiento antes
    verbose=1,
    callbacks=[PlotLossesCallback(), early_stopping]
)

# Realiza las predicciones
predictions = model.predict(X_dtmTest)

ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)

# Confusion Matrix
print('Confusion Matrix EarlyStopping')
print(' ')

```

```

cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report
print('Classification Report EarlyStopping')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

"""La detención se generó a los 11 epoch

###Ajustar la Tasa de Aprendizaje:
Cambiar la tasa de aprendizaje durante el entrenamiento.
"""

from tensorflow.keras.optimizers.schedules import ExponentialDecay

# Configurar un decaimiento exponencial de la tasa de aprendizaje
initial_learning_rate = 0.01
lr_schedule = ExponentialDecay(
    initial_learning_rate,
    decay_steps=100000,
    decay_rate=0.96,
    staircase=True
)

# Usar el decaimiento de la tasa de aprendizaje en el optimizador
optimizer = RMSprop(learning_rate=lr_schedule)

model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Ajustar el modelo con los datos
model.fit(X_dtm, Y_dtm, validation_data=(X_dtmTest, Y_dtmTest), epochs=11, verbose=1,
callbacks=[PlotLossesCallback()])

# Realiza las predicciones
predictions = model.predict(X_dtmTest)

ynewClass = np.argmax(predictions, axis=1)

```

```

yoldClass = np.asarray(y_test)

# Confusion Matrix
print('Confusion Matrix con Ajuste de Tasa y Epoch = 11')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report
print('Classification Report con Ajuste de Tasa y Epoch = 11')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

model.fit(
    X_dtm, Y_dtm,
    validation_data=(X_dtmTest, Y_dtmTest),
    epochs=50, # Puedes aumentar el número de epochs ya que early stopping puede detener el
entrenamiento antes
    verbose=1,
    callbacks=[PlotLossesCallback(), early_stopping])

""""Se detuvo a los 15 epoch""""

# Realiza las predicciones
predictions = model.predict(X_dtmTest)

ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)

# Confusion Matrix
print('Confusion Matrix con Ajuste de Tasa y early_stopping')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')

```

```

print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report
print('Classification Report con Ajuste de Tasa y early_stopping')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

"""###Regularización:
Implementar regularización L1, L2 o Dropout para prevenir el sobreajuste penalizando los pesos
grandes en las neuronas.
"""

from tensorflow.keras.regularizers import l1_l2

model.add(Dense(
    units=64,
    activation='relu',
    kernel_regularizer=l1_l2(l1=0.01, l2=0.01) # Regularización L1 y L2
))
model.add(Dropout(0.5)) # Aumentar el dropout puede ayudar si hay sobreajuste

# Ajustar el modelo con los datos
model.fit(X_dtm, Y_dtm, validation_data=(X_dtmTest, Y_dtmTest), epochs=16, verbose=1,
callbacks=[PlotLossesCallback()])

# Realiza las predicciones
predictions = model.predict(X_dtmTest)

ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)

# Confusion Matrix
print('Confusion Matrix Regularización y Epoch = 16')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')

```

```

print(' ')
# Classification Report
print('Classification Report Regularización y Epoch = 16')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

```

""""##Sobremuestreo

El sobremuestreo tiende a mejorar el recall de las clases minoritarias a costa de aumentar los falsos positivos (disminuyendo la precisión), pero puede aumentar el F1-score si el balance entre precisión y recall es óptimo.

""""

```

import seaborn as sns
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report
from sklearn.svm import SVC
# Seleccionar las columnas relevantes para la clasificación
df = df[['text_new', 'categoria', 'subcategoria', 'resultado']]

# Dividir los datos en conjuntos de entrenamiento y prueba
#X_train, X_test, y_train, y_test = train_test_split(df['text_new'], df[['categoria', 'subcategoria',
'resultado']], test_size=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(df['text_new'], df['resultado'].map({'Positivo':0,
'Negativo':1, 'Neutro':2}), test_size=0.2, random_state=42)

# Crear el vectorizador TF-IDF para representar los textos como características numéricas
vectorizer = TfidfVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Entrenar un clasificador SVM lineal
# classifier_cat = LinearSVC()
# classifier_cat.fit(X_train_vec, y_train['categoria']) # Utilizar las tres columnas de y_train o #
Utilizar solo la columna 'sentimiento' de y_train

# classifier_subcat = LinearSVC()
# classifier_subcat.fit(X_train_vec, y_train['subcategoria']) # Utilizar las tres columnas de
y_train o # Utilizar solo la columna 'sentimiento' de y_train

classifier_sent = LinearSVC()
classifier_sent.fit(X_train_vec, y_train) # Utilizar las tres columnas de y_train o # Utilizar solo
la columna 'sentimiento' de y_train

```

```

# Evaluar el modelo en el conjunto de prueba
# y_pred_cat = classifier_cat.predict(X_test_vec)
# y_pred_subcat = classifier_subcat.predict(X_test_vec)
y_pred_sent = classifier_sent.predict(X_test_vec)

# print("Reporte de clasificación para la variable 'categoria':")
# print(classification_report(y_test['categoria'], y_pred_cat))
# print("Reporte de clasificación para la variable 'subcategoria':")
# print(classification_report(y_test['subcategoria'], y_pred_subcat))
# print("Reporte de clasificación para la variable 'resultado':")
# print(classification_report(y_test['resultado'], y_pred_sent))

# Clasificar un nuevo texto
def classify_text(text):
    text_vec = vectorizer.transform([text])
    #predictions = classifier.predict(text_vec)
    # category = classifier_cat.predict(text_vec)[0]
    # subcategory = classifier_subcat.predict(text_vec)[0]
    sentiment = classifier_sent.predict(text_vec)[0]
    #sentiment = classifier.predict(text_vec)
    #category, subcategory, sentiment = classifier.predict(text_vec)
    return sentiment #category, subcategory,

# Ejemplo de clasificación de un nuevo texto
# new_text = "Este es un texto de prueba"
# category, subcategory, sentiment = classify_text(new_text)#category, subcategory,

# Verificar si hay valores NaN en resultado
print("Valores NaN en resultado:", y_train.isna().sum())

from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import LabelEncoder
from keras.utils import np_utils

X_train_dtm = vect.fit_transform(X_train)
X_test_dtm = vect.transform(X_test)

# Convertir las etiquetas de texto a numéricas
encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train)
y_test_encoded = encoder.transform(y_test)

# Aplicar SMOTE para el sobremuestreo en el conjunto de entrenamiento
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_dtm, y_train_encoded)

```

```

# Convertir las etiquetas sobremuestreadas a formato categórico para Keras
Y_train_smote = np_utils.to_categorical(y_train_smote, num_classes=3)
Y_test_dtm = np_utils.to_categorical(y_test_encoded, num_classes=3)

print("Tamaño de matriz X_Validación:")
display(X_train_smote.shape)
print("Tamaño de matriz Y_Validación:")
display(Y_train_smote.shape)

# Definir el modelo
model = Sequential()
model.add(Dense(units=128, input_shape=(X_train_smote.shape[1],)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(units=3))
model.add(Activation('softmax'))
model.summary()

# Compilar el modelo
model.compile(loss='categorical_crossentropy', optimizer='rmsprop', metrics=['accuracy'])

import numpy, scipy.sparse
spX_bd_dtm = scipy.sparse.csr_matrix(X_train_smote)
spY_bd_dtm = scipy.sparse.csr_matrix(Y_train_smote)
spX_bd_dtmTest = scipy.sparse.csr_matrix(X_test_dtm)
spY_bd_dtmTest = scipy.sparse.csr_matrix(Y_test_dtm)

spX_bd_dtm.sort_indices()
spY_bd_dtm.sort_indices()
spX_bd_dtmTest.sort_indices()
spY_bd_dtmTest.sort_indices()

spX_bd_dtm

X_train_smote

# Ajustar el modelo con los datos sobremuestreados
model.fit(X_train_smote, Y_train_smote, validation_data=(X_test_dtm, Y_test_dtm), epochs=1,
verbose=1, callbacks=[PlotLossesCallback()])

# Realiza las predicciones
predictions = model.predict(X_test_dtm)

ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)

```



```

# Confusion Matrix
print('Confusion Matrix epochs=1 con sobremuestreo')
print(' ')
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
# Classification Report
print('Classification Report epochs=1 con sobremuestreo')
print(' ')
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))

```

```

from tensorflow.keras.callbacks import EarlyStopping

```

```

# Configurar early stopping
early_stopping = EarlyStopping(
    monitor='val_loss', # Otra opción común es 'val_accuracy'
    patience=10,      # Número de epochs sin mejora tras los cuales se detiene el entrenamiento
    restore_best_weights=True # Restaura los pesos del modelo desde la epoch con el mejor valor
                        # de la métrica monitoreada.
)

```

```

# Incluir early stopping en los callbacks durante el entrenamiento
model.fit(
    X_train_smote, Y_train_smote,
    validation_data=(X_test_dtm, Y_test_dtm),
    epochs=50, # Puedes aumentar el número de epochs ya que early stopping puede detener el
               # entrenamiento antes
    verbose=1,
    callbacks=[PlotLossesCallback(), early_stopping]
)

```

Se detuvo en epoch = 11

```

# Ajustar el modelo con los datos sobremuestreados

```

```
#model.fit(X_train_smote, Y_train_smote, validation_data=(X_test_dtm, Y_test_dtm),
epochs=16, verbose=1, callbacks=[PlotLossesCallback()])
```

```
# Evaluate the model
scores = model.evaluate(X_test_dtm, Y_test_dtm)
print("\n%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
```

""""El modelo muestra una alta precisión tanto en el conjunto de entrenamiento como en el de validación, lo cual es favorable para la usabilidad del modelo.

```
####Epoch = 1
```

- La precisión de entrenamiento es de alrededor del 94.02%, mientras que la precisión de validación es del 94.52%. Esto sugiere que el modelo generaliza bien ya que la precisión en los datos de validación es incluso más alta que en los datos de entrenamiento.

- La curva de pérdida (loss) final fue de 0.1980 y la de validación de 0.2390, lo que indica un buen ajuste del modelo.

```
####Epoch = 11
```

- La precisión de entrenamiento es de alrededor del 94.02%, mientras que la precisión de validación es del 94.47%. Esto sugiere que el modelo generaliza bien ya que la precisión en los datos de validación es incluso más alta que en los datos de entrenamiento.

- La curva de pérdida (loss) final fue de 0.1985 y la de validación de 0.2431, lo que indica un buen ajuste del modelo a los datos sin señales evidentes de sobreajuste.

""""

```
# Realiza las predicciones
predictions = model.predict(X_test_dtm)
```

```
ynewClass = np.argmax(predictions, axis=1)
yoldClass = np.asarray(y_test)
```

```
# Confusion Matrix
```

```
print('Confusion Matrix Oversampling & early_stopping ')
print(' ')
```

```
cmtx = pd.DataFrame(
    confusion_matrix(yoldClass, ynewClass, labels=[0, 1, 2]),
    index=['True:Negativo', 'True:Positivo', 'True:Neutro'],
    columns=['Pred:Negativo', 'Pred:Positivo', 'Pred:Neutro'])
print(cmtx)
```

```
print(' ')
print('precision ↓')
print('recall →')
print('f1-score  $\bar{x}$ ')
print(' ')
```

```
# Classification Report
print('Classification Report Oversampling & early_stopping')
print(' ')
```

```
target_names = ['Negativo', 'Positivo', 'Neutro']
print(classification_report(yoldClass, ynewClass, target_names=target_names))
```

```
""" - La matriz de confusión muestra que el modelo predice bien la clase 'Positivo', pero tiene
dificultades con la clase 'Neutro', como lo indican los valores más bajos de precisión y recall para
esa clase. Esto podría deberse a menos ejemplos de entrenamiento para esa clase o a que los
textos neutros son más difíciles de diferenciar.
- Clasificación muestra que los valores de precisión, recall y f1-score son altos para la clase
'Positivo', pero la clase 'Neutro' muestra un rendimiento inferior. Esto sugiere que el modelo
podría beneficiarse de un enfoque más equilibrado para todas las clases.
"""
```

```
# Commented out IPython magic to ensure Python compatibility.
# %load_ext tensorboard
import datetime
```

```
# Directorio donde se almacenarán los logs
log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
```

```
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)
```

```
model.fit(
    X_train_smote,
    Y_train_smote,
    validation_data=(X_test_dtm, Y_test_dtm),
    epochs=10, # Ejemplo con más epochs
    verbose=1,
    callbacks=[tensorboard_callback]
)
```

```
# Inicia TensorBoard dentro del notebook
# %tensorboard --logdir logs/fit
```

Anexo IV. Resultados de Modelo 3 – BERT - Sentimiento

Durante el entrenamiento del modelo, se observaron mejoras constantes en la precisión y la puntuación F1 a lo largo de 10 épocas. En cada iteración, se registraron las pérdidas, la precisión y la puntuación F1. Por ejemplo, en la tercera época, la pérdida fue de 0.5972, con una precisión de 0.9308 y una puntuación F1 de 0.9339. Estas métricas continuaron mejorando hasta la décima época, donde la pérdida fue de 0.8539, la precisión alcanzó 0.9574 y la puntuación F1 fue de 0.9571. Se guardaron modelos y se generaron matrices de confusión y reportes de clasificación para evaluar el rendimiento en cada pliegue de la validación cruzada, mostrando una precisión y *recall* altos para las clases 'Negativo', 'Neutro' y 'Positivo'. La precisión ponderada y el *recall* fueron consistentemente altos, indicando un buen rendimiento general del modelo.

Epoch: 3.00/10

Loss: 0.5972

Accuracy: 0.9308

F1 Score: 0.9339

Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9308

Epoch: 4.00/10

Loss: 0.5087

Accuracy: 0.9537

F1 Score: 0.9540

Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9537

Epoch: 5.00/10

Loss: 0.6969

Accuracy: 0.9582

F1 Score: 0.9570

Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9582

Epoch: 6.00/10

Loss: 0.6635

Accuracy: 0.9523

F1 Score: 0.9526

Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9523

Epoch: 7.00/10

Loss: 0.7187

Accuracy: 0.9567

F1 Score: 0.9563
 Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9567
 Epoch: 8.00/10
 Loss: 0.7523
 Accuracy: 0.9553
 F1 Score: 0.9551
 Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9553
 Epoch: 9.00/10
 Loss: 0.8074
 Accuracy: 0.9563
 F1 Score: 0.9562
 Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9563
 Epoch: 10.00/10
 Loss: 0.8539
 Accuracy: 0.9574
 F1 Score: 0.9571
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9574
 <IPython.core.display.HTML object>
 Epoch: 10.00/10
 Loss: 0.6969
 Accuracy: 0.9582
 F1 Score: 0.9570
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9582
 Modelo guardado en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/model
 Finished training fold 1/5
 Matriz de Confusión:
 [[18413 33 178]
 [112 576 45]
 [520 44 2382]]
 Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_0.csv

Informe de Clasificación:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	18624
1	0.88	0.79	0.83	733
2	0.91	0.81	0.86	2946
accuracy			0.96	22303
macro avg	0.92	0.86	0.89	22303
weighted avg	0.96	0.96	0.96	22303

Matriz de Confusión:

[[18413 33 178]

```
[ 112  576  45]
[ 520  44 2382]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_0.csv

Informe de Clasificación:

	precision	recall	f1-score	support
Negativo	0.97	0.99	0.98	18624
Neutro	0.88	0.79	0.83	733
Positivo	0.91	0.81	0.86	2946
accuracy		0.96		22303
macro avg	0.92	0.86	0.89	22303
weighted avg	0.96	0.96	0.96	22303

Training fold 2/5

D:\Angelica Moreno\Proyecto de grado\proyecto\Lib\site-packages\accelerate\accelerator.py:436: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate: dict_keys(['dispatch_batches', 'split_batches', 'even_batches', 'use_seedable_sampler']). Please pass an `accelerate.DataLoaderConfiguration` instead: dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False, even_batches=True, use_seedable_sampler=True)

warnings.warn(You are adding a <class 'transformers.integrations.integration_utils.TensorBoardCallback'> to the callbacks of this Trainer, but there is already one. The current list of callbacks is

:DefaultFlowCallback

TensorBoardCallback

Iniciando el entrenamiento del modelo...

<IPython.core.display.HTML object>

Epoch: 1.00/10

Loss: 0.8122

Accuracy: 0.9304

F1 Score: 0.9248

Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.9304

Epoch: 2.00/10

Loss: 0.6659

Accuracy: 0.9502

F1 Score: 0.9478

Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9502

Epoch: 3.00/10

Loss: 0.6328

Accuracy: 0.9535

F1 Score: 0.9514

Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9535

Epoch: 4.00/10
Loss: 0.6601
Accuracy: 0.9557
F1 Score: 0.9544
Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9557
Epoch: 5.00/10
Loss: 0.6744
Accuracy: 0.9585
F1 Score: 0.9573
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9585
Epoch: 6.00/10
Loss: 0.6837
Accuracy: 0.9598
F1 Score: 0.9596
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9598
Epoch: 7.00/10
Loss: 0.8121
Accuracy: 0.9619
F1 Score: 0.9609
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9619
Epoch: 8.00/10
Loss: 0.7789
Accuracy: 0.9613
F1 Score: 0.9605
Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9613
Epoch: 9.00/10
Loss: 0.7557
Accuracy: 0.9607
F1 Score: 0.9605
Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9607
Epoch: 10.00/10
Loss: 0.8631
Accuracy: 0.9614
F1 Score: 0.9608
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9614
<IPython.core.display.HTML object>
Epoch: 10.00/10
Loss: 0.8121
Accuracy: 0.9619
F1 Score: 0.9609
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9619
Modelo guardado en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/model
Finished training fold 2/5
<Figure size 432x288 with 0 Axes>
Matriz de Confusión:

```
[[18423  17  181]
 [ 116 561  57]
 [ 464  15 2469]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_1.csv

Informe de Clasificación:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	18621
1	0.95	0.76	0.85	734
2	0.91	0.84	0.87	2948
accuracy			0.96	22303
macro avg	0.94	0.86	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Matriz de Confusión:

```
[[18423  17  181]
 [ 116 561  57]
 [ 464  15 2469]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_1.csv

Informe de Clasificación:

	precision	recall	f1-score	support
Negativo	0.97	0.99	0.98	18621
Neutro	0.95	0.76	0.85	734
Positivo	0.91	0.84	0.87	2948
accuracy			0.96	22303
macro avg	0.94	0.86	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Training fold 3/5

D:\Angelica Moreno\Proyecto de grado\proyecto\Lib\site-packages\accelerate\accelerator.py:436: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate: dict_keys(['dispatch_batches', 'split_batches', 'even_batches', 'use_seedable_sampler']). Please pass an `accelerate.DataLoaderConfiguration` instead:
dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False, even_batches=True, use_seedable_sampler=True)
warnings.warn(
You are adding a <class 'transformers.integrations.integration_utils.TensorBoardCallback'> to the callbacks of this Trainer, but there is already one. The current list of callbacks is


```
:DefaultFlowCallback
TensorBoardCallback
Iniciando el entrenamiento del modelo...
<IPython.core.display.HTML object>
Epoch: 1.00/10
  Loss: 0.5628
  Accuracy: 0.8875
  F1 Score: 0.8986
Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.8875
Epoch: 2.00/10
  Loss: 0.6200
  Accuracy: 0.9476
  F1 Score: 0.9468
Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9476
Epoch: 3.00/10
  Loss: 0.5661
  Accuracy: 0.9577
  F1 Score: 0.9569
Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9577
Epoch: 4.00/10
  Loss: 0.6783
  Accuracy: 0.9587
  F1 Score: 0.9576
Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9587
Epoch: 5.00/10
  Loss: 0.6744
  Accuracy: 0.9604
  F1 Score: 0.9598
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9604
Epoch: 6.00/10
  Loss: 0.6236
  Accuracy: 0.9580
  F1 Score: 0.9581
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9580
Epoch: 7.00/10
  Loss: 0.6973
  Accuracy: 0.9627
  F1 Score: 0.9620
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9627
Epoch: 8.00/10
  Loss: 0.8673
  Accuracy: 0.9616
  F1 Score: 0.9607
Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9616
Epoch: 9.00/10
  Loss: 0.7666
```

Accuracy: 0.9593
 F1 Score: 0.9592
 Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9593
 Epoch: 10.00/10
 Loss: 0.8882
 Accuracy: 0.9621
 F1 Score: 0.9615
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9621
 <IPython.core.display.HTML object>
 Epoch: 10.00/10
 Loss: 0.6973
 Accuracy: 0.9627
 F1 Score: 0.9620
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9627
 Modelo guardado en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/model
 Finished training fold 3/5
 <Figure size 432x288 with 0 Axes>
 Matriz de Confusión:
 [[18262 20 228]
 [127 596 40]
 [396 22 2612]]
 Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_2.csv

Informe de Clasificación:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	18510
1	0.93	0.78	0.85	763
2	0.91	0.86	0.88	3030
accuracy			0.96	22303
macro avg	0.94	0.88	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Matriz de Confusión:

[[18262 20 228]
 [127 596 40]
 [396 22 2612]]
 Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_2.csv

Informe de Clasificación:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Negativo	0.97	0.99	0.98	18510
Neutro	0.93	0.78	0.85	763
Positivo	0.91	0.86	0.88	3030
accuracy		0.96		22303
macro avg	0.94	0.88	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Training fold 4/5

D:\Angelica Moreno\Proyecto de grado\proyecto\Lib\site-packages\accelerate\accelerator.py:436: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate: dict_keys(['dispatch_batches', 'split_batches', 'even_batches', 'use_seedable_sampler']). Please pass an `accelerate.DataLoaderConfiguration` instead:
 dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False, even_batches=True, use_seedable_sampler=True)

warnings.warn(
 You are adding a <class 'transformers.integrations.integration_utils.TensorBoardCallback'> to the callbacks of this Trainer, but there is already one. The current list of callbacks is
 :DefaultFlowCallback

TensorBoardCallback

Iniciando el entrenamiento del modelo...

<IPython.core.display.HTML object>

Epoch: 1.00/10

Loss: 0.6404

Accuracy: 0.9288

F1 Score: 0.9236

Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.9288

Epoch: 2.00/10

Loss: 0.6300

Accuracy: 0.9506

F1 Score: 0.9497

Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9506

Epoch: 3.00/10

Loss: 0.5484

Accuracy: 0.9582

F1 Score: 0.9568

Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9582

Epoch: 4.00/10

Loss: 0.6575

Accuracy: 0.9557

F1 Score: 0.9549

Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9557

Epoch: 5.00/10

Loss: 0.7325

Accuracy: 0.9591

F1 Score: 0.9584
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9591
Epoch: 6.00/10
Loss: 0.6037
Accuracy: 0.9595
F1 Score: 0.9590
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9595
Epoch: 7.00/10
Loss: 0.7999
Accuracy: 0.9590
F1 Score: 0.9588
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9590
Epoch: 8.00/10
Loss: 0.8668
Accuracy: 0.9601
F1 Score: 0.9596
Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9601
Epoch: 9.00/10
Loss: 0.8904
Accuracy: 0.9590
F1 Score: 0.9587
Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9590
Epoch: 10.00/10
Loss: 0.9256
Accuracy: 0.9598
F1 Score: 0.9594
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9598
<IPython.core.display.HTML object>
Epoch: 10.00/10
Loss: 0.8668
Accuracy: 0.9601
F1 Score: 0.9596
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9601
Modelo guardado en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/model
Finished training fold 4/5
<Figure size 432x288 with 0 Axes>
Matriz de Confusión:
[[18246 24 276]
 [116 603 48]
 [394 31 2565]]
Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_3.csv

Informe de Clasificación:
precision recall f1-score support

0	0.97	0.98	0.98	18546
1	0.92	0.79	0.85	767
2	0.89	0.86	0.87	2990

accuracy			0.96	22303
macro avg	0.93	0.88	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Matriz de Confusión:

```
[[18246  24  276]
 [ 116 603  48]
 [ 394  31 2565]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_3.csv

Informe de Clasificación:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Negativo	0.97	0.98	0.98	18546
Neutro	0.92	0.79	0.85	767
Positivo	0.89	0.86	0.87	2990

accuracy			0.96	22303
macro avg	0.93	0.88	0.90	22303
weighted avg	0.96	0.96	0.96	22303

Training fold 5/5

D:\Angelica Moreno\Proyecto de grado\proyecto\Lib\site-

packages\accelerate\accelerator.py:436: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate:

dict_keys(['dispatch_batches', 'split_batches', 'even_batches', 'use_seedable_sampler']). Please pass an `accelerate.DataLoaderConfiguration` instead:

```
dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False,
even_batches=True, use_seedable_sampler=True)
```

```
warnings.warn(
```

You are adding a <class 'transformers.integrations.integration_utils.TensorBoardCallback'> to the callbacks of this Trainer, but there is already one. The currentlist of callbacks is

```
:DefaultFlowCallback
```

```
TensorBoardCallback
```

Iniciando el entrenamiento del modelo...

<IPython.core.display.HTML object>

Epoch: 1.00/10

Loss: 0.5011

Accuracy: 0.9109

F1 Score: 0.9151

Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.9109
Epoch: 2.00/10
Loss: 0.7533
Accuracy: 0.9470
F1 Score: 0.9448
Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9470
Epoch: 3.00/10
Loss: 0.6478
Accuracy: 0.9554
F1 Score: 0.9550
Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9554
Epoch: 4.00/10
Loss: 0.8225
Accuracy: 0.9581
F1 Score: 0.9564
Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9581
Epoch: 5.00/10
Loss: 0.6726
Accuracy: 0.9598
F1 Score: 0.9589
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9598
Epoch: 6.00/10
Loss: 0.6523
Accuracy: 0.9628
F1 Score: 0.9620
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9628
Epoch: 7.00/10
Loss: 0.6574
Accuracy: 0.9597
F1 Score: 0.9598
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9597
Epoch: 8.00/10
Loss: 0.7897
Accuracy: 0.9620
F1 Score: 0.9617
Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9620
Epoch: 9.00/10
Loss: 0.8293
Accuracy: 0.9621
F1 Score: 0.9617
Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9621
Epoch: 10.00/10
Loss: 0.9170
Accuracy: 0.9614
F1 Score: 0.9610
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9614

<IPython.core.display.HTML object>

Epoch: 10.00/10

Loss: 0.6523

Accuracy: 0.9628

F1 Score: 0.9620

Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9628

Modelo guardado en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/model

Finished training fold 5/5

<Figure size 432x288 with 0 Axes>

Matriz de Confusión:

```
[[18382  53  175]
```

```
 [ 99 575  43]
```

```
 [ 436  24 2515]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_4.csv

Informe de Clasificación:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.97	0.99	0.98	18610
---	------	------	------	-------

1	0.88	0.80	0.84	717
---	------	------	------	-----

2	0.92	0.85	0.88	2975
---	------	------	------	------

accuracy			0.96	22302
----------	--	--	------	-------

macro avg	0.92	0.88	0.90	22302
-----------	------	------	------	-------

weighted avg	0.96	0.96	0.96	22302
--------------	------	------	------	-------

Matriz de Confusión:

```
[[18382  53  175]
```

```
 [ 99 575  43]
```

```
 [ 436  24 2515]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_4.csv

Informe de Clasificación:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Negativo	0.97	0.99	0.98	18610
----------	------	------	------	-------

Neutro	0.88	0.80	0.84	717
--------	------	------	------	-----

Positivo	0.92	0.85	0.88	2975
----------	------	------	------	------

accuracy			0.96	22302
----------	--	--	------	-------

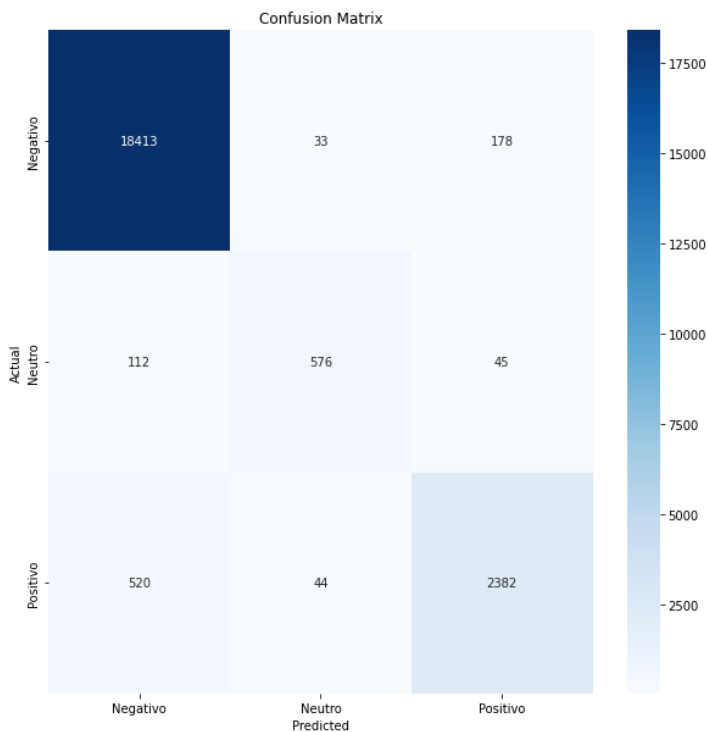
macro avg	0.92	0.88	0.90	22302
-----------	------	------	------	-------

weighted avg	0.96	0.96	0.96	22302
--------------	------	------	------	-------

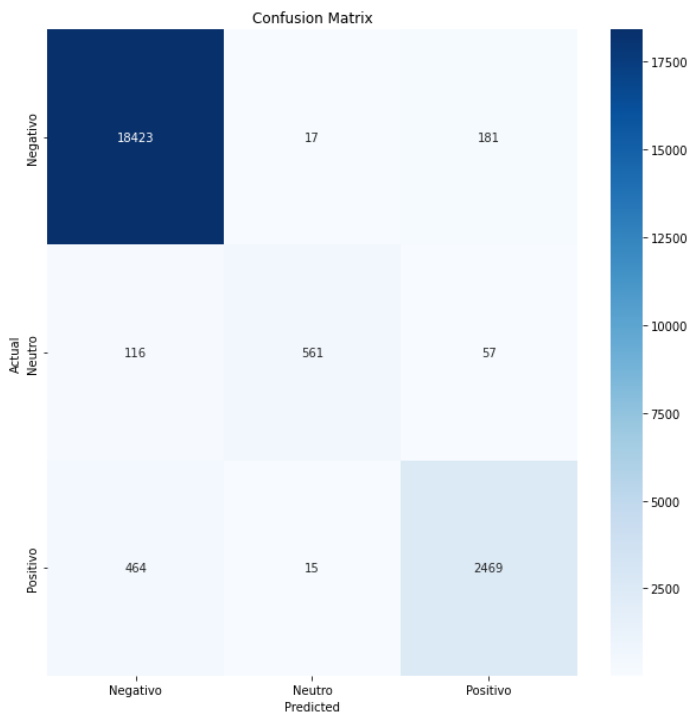
Anexo V. Resultados de Modelo 3 – BERT – Sentimiento – Matrices y Curvas ROC

Durante el proceso de entrenamiento y evaluación del modelo BERT para el análisis de sentimiento, se generaron múltiples matrices de confusión y curvas ROC para cada pliegue de la validación cruzada. Las matrices de confusión revelaron un alto nivel de precisión en la clasificación de sentimientos negativos, neutros y positivos, con la clase 'Negativo' mostrando una precisión y *recall* particularmente altos en todos los pliegues. Las curvas ROC demostraron la capacidad del modelo para distinguir entre las diferentes clases con un área bajo la curva (AUC) elevada para cada clase, confirmando la robustez del modelo. Estas métricas indican que el modelo BERT es eficaz y fiable para la tarea de análisis de sentimiento, mostrando un buen balance entre precisión y *recall* y una fuerte capacidad de discriminación entre las clases.

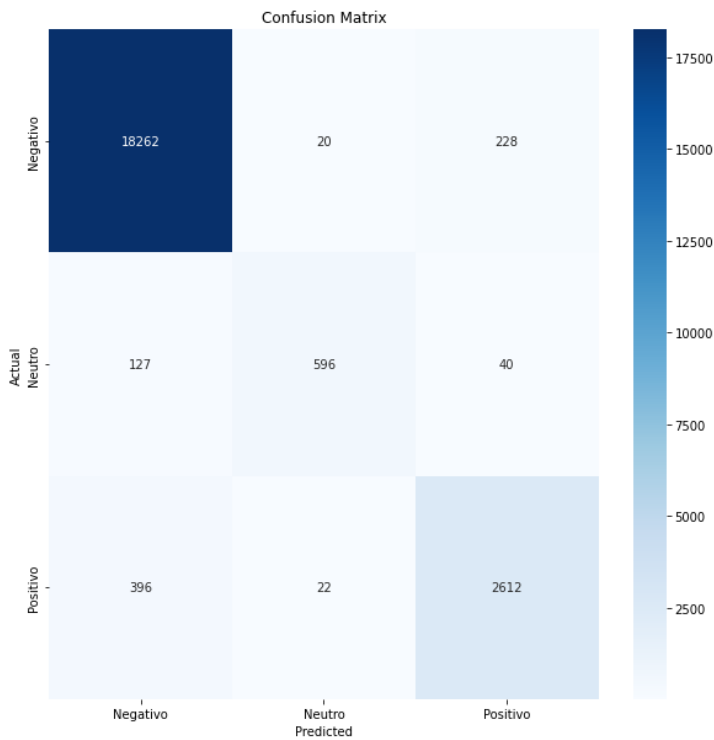
Confusion_Matrix_Fold_0



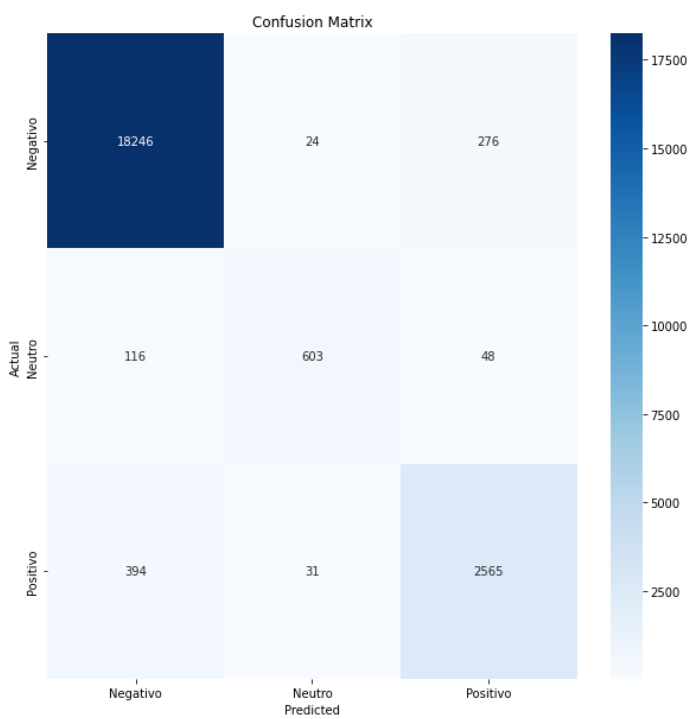
Confusion_Matrix_Fold_1



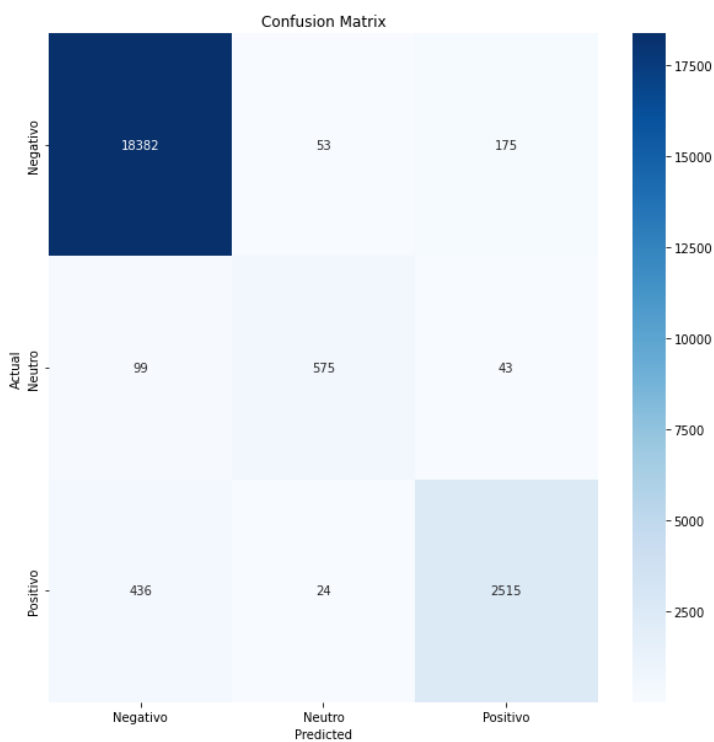
Confusion_Matrix_Fold_2



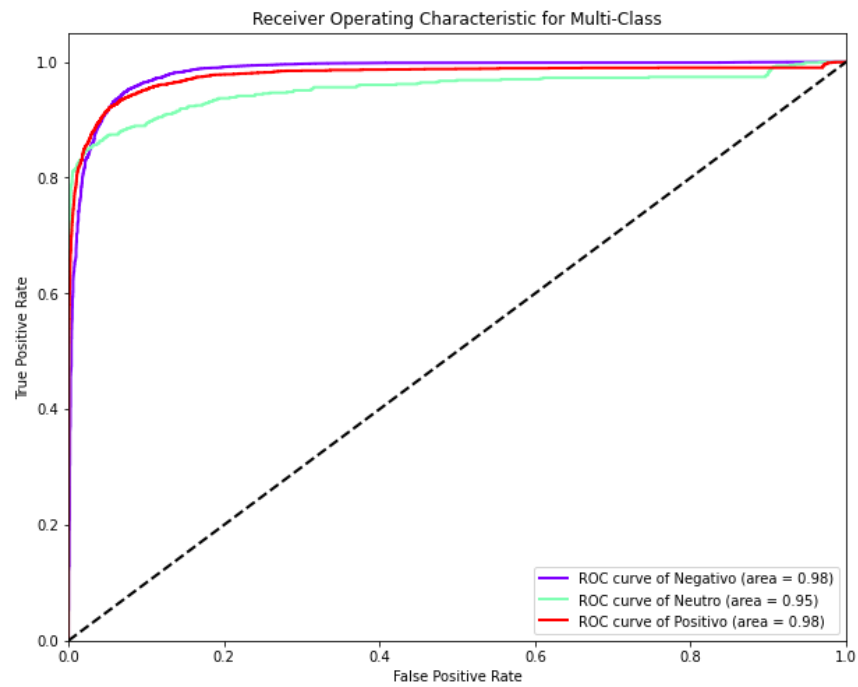
Confusion_Matrix_Fold_3



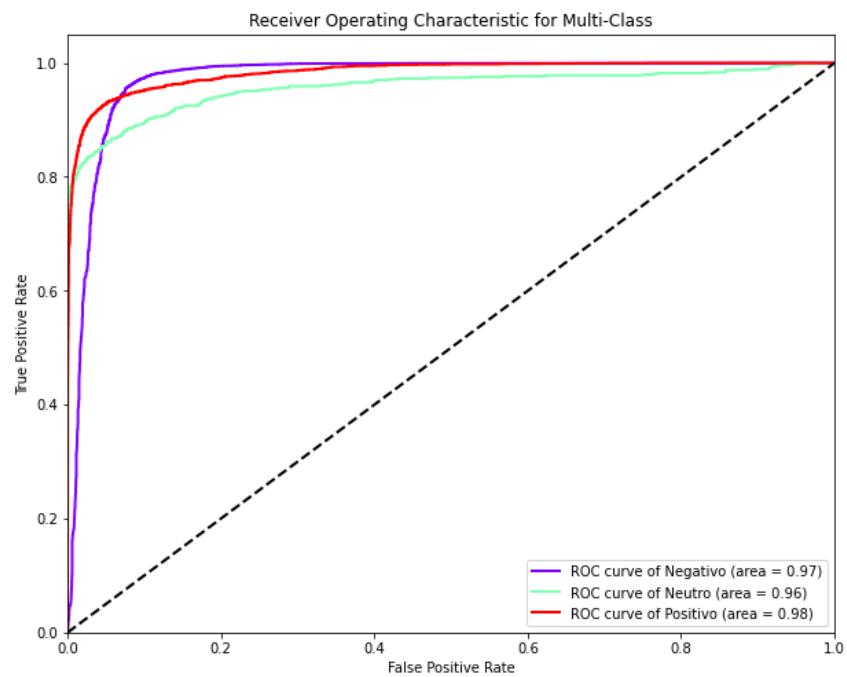
Confusion_Matrix_Fold_4



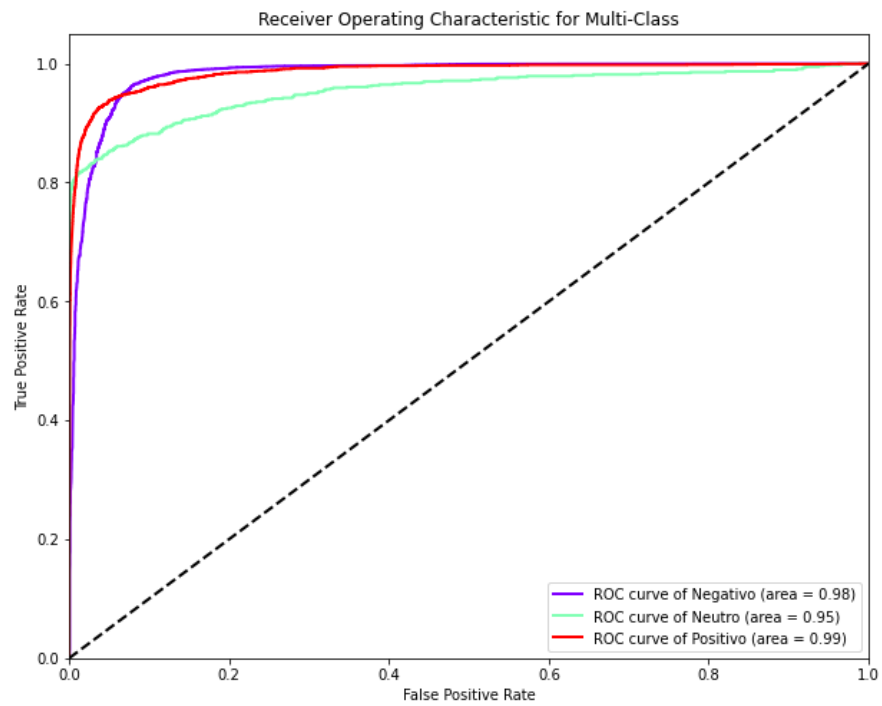
ROC_Curves_Fold_0



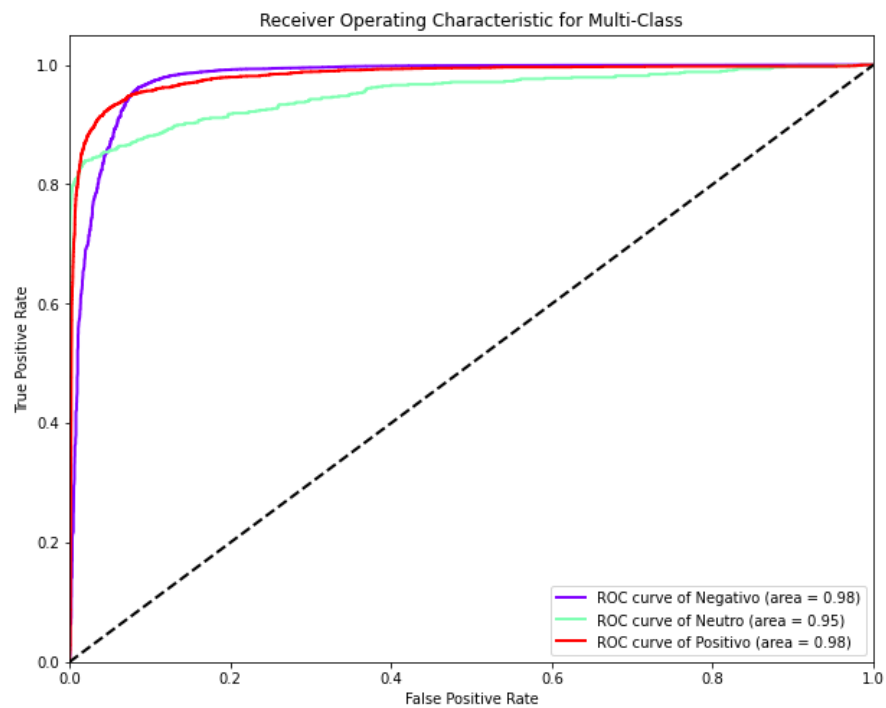
ROC_Curves_Fold_1



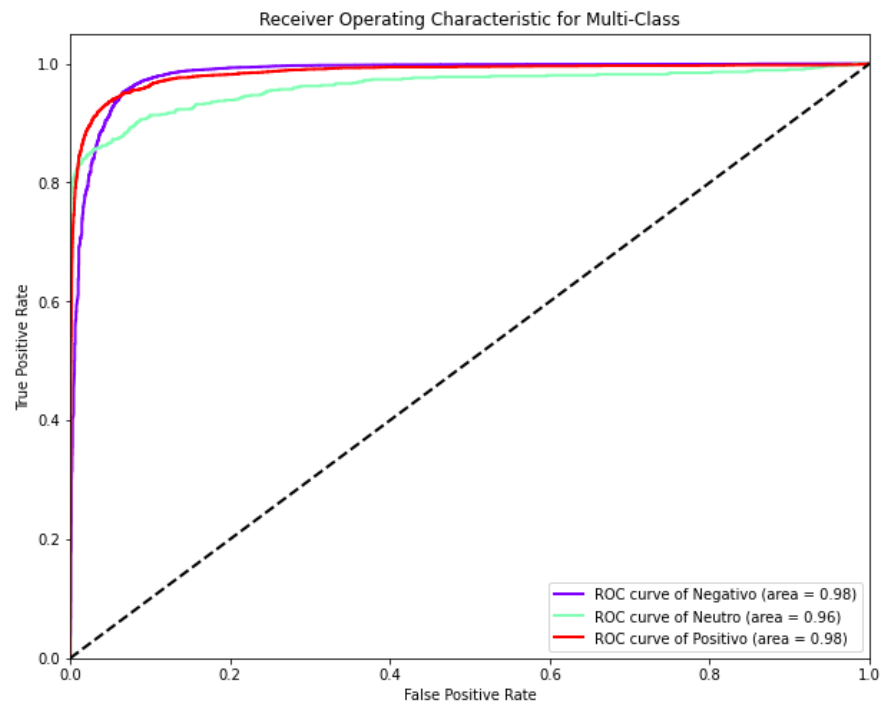
ROC_Curves_Fold_2



ROC_Curves_Fold_3



ROC_Curves_Fold_4



Anexo VI. Resultados de Modelo 3 – BERT – PQRS

Durante el proceso de entrenamiento y evaluación del modelo BERT para el análisis de sentimiento, se generaron múltiples matrices de confusión y curvas ROC para cada pliegue de la validación cruzada. Las matrices de confusión revelaron un alto nivel de precisión en la clasificación de sentimientos negativos, neutros y positivos, con la clase 'Negativo' mostrando una precisión y *recall* particularmente altos en todos los pliegues. Las curvas ROC demostraron la capacidad del modelo para distinguir entre las diferentes clases con un área bajo la curva (AUC) elevada para cada clase, confirmando la robustez del modelo. Estas métricas indican que el modelo BERT es eficaz y fiable para la tarea de análisis de sentimiento, mostrando un buen balance entre precisión y *recall* y una fuerte capacidad de discriminación entre las clases.

```
:DefaultFlowCallback
TensorBoardCallback
Iniciando el entrenamiento del modelo...
<IPython.core.display.HTML object>
Epoch: 1.00/10
  Loss: 0.6100
  Accuracy: 0.8373
  F1 Score: 0.8400
Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.8373
Epoch: 2.00/10
  Loss: 0.3450
  Accuracy: 0.9464
  F1 Score: 0.9464
Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9464
Epoch: 3.00/10
  Loss: 0.3272
  Accuracy: 0.9592
  F1 Score: 0.9592
Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9592
Epoch: 4.00/10
  Loss: 0.2866
  Accuracy: 0.9640
  F1 Score: 0.9642
Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9640
Epoch: 5.00/10
```

Loss: 0.2967
Accuracy: 0.9712
F1 Score: 0.9712
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9712
Epoch: 6.00/10
Loss: 0.2953
Accuracy: 0.9715
F1 Score: 0.9715
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9715
Epoch: 7.00/10
Loss: 0.2951
Accuracy: 0.9729
F1 Score: 0.9729
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9729
Epoch: 8.00/10
Loss: 0.3235
Accuracy: 0.9761
F1 Score: 0.9761
Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9761
Epoch: 9.00/10
Loss: 0.3307
Accuracy: 0.9767
F1 Score: 0.9767
Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9767
Epoch: 10.00/10
Loss: 0.3292
Accuracy: 0.9768
F1 Score: 0.9767
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9768
<IPython.core.display.HTML object>
Epoch: 10.00/10
Loss: 0.3292
Accuracy: 0.9768
F1 Score: 0.9767
Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9768
Finished training fold 4/5
<Figure size 432x288 with 0 Axes>
Matriz de Confusión:
[[250 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 2 0 0 0 0 0 0 2 0 0 0 0 0]
 [0 149 0 0 1 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 0]
 [0 0 168 0 0 0 0 2 0 0 0 0 0 0 0
 0 0 0 0 1 0 0 0 0 0 0 0 0 0]
 [1 0 0 265 2 0 0 2 1 0 0 0 0 0 0
 0 0 0 0 2 0 0 0 0 0 0 0 0 0]]

```

[ 1 1 0 0 382 0 1 3 2 0 0 0 0 0
0 3 1 0 1 0 0 0 0 1 0 0 0]
[ 0 0 1 0 0 69 0 3 0 0 0 0 0 1
0 0 3 0 0 1 0 0 0 2 0 0 1]
[ 0 1 0 0 1 0 113 2 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 2 0]
[ 0 1 1 2 6 0 0 3336 9 0 1 1 1 0
1 8 30 2 14 5 1 0 3 13 2 9 0]
[ 1 0 0 1 2 0 1 91995 0 0 0 0 0
0 3 8 0 2 0 0 0 0 3 0 4 0]
[ 0 0 0 0 0 0 0 1 0 188 0 1 0 0
0 1 2 0 2 0 1 0 1 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 79 0 0 0
0 0 0 0 1 0 0 0 1 1 0 1 0]
[ 0 0 1 0 0 0 0 2 0 0 0 292 0 0
0 2 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 61 0
0 0 2 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 89
0 0 1 0 0 1 0 0 0 2 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
298 0 1 0 0 0 0 0 0 0 0 0 0]
[ 1 0 0 0 4 0 0 6 3 1 0 2 1 0
1 2296 7 1 5 1 2 2 0 1 0 2 0]
[ 0 0 0 0 2 0 0 23 4 1 0 0 0 0
0 4 3118 0 10 0 0 0 0 6 0 10 0]
[ 0 0 0 0 0 0 0 3 0 0 0 0 0 0
0 0 3 507 0 0 0 1 2 0 0 1 0]
[ 0 0 2 0 2 0 1 11 5 1 1 1 0 0
0 2 23 2 2481 0 1 1 2 12 0 1 0]
[ 1 0 0 0 0 0 1 4 1 0 0 0 0 0
1 0 0 0 0 336 0 0 0 1 0 2 0]
[ 0 0 0 0 0 0 0 3 0 0 0 0 0 0
0 0 2 0 2 0 173 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
0 0 1 0 2 2 0 367 1 1 0 0 0]
[ 0 0 0 1 0 0 0 3 2 0 0 1 0 0
0 0 4 1 0 1 0 0 777 1 0 1 1]
[ 2 0 1 1 0 0 0 6 4 0 0 1 0 0
0 1 5 0 3 0 0 1 2 1724 0 0 2]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 107 0 0]
[ 1 1 0 0 0 0 0 6 1 0 0 0 1 1
0 6 10 0 2 1 1 0 2 3 12117 0]
[ 0 1 0 0 1 0 0 5 0 0 0 0 1 0
0 1 0 0 1 0 0 1 0 1 0 1 48]]

```


Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_3.csv

Informe de Clasificación:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	254
1	0.97	0.99	0.98	151
2	0.97	0.98	0.97	171
3	0.98	0.97	0.98	273
4	0.95	0.96	0.96	396
5	1.00	0.85	0.92	81
6	0.97	0.95	0.96	119
7	0.97	0.97	0.97	3446
8	0.98	0.98	0.98	2029
9	0.98	0.95	0.97	197
10	0.98	0.95	0.96	83
11	0.98	0.98	0.98	297
12	0.94	0.97	0.95	63
13	0.98	0.96	0.97	93
14	0.99	1.00	0.99	299
15	0.99	0.98	0.98	2336
16	0.97	0.98	0.97	3178
17	0.99	0.98	0.98	517
18	0.98	0.97	0.98	2549
19	0.97	0.97	0.97	347
20	0.97	0.96	0.96	180
21	0.98	0.98	0.98	376
22	0.98	0.98	0.98	793
23	0.97	0.98	0.98	1753
24	0.97	1.00	0.99	107
25	0.98	0.98	0.98	2154
26	0.92	0.79	0.85	61
accuracy			0.98	22303
macro avg	0.97	0.96	0.97	22303
weighted avg	0.98	0.98	0.98	22303

Matriz de Confusión:

```
[[ 250  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  2  0  0  0  0  0  2  0  0  0  0  0]
 [  0 149  0  0  1  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0  0  1  0]
 [  0  0 168  0  0  0  0  2  0  0  0  0  0  0  0
   0  0  0  0  1  0  0  0  0  0  0  0  0]
 [  1  0  0 265  2  0  0  2  1  0  0  0  0  0  0]
```

```

0 0 0 0 2 0 0 0 0 0 0 0 0 0]
[ 1 1 0 0 382 0 1 3 2 0 0 0 0 0
0 3 1 0 1 0 0 0 0 1 0 0 0 0]
[ 0 0 1 0 0 69 0 3 0 0 0 0 0 1
0 0 3 0 0 1 0 0 0 2 0 0 0 1]
[ 0 1 0 0 1 0 113 2 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 2 0]
[ 0 1 1 2 6 0 0 3336 9 0 1 1 1 0
1 8 30 2 14 5 1 0 3 13 2 9 0]
[ 1 0 0 1 2 0 1 9 1995 0 0 0 0 0
0 3 8 0 2 0 0 0 0 3 0 4 0]
[ 0 0 0 0 0 0 0 1 0 188 0 1 0 0
0 1 2 0 2 0 1 0 1 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 79 0 0 0
0 0 0 0 1 0 0 0 1 1 0 1 0]
[ 0 0 1 0 0 0 0 2 0 0 0 292 0 0
0 2 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 61 0
0 0 2 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 89
0 0 1 0 0 1 0 0 0 2 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
298 0 1 0 0 0 0 0 0 0 0 0 0]
[ 1 0 0 0 4 0 0 6 3 1 0 2 1 0
1 2296 7 1 5 1 2 2 0 1 0 2 0]
[ 0 0 0 0 2 0 0 23 4 1 0 0 0 0
0 4 3118 0 10 0 0 0 0 6 0 10 0]
[ 0 0 0 0 0 0 0 3 0 0 0 0 0 0
0 0 3 507 0 0 0 1 2 0 0 1 0]
[ 0 0 2 0 2 0 1 11 5 1 1 1 0 0
0 2 23 2 2481 0 1 1 2 12 0 1 0]
[ 1 0 0 0 0 0 1 4 1 0 0 0 0 0
1 0 0 0 0 336 0 0 0 1 0 2 0]
[ 0 0 0 0 0 0 0 3 0 0 0 0 0 0
0 0 2 0 2 0 173 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
0 0 1 0 2 2 0 367 1 1 0 0 0]
[ 0 0 0 1 0 0 0 3 2 0 0 1 0 0
0 0 4 1 0 1 0 0 777 1 0 1 1]
[ 2 0 1 1 0 0 0 6 4 0 0 1 0 0
0 1 5 0 3 0 0 1 2 1724 0 0 2]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 107 0 0]
[ 1 1 0 0 0 0 0 6 1 0 0 0 1 1
0 6 10 0 2 1 1 0 2 3 12117 0]
[ 0 1 0 0 1 0 0 5 0 0 0 0 1 0

```

0 1 0 0 1 0 0 1 0 1 0 1 48]]

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_3.csv

Informe de Clasificación:

	precision	recall	f1-score	support
Aplicación	0.97	0.98	0.98	254
Cajeros	0.97	0.99	0.98	151
Cancelación de Productos	0.97	0.98	0.97	171
Características	0.98	0.97	0.98	273
Cobros injustificados	0.95	0.96	0.96	396
Comentario no asociado a servicio	1.00	0.85	0.92	81
Competencia	0.97	0.95	0.96	119
Críticas	0.97	0.97	0.97	3446
Desconfianza	0.98	0.98	0.98	2029
Entrega	0.98	0.95	0.97	197
Fallas en la plataforma	0.98	0.95	0.96	83
Grupo Aval	0.98	0.98	0.98	297
Información sobre espacios de atención	0.94	0.97	0.95	63
Interés	0.98	0.96	0.97	93
Luis Carlos Sarmiento Angulo	0.99	1.00	0.99	299
Marca	0.99	0.98	0.98	2336
No identificado	0.97	0.98	0.97	3178
Oficina	0.99	0.98	0.98	517
Procesos de Solicitud	0.98	0.97	0.98	2549
Página Web	0.97	0.97	0.97	347
Reacciones frente al mismo	0.97	0.96	0.96	180
Respuesta	0.98	0.98	0.98	376
Servilínea	0.98	0.98	0.98	793
Solicitud Específica	0.97	0.98	0.98	1753
Solicitud de nuevos productos	0.97	1.00	0.99	107
Temas externos	0.98	0.98	0.98	2154
Whatsapp	0.92	0.79	0.85	61
accuracy		0.98		22303
macro avg	0.97	0.96	0.97	22303
weighted avg	0.98	0.98	0.98	22303

Training fold 5/5

D:\Angelica Moreno\Proyecto de grado\proyecto\Lib\site-packages\accelerate\accelerator.py:436: FutureWarning: Passing the following arguments to `Accelerator` is deprecated and will be removed in version 1.0 of Accelerate: dict_keys(['dispatch_batches', 'split_batches', 'even_batches', 'use_seedable_sampler']). Please pass an `accelerate.DataLoaderConfiguration` instead:

```
dataloader_config = DataLoaderConfiguration(dispatch_batches=None, split_batches=False,
even_batches=True, use_seedable_sampler=True)
warnings.warn(
You are adding a <class 'transformers.integrations.integration_utils.TensorBoardCallback'> to
the callbacks of this Trainer, but there is already one. The currentlist of callbacks is
:DefaultFlowCallback
TensorBoardCallback
Iniciando el entrenamiento del modelo...
<IPython.core.display.HTML object>
Epoch: 1.00/10
  Loss: 0.6899
  Accuracy: 0.8135
  F1 Score: 0.8156
Callback de evaluación llamado. Epoch: 1.0 - Eval Accuracy: 0.8135
Epoch: 2.00/10
  Loss: 0.3474
  Accuracy: 0.9446
  F1 Score: 0.9446
Callback de evaluación llamado. Epoch: 2.0 - Eval Accuracy: 0.9446
Epoch: 3.00/10
  Loss: 0.2818
  Accuracy: 0.9619
  F1 Score: 0.9620
Callback de evaluación llamado. Epoch: 3.0 - Eval Accuracy: 0.9619
Epoch: 4.00/10
  Loss: 0.3172
  Accuracy: 0.9682
  F1 Score: 0.9681
Callback de evaluación llamado. Epoch: 4.0 - Eval Accuracy: 0.9682
Epoch: 5.00/10
  Loss: 0.3131
  Accuracy: 0.9690
  F1 Score: 0.9689
Callback de evaluación llamado. Epoch: 5.0 - Eval Accuracy: 0.9690
Epoch: 6.00/10
  Loss: 0.3207
  Accuracy: 0.9709
  F1 Score: 0.9708
Callback de evaluación llamado. Epoch: 6.0 - Eval Accuracy: 0.9709
Epoch: 7.00/10
  Loss: 0.3261
  Accuracy: 0.9740
  F1 Score: 0.9740
Callback de evaluación llamado. Epoch: 7.0 - Eval Accuracy: 0.9740
Epoch: 8.00/10
  Loss: 0.3164
```

Accuracy: 0.9766
 F1 Score: 0.9766
 Callback de evaluación llamado. Epoch: 8.0 - Eval Accuracy: 0.9766
 Epoch: 9.00/10
 Loss: 0.3056
 Accuracy: 0.9765
 F1 Score: 0.9765
 Callback de evaluación llamado. Epoch: 9.0 - Eval Accuracy: 0.9765
 Epoch: 10.00/10
 Loss: 0.3272
 Accuracy: 0.9771
 F1 Score: 0.9771
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9771
 <IPython.core.display.HTML object>
 Epoch: 10.00/10
 Loss: 0.3272
 Accuracy: 0.9771
 F1 Score: 0.9771
 Callback de evaluación llamado. Epoch: 10.0 - Eval Accuracy: 0.9771
 Finished training fold 5/5
 <Figure size 432x288 with 0 Axes>

Matriz de Confusión:

```

[[ 225  0  0  0  0  0  0  1  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  1  1  0  0  0]
 [  0 171  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [  0  0 170  0  0  0  0  1  0  0  0  0  0  1  0
   0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [  0  0  0 268  0  0  0  1  1  0  0  0  0  0  0
   0  1  2  0  2  0  0  0  0  0  0  0  0  0]
 [  0  0  1  0 383  0  0  2  2  0  0  0  0  0  0
   0  1  3  0  2  0  0  1  1  3  0  1  0  0]
 [  1  0  0  0  0  83  0  2  0  0  0  0  0  0  1
   0  0  0  0  0  0  0  0  1  2  0  0  0  0]
 [  0  0  0  0  0  0  87  1  0  0  0  0  0  0  0
   0  1  2  0  2  0  0  0  0  0  0  0  2  0]
 [  3  2  0  2  4  1  2 3370  8  1  0  1  1  0
   2 10 42  0 12  3  1  0  4  9  0 10  0]
 [  0  0  0  1  1  0  0  4 1983  0  0  0  0  0
   0  2  7  0  3  0  0  0  0  3  0  0  0  0]
 [  0  0  0  0  0  0  0  0  0  0 167  0  0  0  0
   0  0  0  0  2  0  0  1  1  0  0  1  0  0]
 [  0  0  0  0  0  0  0  0  1  0  69  0  0  0  0
   0  0  1  0  1  2  0  0  1  2  0  1  0  0]
 [  0  0  0  0  0  0  0  0  0  0  0 262  0  0  0
   1  0  1  0  0  0  0  0  0  0  0  1  0  0]

```

```
[ 1 0 0 0 0 0 0 1 0 0 0 0 55 0
 0 0 0 0 0 0 0 0 0 2 0 0 0]
[ 0 0 0 0 0 0 0 0 1 0 0 0 0 91
 0 0 0 0 0 0 0 0 0 1 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 270 0 2 0 0 0 0 1 0 0 0 0 0]
[ 0 0 0 0 2 0 1 5 2 0 0 0 0 0
 0 2313 7 0 6 0 0 1 1 5 0 4 0]
[ 3 0 0 0 2 0 0 26 3 1 0 0 1 0
 0 6 3130 0 10 3 2 0 0 5 1 4 1]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
 0 0 1 474 0 0 0 0 1 0 0 1 0]
[ 1 2 1 0 4 0 1 8 2 1 0 1 0 0
 0 2 14 0 2457 2 1 0 1 4 1 12 1]
[ 2 0 0 1 0 0 1 0 1 0 0 0 0 0
 0 2 2 0 2 351 0 0 0 2 0 1 0]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
 0 0 3 0 0 0 216 0 0 1 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 3 0 0 388 0 0 0 0 0]
[ 1 0 0 0 0 0 0 3 0 0 0 0 0 0
 0 3 5 0 1 0 1 0 782 3 0 0 0]
[ 0 0 0 0 0 0 0 8 2 0 0 0 0 1
 0 5 12 0 6 0 0 1 3 1738 0 3 1]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 3 119 1 0]
[ 1 0 0 0 2 0 2 7 1 0 0 1 0 1
 2 8 7 1 3 0 0 0 2 3 0 2111 0]
[ 0 0 0 1 0 0 0 0 1 0 0 0 0 0
 0 1 3 0 1 0 0 0 0 0 0 2 58]]
```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_4.csv

Informe de Clasificación:

	precision	recall	f1-score	support
0	0.95	0.99	0.97	228
1	0.98	1.00	0.99	171
2	0.99	0.99	0.99	172
3	0.98	0.97	0.98	275
4	0.96	0.96	0.96	400
5	0.99	0.92	0.95	90
6	0.92	0.92	0.92	95
7	0.98	0.97	0.97	3488
8	0.99	0.99	0.99	2004
9	0.98	0.97	0.98	172

10	1.00	0.88	0.94	78
11	0.99	0.99	0.99	265
12	0.95	0.93	0.94	59
13	0.97	0.98	0.97	93
14	0.98	0.99	0.99	273
15	0.98	0.99	0.98	2347
16	0.96	0.98	0.97	3198
17	1.00	0.99	0.99	479
18	0.98	0.98	0.98	2516
19	0.97	0.96	0.97	365
20	0.98	0.97	0.98	222
21	0.99	0.99	0.99	391
22	0.98	0.98	0.98	799
23	0.97	0.98	0.97	1780
24	0.98	0.97	0.98	123
25	0.98	0.98	0.98	2152
26	0.95	0.87	0.91	67

accuracy		0.98	22302
macro avg	0.97	0.97	0.97 22302
weighted avg	0.98	0.98	0.98 22302

Matriz de Confusión:

```
[[ 225  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  1  1  0  0  0  0]
 [  0 171  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [  0  0 170  0  0  0  0  0  1  0  0  0  0  0  1  0
   0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [  0  0  0 268  0  0  0  1  1  0  0  0  0  0  0  0
   0  1  2  0  2  0  0  0  0  0  0  0  0  0  0]
 [  0  0  1  0 383  0  0  2  2  0  0  0  0  0  0  0
   0  1  3  0  2  0  0  1  1  3  0  1  0  0  0]
 [  1  0  0  0  0 83  0  2  0  0  0  0  0  0  0  1
   0  0  0  0  0  0  0  0  0  1  2  0  0  0  0]
 [  0  0  0  0  0  0 87  1  0  0  0  0  0  0  0  0
   0  1  2  0  2  0  0  0  0  0  0  0  2  0  0]
 [  3  2  0  2  4  1  2 3370  8  1  0  1  1  0  0
   2 10 42  0 12  3  1  0  4  9  0 10  0  0]
 [  0  0  0  1  1  0  0  4 1983  0  0  0  0  0  0
   0  2  7  0  3  0  0  0  0  3  0  0  0  0  0]
 [  0  0  0  0  0  0  0  0  0 167  0  0  0  0  0
   0  0  0  0  2  0  0  1  1  0  0  1  0  0  0]
 [  0  0  0  0  0  0  0  0  0  1 69  0  0  0  0
   0  0  1  0  1  2  0  0  1  2  0  1  0  0  0]
 [  0  0  0  0  0  0  0  0  0  0  0 262  0  0  0]
```

```

1 0 1 0 0 0 0 0 0 0 0 1 0]
[ 1 0 0 0 0 0 0 1 0 0 0 0 55 0
0 0 0 0 0 0 0 0 0 2 0 0 0]
[ 0 0 0 0 0 0 0 0 1 0 0 0 0 91
0 0 0 0 0 0 0 0 0 1 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
270 0 2 0 0 0 0 1 0 0 0 0 0]
[ 0 0 0 0 2 0 1 5 2 0 0 0 0 0
0 2313 7 0 6 0 0 1 1 5 0 4 0]
[ 3 0 0 0 2 0 0 26 3 1 0 0 1 0
0 6 3130 0 10 3 2 0 0 5 1 4 1]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
0 0 1 474 0 0 0 0 1 0 0 1 0]
[ 1 2 1 0 4 0 1 8 2 1 0 1 0 0
0 2 14 0 2457 2 1 0 1 4 1 12 1]
[ 2 0 0 1 0 0 1 0 1 0 0 0 0 0
0 2 2 0 2 351 0 0 0 2 0 1 0]
[ 0 0 0 0 0 0 0 2 0 0 0 0 0 0
0 0 3 0 0 0 216 0 0 1 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 3 0 0 388 0 0 0 0 0]
[ 1 0 0 0 0 0 0 3 0 0 0 0 0 0
0 3 5 0 1 0 1 0 782 3 0 0 0]
[ 0 0 0 0 0 0 0 8 2 0 0 0 0 1
0 5 12 0 6 0 0 1 3 1738 0 3 1]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 3 119 1 0]
[ 1 0 0 0 2 0 2 7 1 0 0 1 0 1
2 8 7 1 3 0 0 0 2 3 0 2111 0]
[ 0 0 0 1 0 0 0 0 1 0 0 0 0 0
0 1 3 0 1 0 0 0 0 0 0 2 58]]

```

Matriz de confusión guardada en D:/Angelica Moreno/Proyecto de grado/Modelo/BERT-Si-Balanceo-Si-VC/2epocasVC2/confusion_matrix_fold_4.csv

Informe de Clasificación:

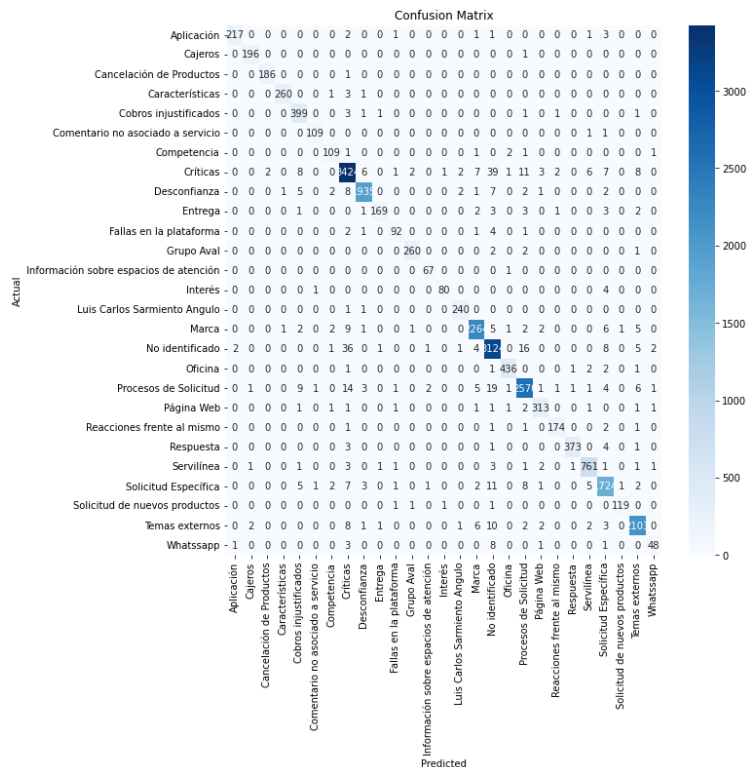
	precision	recall	f1-score	support
Aplicación	0.95	0.99	0.97	228
Cajeros	0.98	1.00	0.99	171
Cancelación de Productos		0.99	0.99	0.99 172
Características	0.98	0.97	0.98	275
Cobros injustificados	0.96	0.96	0.96	400
Comentario no asociado a servicio		0.99	0.92	0.95 90
Competencia	0.92	0.92	0.92	95
Críticas	0.98	0.97	0.97	3488
Desconfianza	0.99	0.99	0.99	2004

Entrega	0.98	0.97	0.98	172	
Fallas en la plataforma	1.00	0.88	0.94	78	
Grupo Aval	0.99	0.99	0.99	265	
Información sobre espacios de atención		0.95	0.93	0.94	59
Interés	0.97	0.98	0.97	93	
Luis Carlos Sarmiento Angulo		0.98	0.99	0.99	273
Marca	0.98	0.99	0.98	2347	
No identificado	0.96	0.98	0.97	3198	
Oficina	1.00	0.99	0.99	479	
Procesos de Solicitud	0.98	0.98	0.98	2516	
Página Web	0.97	0.96	0.97	365	
Reacciones frente al mismo		0.98	0.97	0.98	222
Respuesta	0.99	0.99	0.99	391	
Servilínea	0.98	0.98	0.98	799	
Solicitud Específica	0.97	0.98	0.97	1780	
Solicitud de nuevos productos		0.98	0.97	0.98	123
Temas externos	0.98	0.98	0.98	2152	
Whatsapp	0.95	0.87	0.91	67	
accuracy			0.98	22302	
macro avg	0.97	0.97	0.97	22302	
weighted avg	0.98	0.98	0.98	22302	

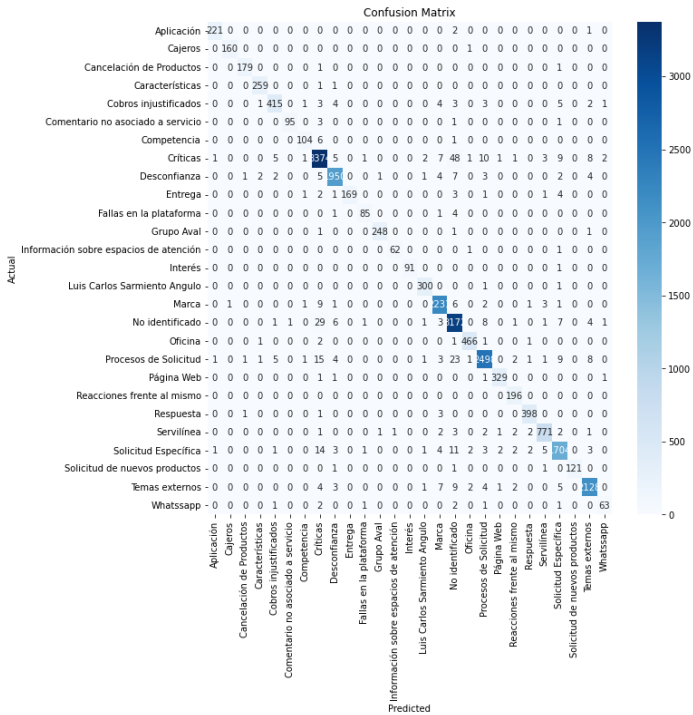
<Figure size 432x288 with 0 Axes>

Durante el proceso de entrenamiento y evaluación del modelo BERT para el análisis de sentimiento, se generaron múltiples matrices de confusión y curvas ROC para cada pliegue de la validación cruzada. Las matrices de confusión revelaron un alto nivel de precisión en la clasificación de sentimientos negativos, neutros y positivos, con la clase 'Negativo' mostrando una precisión y *recall* particularmente altos en todos los pliegues. Las curvas ROC demostraron la capacidad del modelo para distinguir entre las diferentes clases con un área bajo la curva (AUC) elevada para cada clase, confirmando la robustez del modelo. Estas métricas indican que el modelo BERT es eficaz y fiable para la tarea de análisis de sentimiento, mostrando un buen balance entre precisión y *recall* y una fuerte capacidad de discriminación entre las clases.

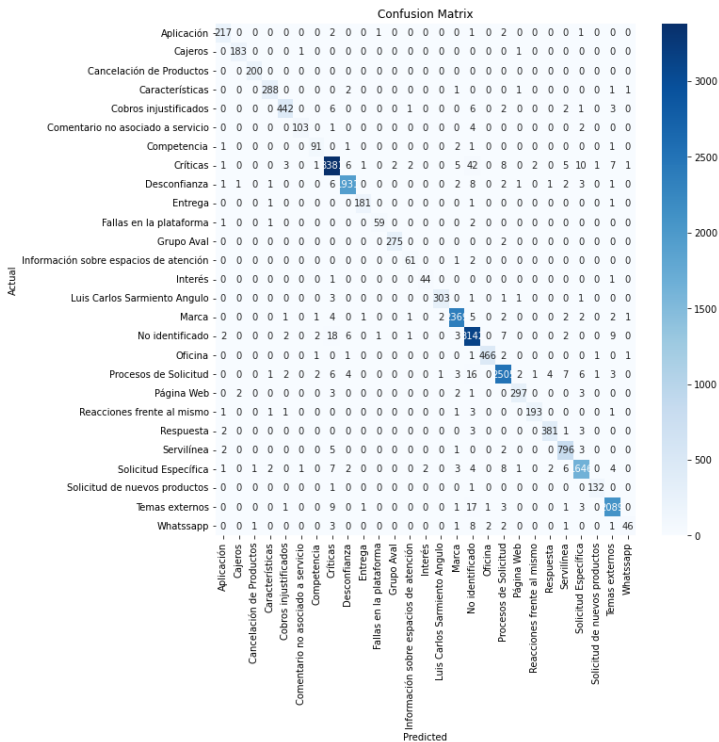
Confusion_Matrix_Fold_0



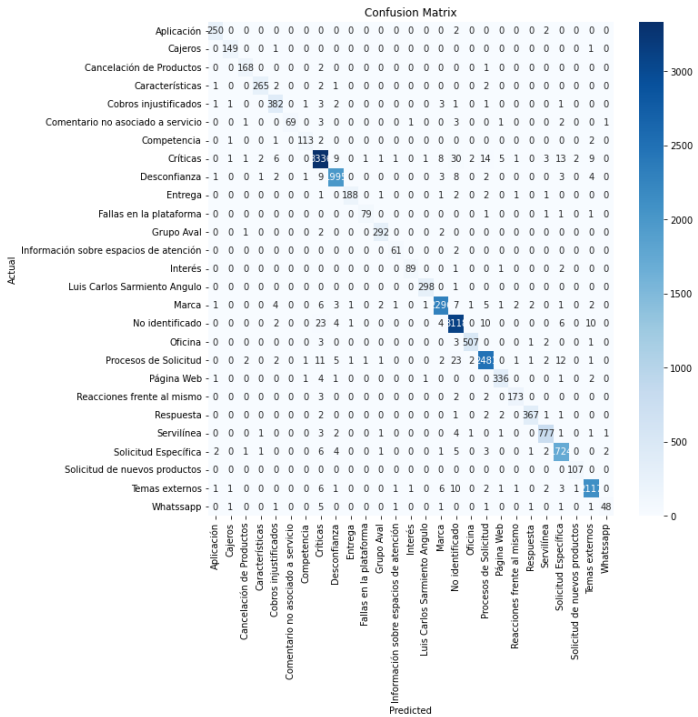
Confusion_Matrix_Fold_1



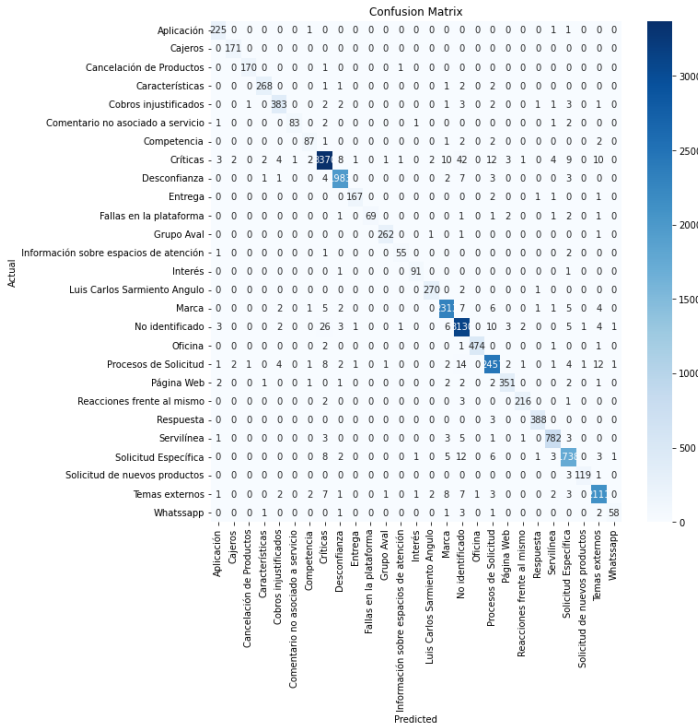
Confusion_Matrix_Fold_2



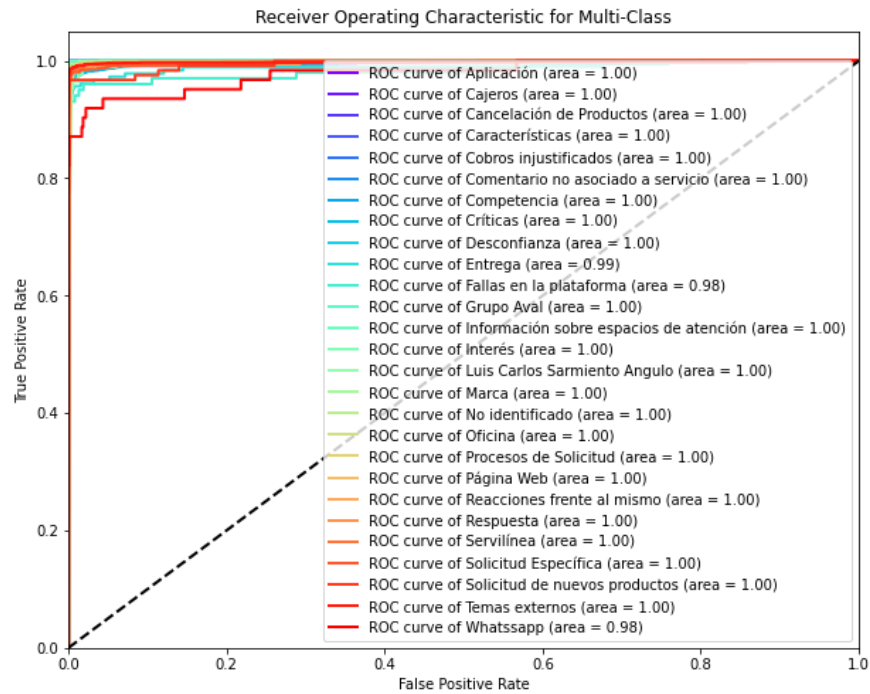
Confusion_Matrix_Fold_3



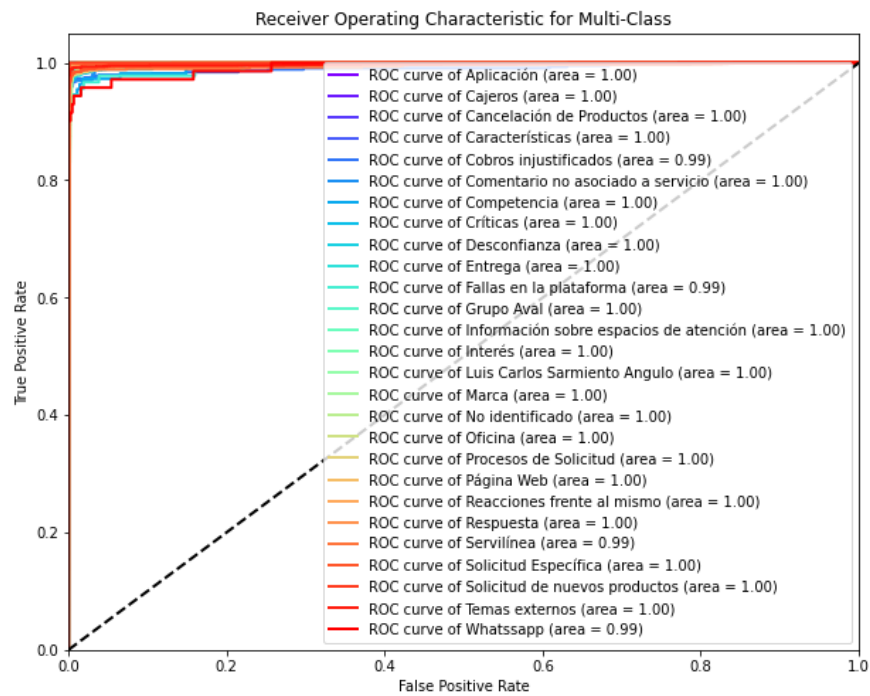
Confusion_Matrix_Fold_4



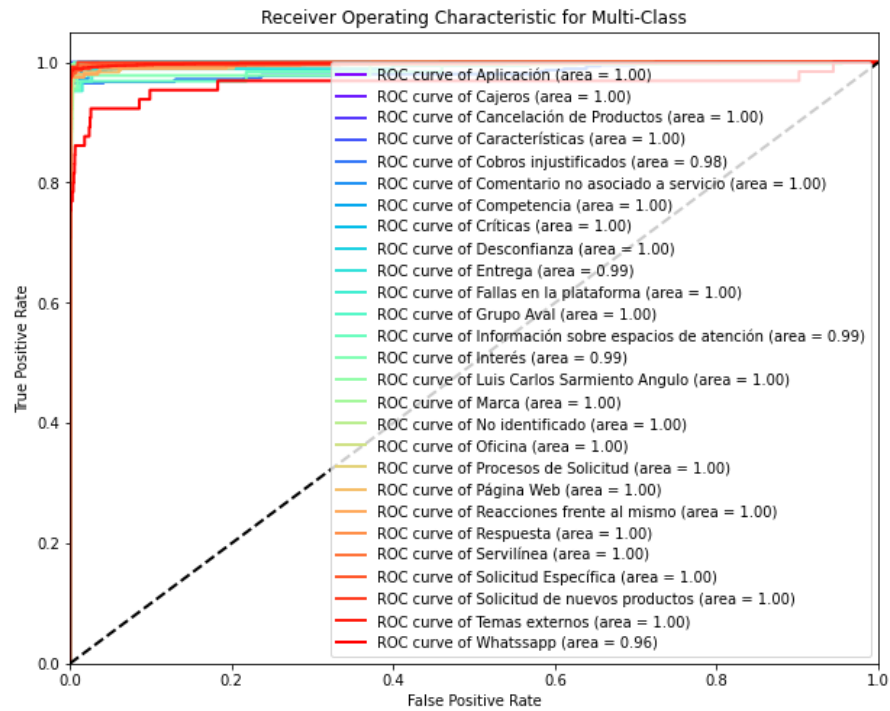
ROC_Curves_Fold_0



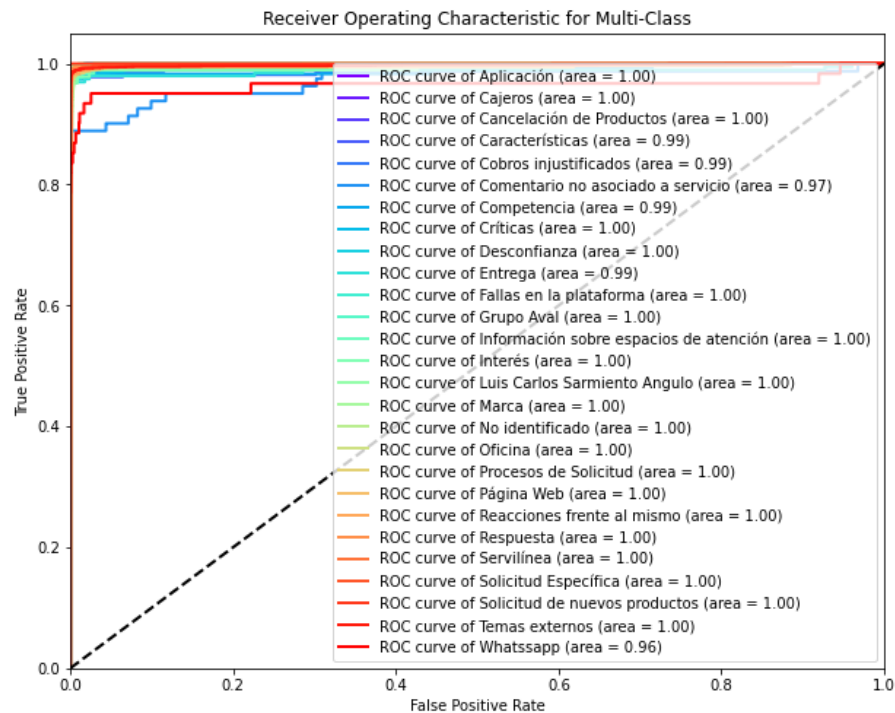
ROC_Curves_Fold_1



ROC_Curves_Fold_2



ROC_Curves_Fold_3



ROC_Curves_Fold_4

